Netcool Impact Lab Guide

The guide has been split into section files in the `guides/` directory.

- 1. Impact fundamentals and UI navigation
- 2. Impact data source adapters (DSAs)
- 3. Impact policy language (IPL)
- 4. Event enrichment
- 5. Scheduled automation
- 6. Event escalation
- 7. Automation and hibernation
- 8. Making a Java call
- 9. Using the Eval function
- 10. Organizing artifacts using projects
- 11. Granting users access to Impact
- Addendum. Managing policies and services
- Lab completion summary

1. Impact fundamentals and UI navigation

1.1 Netcool Impact overview

IBM Netcool Impact is an event enrichment and automation engine that extends the capabilities of Netcool/OMNIbus. While OMNIbus collects and consolidates events from across the IT environment, Impact adds intelligence to those events by retrieving external data, running policy-driven automation, and writing results back to the ObjectServer or to other target systems.

The core value of Impact is that it can reach outside the event stream and pull in contextual information that the probe or ObjectServer cannot provide on its own. A probe receives a raw event from a device and populates standard fields such as `Node`, `Summary`, and `Severity`. Impact can then look up that node in a CMDB, a SQL database, or a REST API, discover the customer it belongs to, the service it supports, and the on-call engineer responsible for it, and write all of that context back to the alert. Operators see enriched, actionable alerts rather than raw device events.

Beyond enrichment, Impact can perform automated actions: opening or updating tickets in ServiceNow or Remedy, running runbooks, sending notifications, suppressing known maintenance events, and escalating unacknowledged alerts based on time-based rules. These actions are driven by policies — units of IPL (Impact Policy Language) logic that define what to do and when.

1.2 Core architecture and components

The Impact architecture is built around a central server that runs policies against event data and connects to external systems through a configurable set of adapters.

Key architecture components:

- **Impact Server**: The core processing engine that receives event triggers from OMNIbus, runs IPL policies, and communicates with external data sources.
- **Impact GUI**: The browser-based administration interface used to create and manage data source adapters (DSAs), services, and policies.
- **ObjectServer DSA**: A built-in adapter that connects Impact to the OMNIbus ObjectServer, allowing policies to read event data and write enriched values back to alerts.
- **Data source adapters (DSAs)**: Named connectors that give policies access to external systems such as SQL databases, REST APIs, LDAP directories, and JDBC data sources.
- **Services**: Named entry points that define when and how policies are triggered — for example, in response to ObjectServer events, on a schedule, or by an external call.
- **Policies**: Units of IPL logic that perform the actual enrichment and automation work when invoked by a service.
- **Impact Policy Language (IPL)**: The scripting language used to write policy logic. IPL provides variables, conditionals, loops, data source lookups, and built-in functions for working with event data.

1.3 How Impact fits into the OMNIbus architecture

Impact does not replace OMNIbus — it extends it. In a typical deployment, OMNIbus probes continue to collect events and store them in the ObjectServer as they always have. Impact sits alongside the ObjectServer and subscribes to changes in the event stream through a service connected to the ObjectServer DSA.

When a new event arrives in the ObjectServer and matches the service filter, Impact receives a copy of the event data and begins policy processing. The policy can query external systems, perform calculations, and then write results back to the ObjectServer by updating fields in `alerts.status`. From the operator's perspective, the enriched fields simply appear on the alert as though the probe had populated them from the start.

This separation of concerns is important: probes handle ingestion and normalization, the ObjectServer handles storage and deduplication, and Impact handles enrichment and automation. Each component focuses on its own job, and they work together through well-defined interfaces.

1.4 Key terms and concepts

The following terms appear throughout the Impact lab guide and should be understood before beginning the hands-on exercises:

- **Policy**: A named unit of IPL code that performs enrichment or automation logic when invoked by a service.
- **Service**: An Impact configuration object that defines when policies run and what event data they receive.
- **DSA (Data Source Adapter)**: A named connector that provides policies with access to an external data source.
- **IPL (Impact Policy Language)**: The scripting language used to write Impact policy logic.
- **Event reader**: A service type that subscribes to the ObjectServer event stream and triggers policies when events match a defined filter.
- **Event trigger**: The event data object passed to a policy when a service fires. It contains the field values of the triggering alert.
- **GetByFilter**: An IPL function that queries a data source and returns rows matching a specified condition.
- **GetByKey**: An IPL function that queries a data source and returns rows matching a unique key value.
- **Log**: An IPL function that writes a message to the Impact server log for debugging or auditing purposes.

1.5 Lab environment and assumptions

This lab environment uses the same virtual machine as the OMNIbus labs. Netcool Impact is installed on the lab server alongside OMNIbus and is accessible through a web browser. The lab server runs both the OMNIbus ObjectServer and the Impact Server.

The lab virtual machines include the following pre-installed components relevant to Impact:

- IBM Netcool/Impact server.
- The Impact GUI, accessible through a browser on port 17311 at `http://<your-server>:17311/ibm/console`.
- The OMNIbus ObjectServer (EVENTS), which Impact uses as its primary event source.
- A sample SQL database pre-loaded with device inventory and customer records used in the enrichment labs.

You should be comfortable with the OMNIbus ObjectServer from the previous lab chapters. Impact builds directly on that knowledge — the same events, the same alert fields, and the same `alerts.status` table are used throughout the Impact exercises.

1.6 UI navigation overview

Objective In this exercise, you will log in to the Netcool Impact GUI, explore the main navigation areas, and become familiar with the layout and purpose of each section. By the end of the exercise, you will be able to navigate to the Data Model panel, the Services panel, and the Policy editor, and understand what each section of the interface is used for.

Prerequisites

- SSH access to the assigned lab server.
- The OMNIbus ObjectServer (EVENTS) is running (completed in the OMNIbus labs).
- A web browser is available on the workstation.

Background The Impact GUI is a web-based administration interface that provides a single location for all Impact configuration tasks. It is organized into several panels that map directly to Impact concepts: data sources, services, and policies are each managed in their own dedicated section.

The Impact GUI is organized around a top navigation bar with tabs. Clicking a tab loads its management panel in the main workspace, with a list pane on the left and a configuration detail area on the right. The primary tabs are:

- **Data Model**: Create and manage DSAs that connect policies to external systems.
- **Policies**: Access the policy editor, where IPL code is written, saved, and organized by policy group.
- **Services**: Manage services that define when and how policies are invoked.
- **Operator View**: A lightweight event list view pulled directly from the connected ObjectServer, useful for verifying that enrichment results are appearing on alerts.

- **Maintenance Window**: Configure maintenance windows for suppression of alerts during planned outages.
- **Reports**: View operational reports and policy execution statistics.

Tasks

1. Connect to the lab server by using SSH as `root`.
2. Set the required Impact environment variables if they are not already set from the OMNIbus labs:

   ```
   export NCHOME=/opt/IBM/tivoli/netcool
   export OMNIHOME=$NCHOME/omnibus
   export IMPACT_HOME=/opt/IBM/tivoli/impact
   ```

   To persist these values across new shell logins, add them to your `~/.bash_profile` and source it:

   ```
   echo 'export IMPACT_HOME=/opt/IBM/tivoli/impact' >> ~/.bash_profile
   source ~/.bash_profile
   ```

   The `NCHOME` and `OMNIHOME` variables should already be present from the OMNIbus labs. If they are not, add them as well:

   ```
   echo 'export NCHOME=/opt/IBM/tivoli/netcool' >> ~/.bash_profile
   echo 'export OMNIHOME=$NCHOME/omnibus' >> ~/.bash_profile
   source ~/.bash_profile
   ```

3. Verify that all three environment variables are set correctly:

   ```
   echo $IMPACT_HOME
   echo $NCHOME
   echo $OMNIHOME
   ```

   You should see:

   ```
   /opt/IBM/tivoli/impact
   /opt/IBM/tivoli/netcool
   /opt/IBM/tivoli/netcool/omnibus
   ```

   If any variable prints as blank, return to step 2 and ensure the relevant line was added to `~/.bash_profile` and sourced correctly before continuing.

4. Before starting the Impact Server, verify that the OMNIbus ObjectServer EVENTS is running:

   ```
   $OMNIHOME/bin/nco_ping EVENTS
   ```

   Confirm that the output includes: `NCO_PING: Server available.`

   If the ObjectServer is not running, start it before proceeding:

   ```
   $OMNIHOME/bin/nco_objserv -name EVENTS &
   ```

   Then re-run `nco_ping` to confirm it is available before continuing.

5. Start the Impact Server:

   ```
   $IMPACT_HOME/bin/startImpactServer.sh
   ```

   The command outputs startup progress directly to the terminal. Confirm that the output includes the following lines before proceeding:

   ```
   Starting server NCI.
   Server NCI started with process ID 2402167.
   ```

   The process ID in your output will differ — this is expected. What matters is that both lines appear, confirming the server started successfully.

6. Start the Impact GUI server:

   ```
   $IMPACT_HOME/bin/startGUIServer.sh
   ```

Confirm that the output includes the following lines before proceeding:

```
Starting server ImpactUI.
Server ImpactUI started with process ID 2403600.
```

The process ID in your output will differ — this is expected. What matters is that both lines appear, confirming the GUI server started successfully.

7. Open a browser and navigate to the Impact GUI: `http://<your-server>:17311/ibm/console` Log in using the Impact administrator credentials for this lab:
   - **User**: `impactadmin`
   - **Password**: `netcool`



Figure 1: Impact GUI login screen

8. After successfully logging in, you will be presented with the main landing page for Netcool Impact.

   Navigating Netcool/Impact

   Use the navigation controls at the top of the IBM Tivoli Netcool/Impact window to move between functional areas.

   - Select **Welcome** to return to the starting page. This page provides large shortcut icons to common areas of the product.
   - Select **Data Model** to work with data types, data sources, and related model objects.
   - Select **Policies** to view, create, or manage Impact policies.
   - Select **Services** to access configured services and service-related functions.
   - Select **Operator View** to open the operator-facing event and service views.
   - Select **Maintenance Window** to configure or review scheduled maintenance periods.

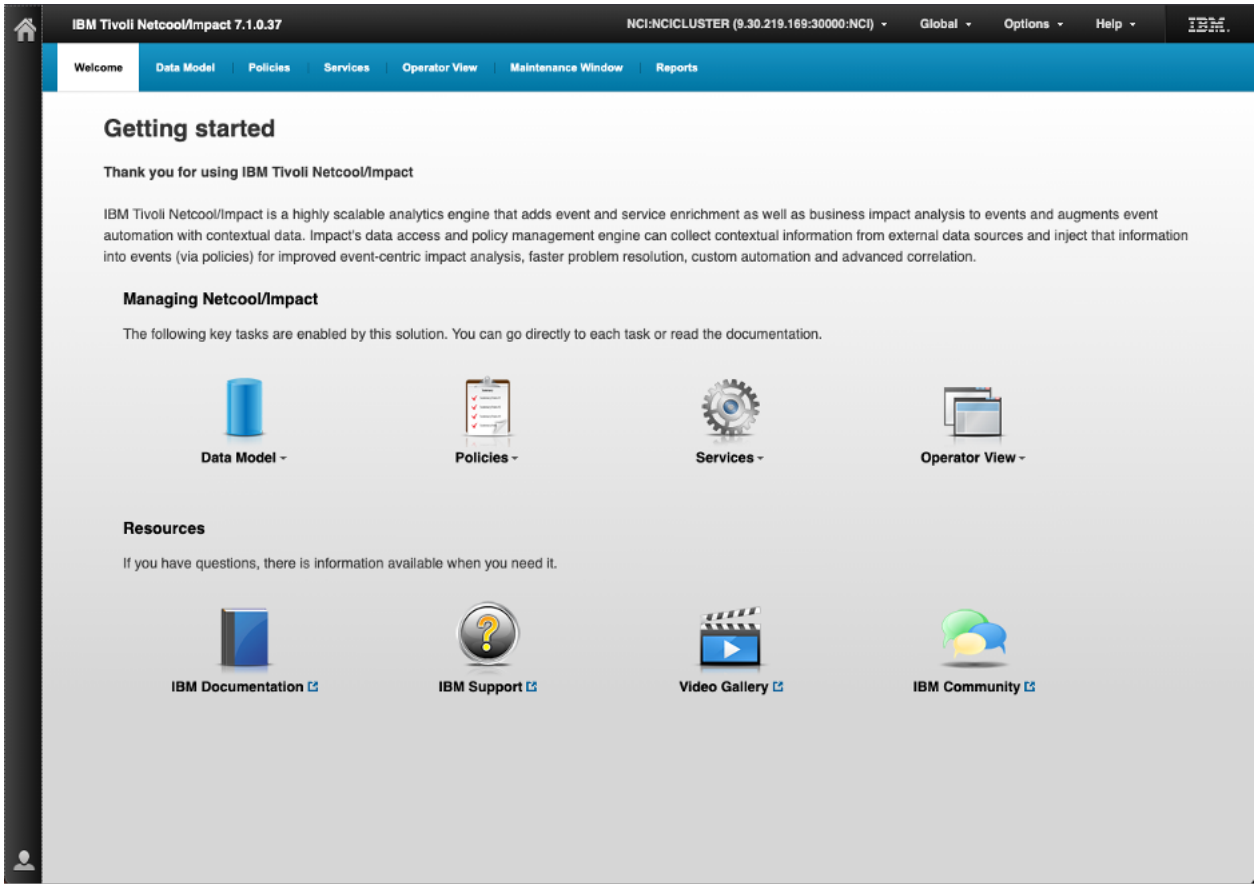


Figure 2: Impact GUI dashboard after login

- Select **Reports** to access reporting functions.
- Use the drop-down arrows next to the large **Data Model**, **Policies**, **Services**, and **Operator View** icons on the Welcome page to open shortcuts within those areas.

The black bar above the main navigation provides global controls:

- Use the **Home** icon at upper left to return to the main starting point.
- Use the **NCI** connection drop-down to identify or change the active Netcool/Impact server connection.
- Use **Global** for global settings or actions.
- Use **Options** for user or interface options.
- Use **Help** to access product help and documentation.

At the bottom of the Welcome page, use the **Resources** links to open IBM Documentation, IBM Support, Video Gallery, or IBM Community resources.

9. Select the **Data Model** tab from the top navigation bar. The Data Model area is where data types and data sources are defined and managed. Observe the available sub-sections and note that this is where DSAs are configured for use by Impact policies. Return to the **Welcome** page when done by selecting **Welcome** from the top navigation bar.

   The left navigation pane lists configured data sources, while the workspace on the right remains blank until you select an item.

   A **data source** is the connection or integration that gives Netcool/Impact access to external or internal information — for example, an ObjectServer, database, ITNM, web service, or URL. A **data type** is the defined structure or view of records exposed through that source, including the fields Impact policies can retrieve, evaluate, or update.

   Visible data sources include `EIC_alertsdb`, `EventrulesDB`, `ITNM`, `ImpactDB`, `Internal`, `Schedule`, `Slack`, `URL`, and several ObjectServer-related DSAs. Expand a data source with the arrow beside it to display its data types. We will explore data sources, DSAs, and data types in more detail in later tasks.

10. Next, select the **Policies** tab from the top navigation bar.

    The screen is divided into two main areas:

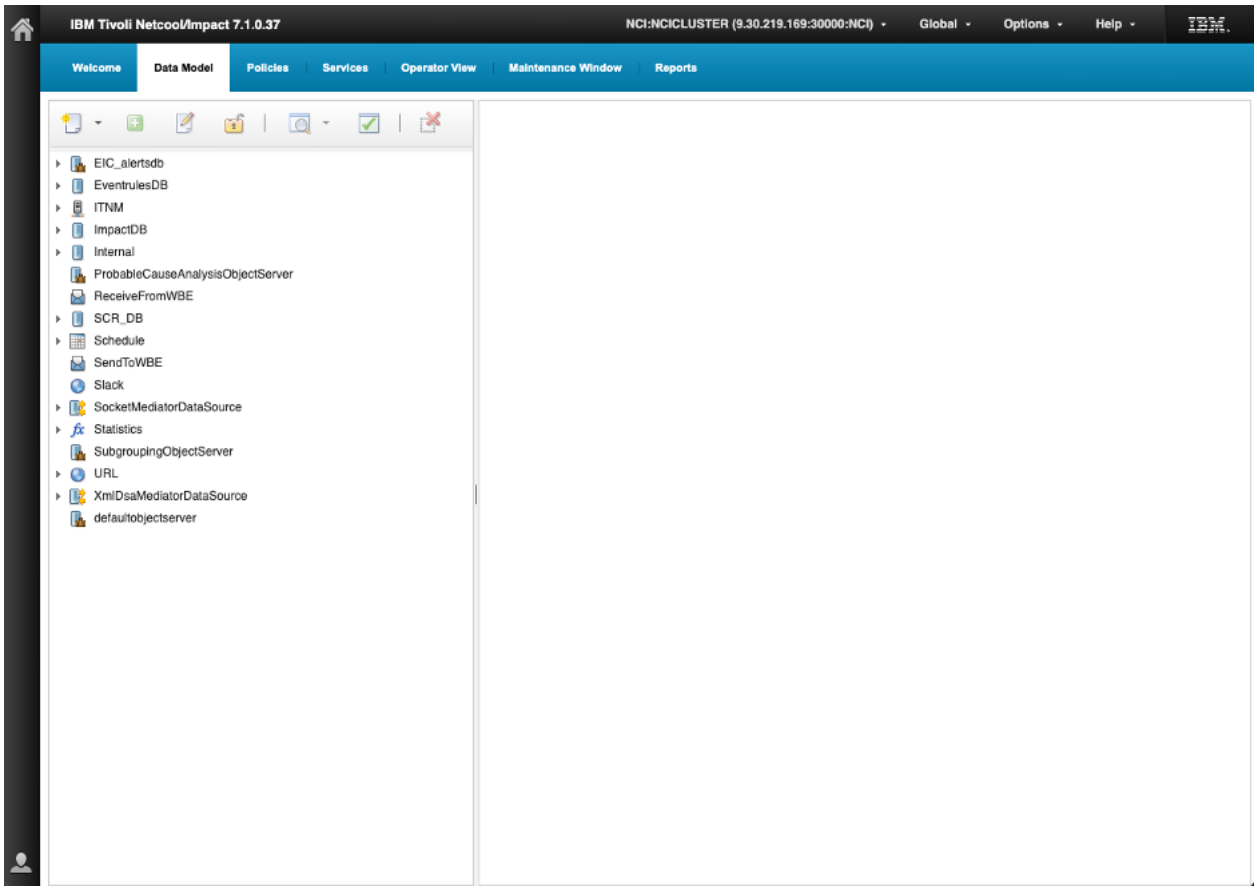


Figure 3: Impact Data Model tab

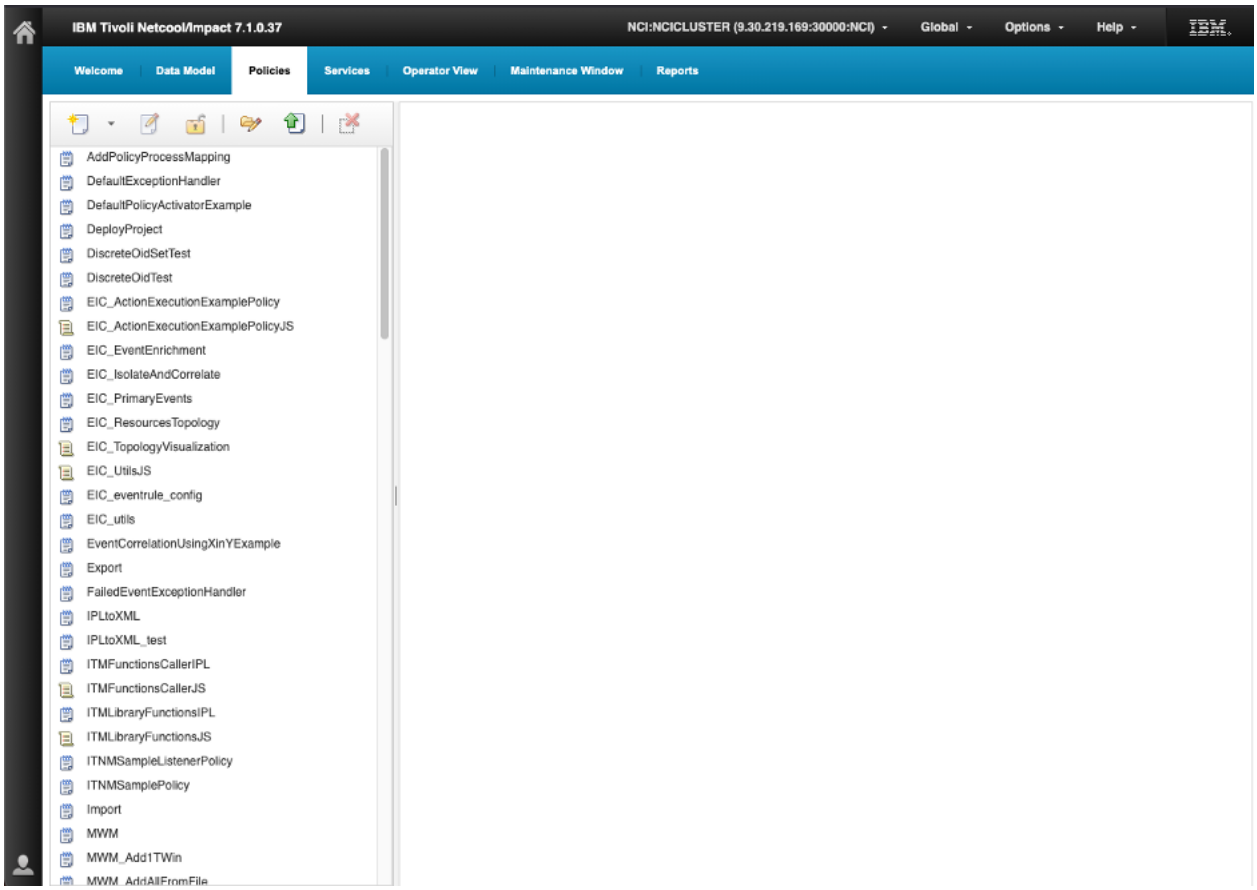


Figure 4: Impact Policies section

The left pane contains a scrollable list of the policies currently available in Impact. You can see policies such as `AddPolicyProcessMapping`, `DefaultExceptionHandler`, various `EIC_...` policies, and other sample or utility policies. The icons identify the policy object or format, such as standard policies, JavaScript policies, or folders.

The toolbar above the policy list provides actions for working with policies, including creating, opening, saving, importing or exporting, and deleting policy objects.

The workspace on the right is blank until you select or open a policy. When you open one, its configuration and policy logic are displayed here for review or editing.

You will work with Impact policies in more detail in later tasks.

11. Next, select the **Services** tab from the top navigation bar.

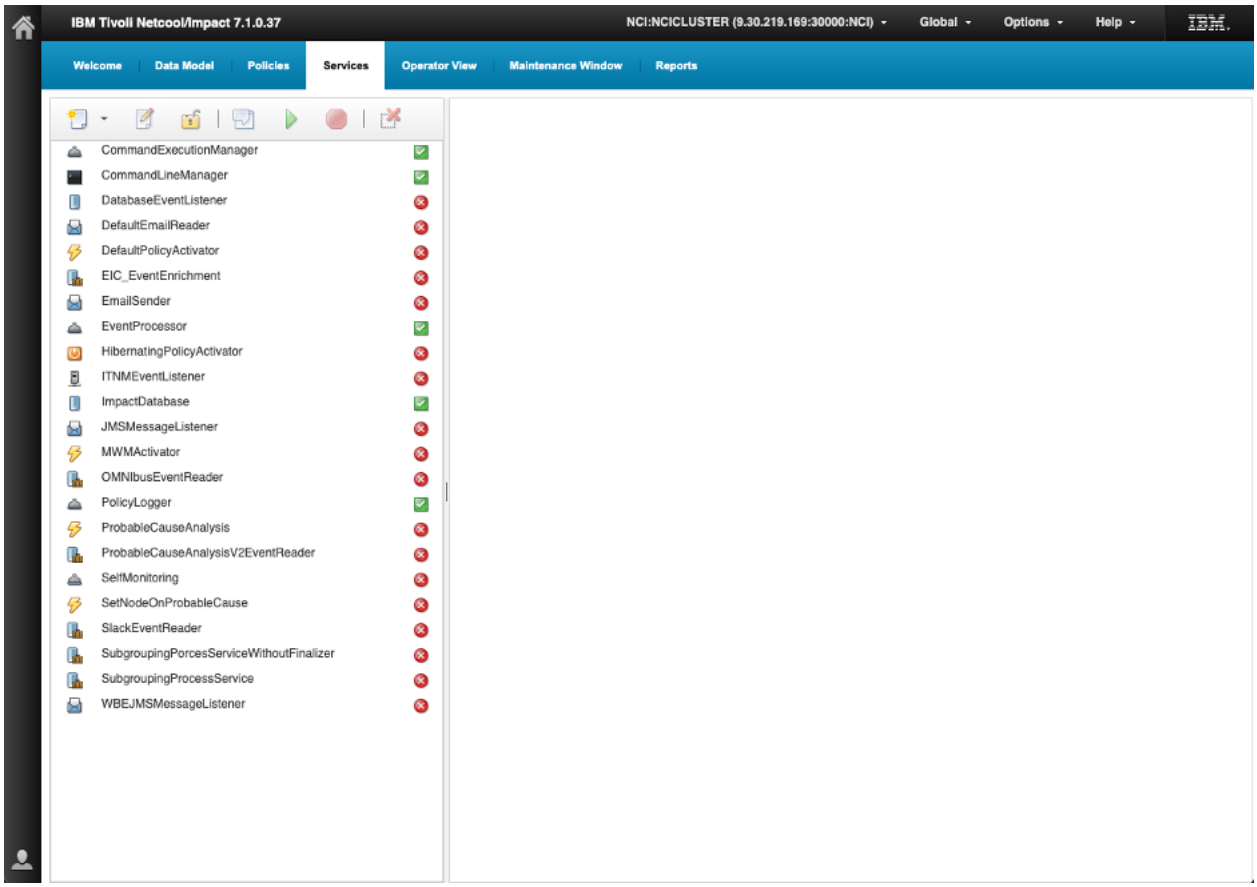


Figure 5: Impact Services tab

Select the **Services** tab to view and manage the Netcool/Impact services configured in the environment. The left pane lists the available services, and the status icon beside each service indicates whether it is running: a green check mark shows the service is active, while a red X shows it is stopped or unavailable. Select a service to view its details in the workspace on the right.

The service types shown include:

- **Managers** — Coordinate and manage a related service function, such as command execution or database activity.
- **Listeners and readers** — Receive or read information from external systems, for example database events, JMS messages, emails, or ITNM events.
- **Activators** — Trigger policy execution when a defined event or condition occurs.
- **Processors** — Process incoming event data and invoke appropriate Impact logic.
- **Senders** — Send information or notifications to another system, such as email.
- **Policy Logger** — Records policy activity and messages for monitoring and troubleshooting.
- **Specialized services** — Provide functions for specific integrations or workflows, such as event enrichment, probable-cause analysis, self-monitoring, Slack event reading, or event sub-grouping.

The toolbar above the list lets you work with the selected service, including starting or stopping it.

12. Select the **Operator View** tab to view and manage the operator views available in Netcool/Impact.

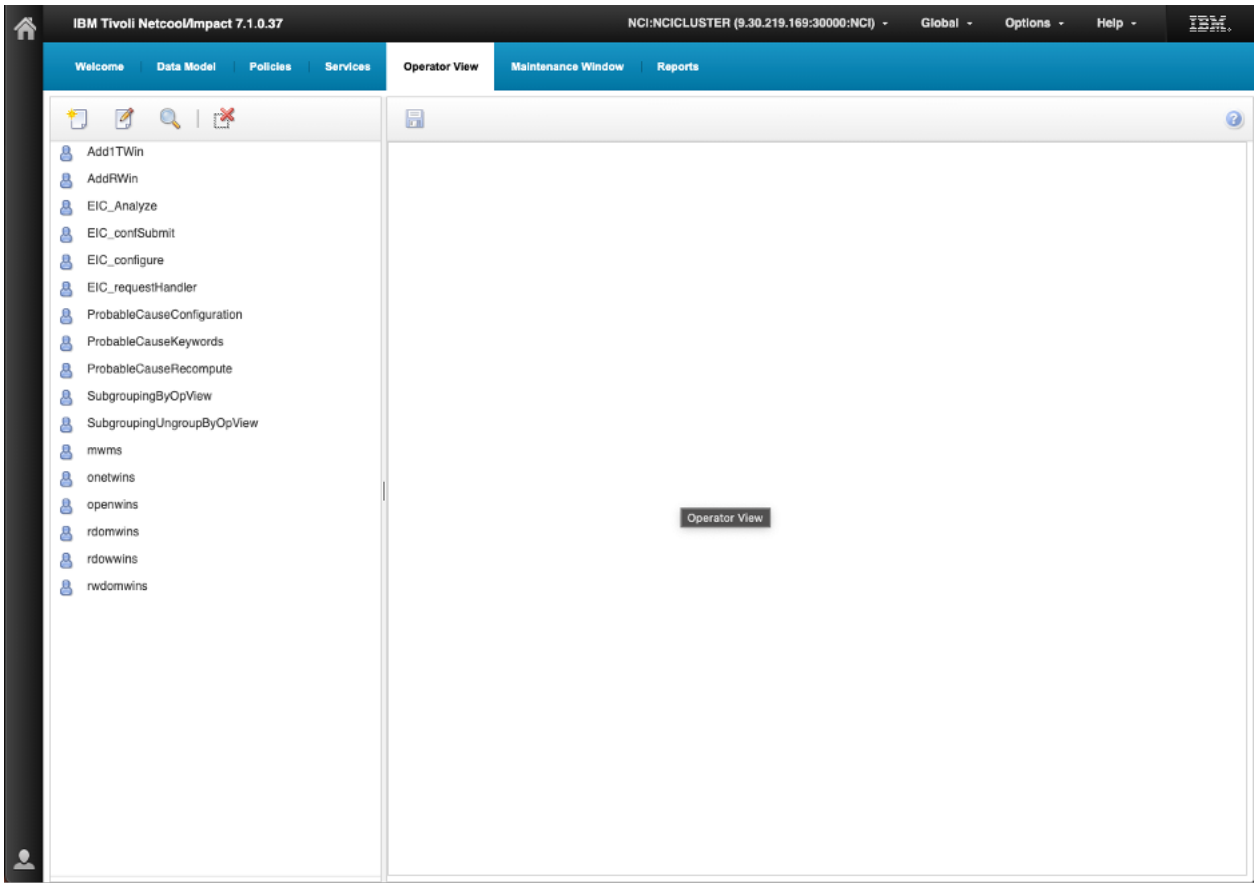


Figure 6: Impact Operator View tab

Operator views provide tailored interfaces that let operators interact with Impact functions, such as initiating workflows, submitting requests, viewing event-related information, or running specific operational actions.

Select an operator view from the list to display its configuration in the workspace on the right. The toolbar above the list lets you create, edit, search for, or delete operator views.

13. Select the **Maintenance Window** tab from the top navigation bar.

    Maintenance windows let you identify events that occur during approved maintenance so that they can be handled differently from unexpected operational events — for example, placed into maintenance rather than treated as active incidents.

    The page opens on **One Time Windows**, which lists maintenance windows that apply for a single defined start and end time. Use the other tabs to manage recurring windows by **Day of Week**, **Weekly**, **Day of Month**, or **Nth Day of Week in Month**.

    The table displays each window's **Description**, event **Filter**, **Start Time**, **End Time**, **Time Zone**, and current **Status**. The status indicators identify windows that are **Active**, **Inactive**, **Expired**, or **Future**. Use **Add New** to define a maintenance window, and use the search field to locate an existing one.

    Netcool/Impact Maintenance Window Management marks a matching event as in maintenance by setting the ObjectServer `alerts.status.SuppressEscl` integer field to 6. This is the field that downstream Event List views, automations, and Impact policies can use to identify and suppress maintenance-related alerts.

    By default, MWM's Policy Activator evaluates active windows about every seven seconds, so an event can briefly appear normally before `SuppressEscl` is updated to 6. When the window ends, MWM can clear the maintenance flag, unless its `clearFlag` setting is configured not to mark events as out of maintenance.

Validation The exercise is successful when all of the following are true:

- The Impact Server and GUI server start successfully.
- You can log in to the Impact GUI and reach the dashboard.
- You can navigate to the **Policies**, **Services**, and **Data Model** tabs using the top navigation bar.

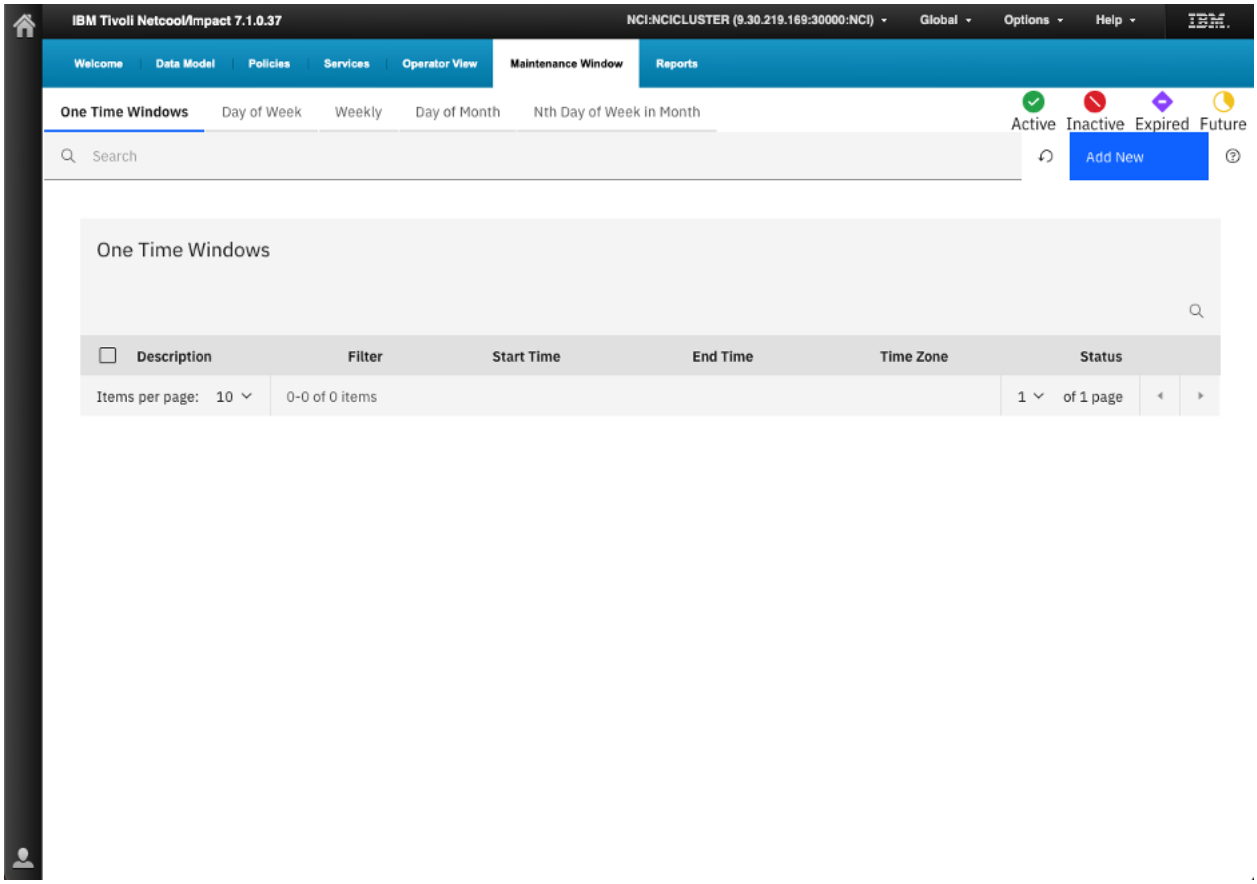


Figure 7: Impact Maintenance Window tab

- You can describe the purpose of each main navigation tab.
- You can identify any pre-configured DSAs in the Data Model panel.
- You can open the policy editor and identify the key input areas without saving a policy.
- You can access the **Operator View** tab and confirm it is displaying current ObjectServer alerts.

Expected result At the end of this exercise, you will be familiar with the Impact GUI layout and will be able to navigate confidently between the main tabs. This provides the foundation for all subsequent exercises, which use the Data Model panel, the Services panel, and the Policy editor to build working Impact configurations.

10. Organizing Impact artifacts using projects

As an Impact environment grows, the set of policies, services, and data types that make it work becomes a significant asset in its own right. Impact **projects** give you a way to organize that set of artifacts into named scopes, keeping related configuration together and making it straightforward to work on one area of the system without the clutter of everything else. Projects also serve as the unit of export and import — packaging a project produces a ZIP archive that can be transferred to another server, stored as a backup, or used to promote configuration from a test environment to production.

Every Impact server has a built-in scope called **Global**. Policies, services, and data types that are not assigned to a named project live in Global and are always visible regardless of which project is selected. All of the artifacts created in this lab currently live in Global.

This section walks through the full project lifecycle: understanding how the project selector works, creating a project and populating it with the lab artifacts, and exporting and re-importing the project to understand how the import process handles existing artifacts.

10.1 Create a project and add lab artifacts

Objective In this exercise, you will open the project selector, create a new project, and populate it with all of the data sources, data types, policies, and services from this lab. By the end of the exercise, you will understand how the project selector scopes the Policies and Services views and how to build a project that captures a complete lab configuration.

Prerequisites

- Completion of sections 4 through 9.
- All lab policies and services are present: `AA_EnrichAlert`, `AA_DeferredEnrichment`, `AA_EscalatePlatinum`, `AA_PingVerify`, `AA_JavaSortDemo`, `AA_EvalActionPolicy`.
- You are logged in to the Impact GUI.

Background

The project selector The project selector is a dropdown in the top-right area of the Impact GUI header bar, immediately to the right of the server name (for example, `NCI:NCICLUSTER (192.168.2.67:30000:NCI)`). It is labelled with the name of the currently active project — **Global** by default.

Clicking the dropdown opens a two-panel menu:

- **Select Project** (left panel): lists all projects defined on this server, including **Global**. Click any entry to switch the active scope. The currently active project is shown in bold.
- **Manage Projects** (right panel): provides the available actions.

Action	Purpose
Create Project	Opens the Create Project dialog
Edit Current Project	Opens the current project's configuration for export or import
Delete Current Project	Permanently removes the current project and all its artifact references (the artifacts themselves are not deleted)
Refresh	Refreshes the project list
Clear my locks	Releases any editor locks held by your current session

What switching projects does When you switch to a named project, the **Policies** and **Services** tabs show only the artifacts assigned to that project. This is a view filter — artifacts in other projects or in Global are not deleted; they are simply not displayed in the current scope. Switching back to **Global** shows all artifacts on the server.

What a project contains A project holds references to policies, services, operator views, data sources, and data types. The underlying artifacts remain on the server; the project records which ones belong to it. Removing an artifact from a project removes the reference only — the artifact itself is unaffected.

A project export/import includes the Impact objects associated with the project, including data-source definitions, data types, policies, services, and project metadata. After import, validate and, where necessary, update environment-specific data-source settings — such as hostnames, ports, database names, credentials, certificates, driver files, filesystem paths, and external endpoints — before enabling services in the target environment. The target Impact server must also have the required DSA implementation, drivers, and supporting configuration installed.

The Create Project dialog The **Create Project** dialog is a single page with:

- **Project Name**: the name for the new project.
- **List By**: a dropdown that switches which type of artifact is shown in the picker below. Available options include **Services**, **Policies**, **Data Types**, **Data Sources**, and others.
- **Available Members** (left column): all artifacts of the selected type that are not yet in the project. Click an item to move it to **Project Members**.
- **Project Members** (right column): artifacts assigned to the project. Click an item to move it back to **Available Members**.
- **Execute**: creates the project with the current member list.
- **Close**: cancels without creating the project.

You work through each member type in turn using the **List By** dropdown, moving items into **Project Members** before clicking **Execute**.

Tasks

1. Locate the project selector in the top-right of the Impact GUI header bar. It currently reads **Global**.
2. Click the **Global** dropdown to open the project selector menu. Take a moment to observe the layout:

- The **Select Project** panel on the left lists the available projects. **Global** is shown in bold as the active project.
- The **Manage Projects** panel on the right lists the available actions.

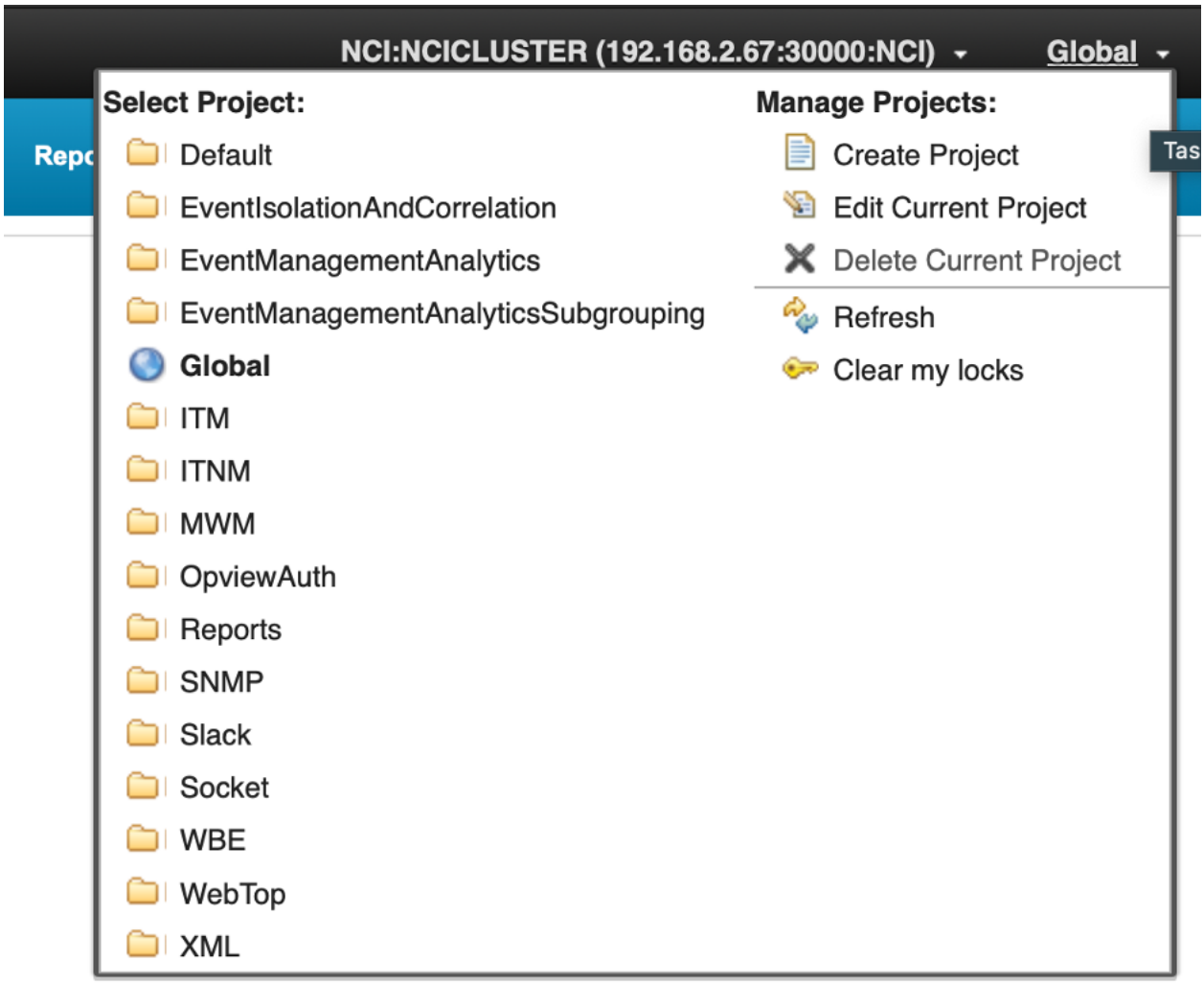


Figure 8: The project selector dropdown showing the Select Project list on the left and the Manage Projects actions on the right, with Global shown as the active project.

3. Click **Create Project** in the **Manage Projects** panel.

   The Create Project dialog opens. Enter `ImpactLabProject` in the **Project Name** field.

4. Use the **List By** dropdown to select **Data Sources**. The **Available Members** list will populate with all data sources defined on this server. Move the following items to **Project Members** by clicking each one:

   - `CMDB`
   - `defaultobjectserver`

5. Use the **List By** dropdown to select **Data Types**. A **Data Source** dropdown appears below it to filter which data source's types are listed.

   First, select `CMDB` in the **Data Source** dropdown. Move the following items to **Project Members**:

   - `DEVICE_INVENTORY`
   - `TAKEACTION`

   Then select `defaultobjectserver` in the **Data Source** dropdown. Move the following item to **Project Members**:

   - `EVENTS`

6. Use the **List By** dropdown to select **Policies**. Move the following items to **Project Members**:

- AA_EnrichAlert
- AA_DeferredEnrichment
- AA_EscalatePlatinum
- AA_PingVerify
- AA_JavaSortDemo
- AA_EvalActionPolicy

7. Use the **List By** dropdown to select **Services**. Move the following items to **Project Members**:

   - EnrichmentEventReader
   - DeferredEnrichmentActivator
   - EscalationActivator
   - HibernatingPolicyActivator
   - EvalActionEventReader
   - EmailSender

8. Review the **Project Members** column for each member type to confirm all items are present, then click **Execute** to create the project.

   ImpactLabProject now appears in the project selector dropdown.

9. Click the project selector dropdown and select ImpactLabProject to switch to that scope. The header label changes from **Global** to **ImpactLabProject**.

   Select the **Data Model** tab. Confirm that DEVICE_INVENTORY, EVENTS, and TAKEACTION are listed under their respective data sources.

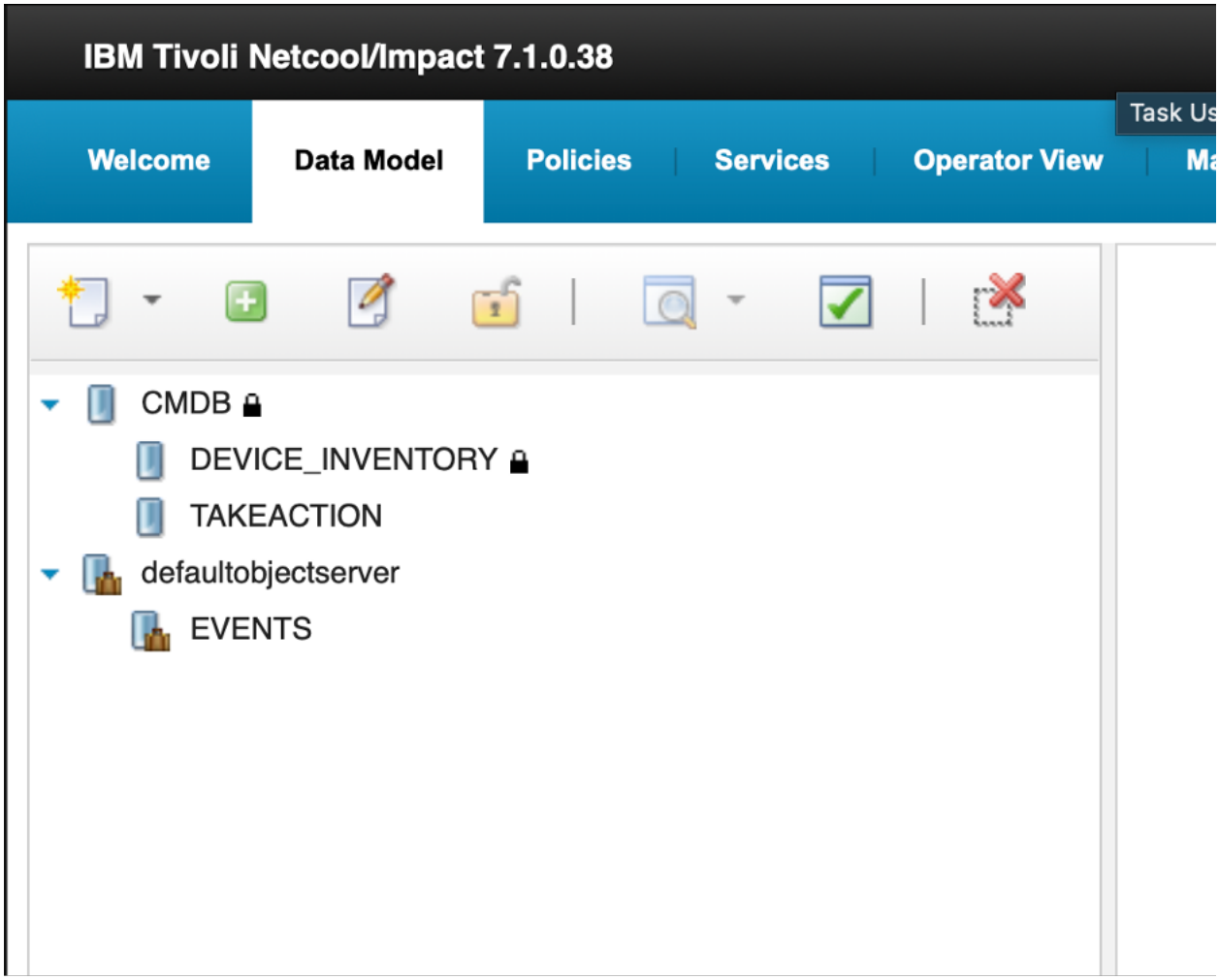


Figure 9: Data Model tab with ImpactLabProject active, showing the three lab data types.

10. Select the **Policies** tab. Confirm that all six lab policies are listed.

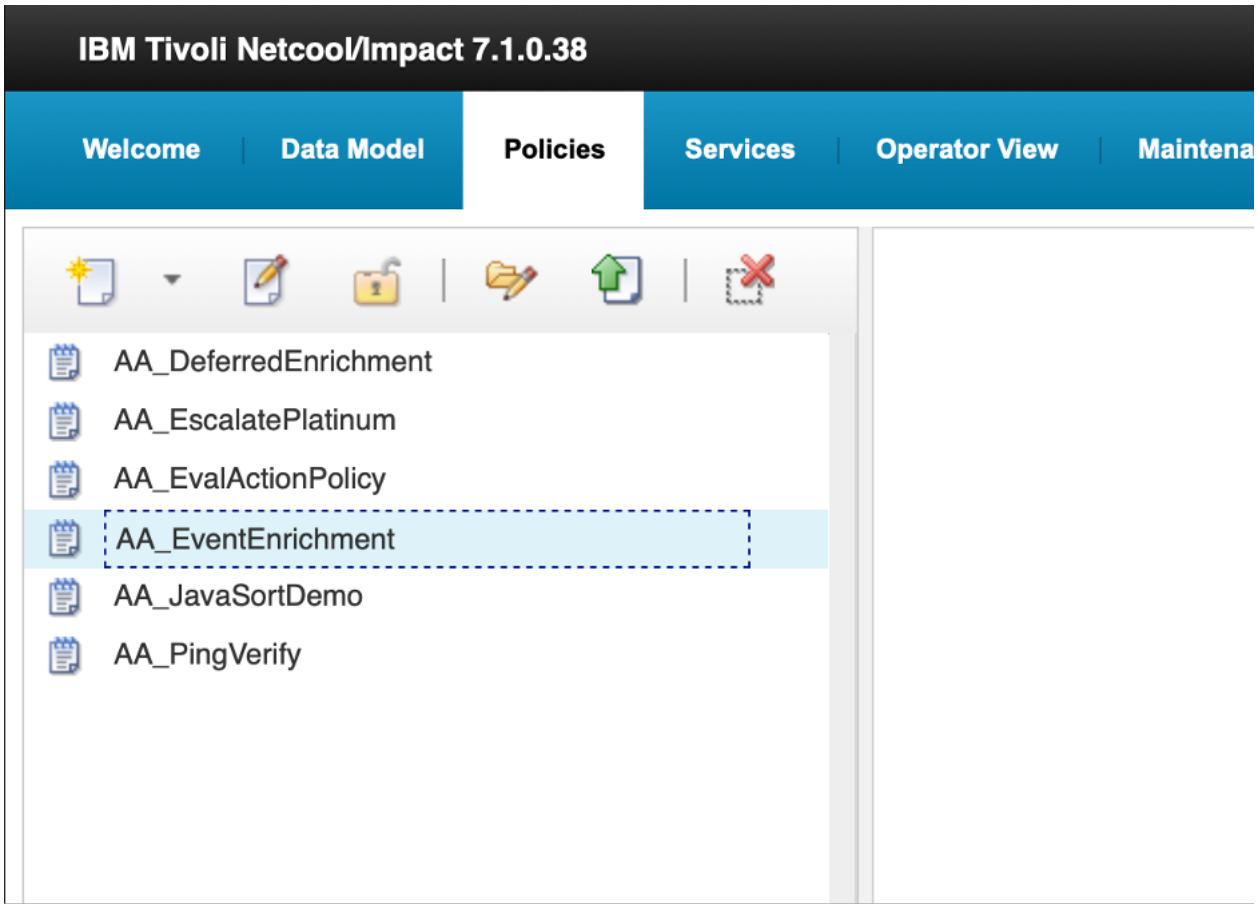


Figure 10: Policies tab with ImpactLabProject active, showing the six lab policies.

11. Select the **Services** tab. Confirm that all six lab services are listed.

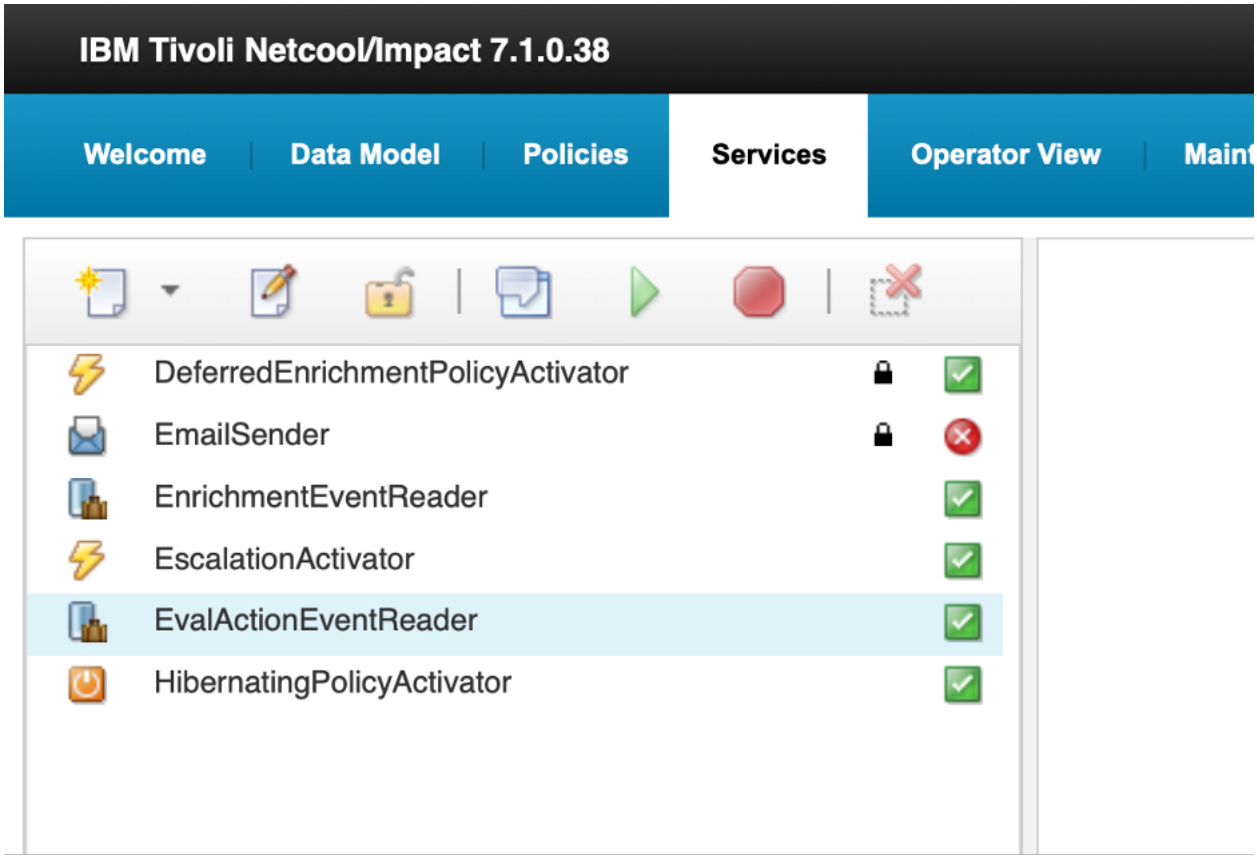


Figure 11: Services tab with ImpactLabProject active, showing the six lab services.

12. Click the project selector dropdown and select **Global** to return to the full scope view.

Validation The exercise is successful when all of the following are true:

- `ImpactLabProject` appears in the project selector dropdown.
- Switching to `ImpactLabProject` shows the six lab policies and six lab services in their respective tabs.
- Switching back to **Global** restores the full artifact list.
- You can describe what the **Global** scope is and why it exists.
- You can explain the purpose of each step in the **Create Project** dialog.

Expected result You have a populated project that captures all of the lab data sources, data types, policies, and services in a single named scope. The next exercise exports this project as a portable archive.

10.2 Export and import the project

Objective In this exercise, you will export `ImpactLabProject` to a directory on the server using the `nci_export` command-line tool, inspect the output, then deliberately delete two lab artifacts to simulate a configuration loss, and restore them using `nci_import`. By the end of the exercise, you will have directly observed how export and import can be used for rollback and recovery, and you will understand the broader use cases for these tools in production environments.

Prerequisites

- Completion of section 10.1.
- `ImpactLabProject` is populated with all lab artifacts.
- SSH access to the lab server.
- The Impact server is running.

Background

nci_export and nci_import Export and import of Impact projects are performed from the command line using `nci_export` and `nci_import`, located in `$IMPACT_HOME/bin`.

`nci_export` connects to the running Impact server instance and writes the project's artifact definitions — data sources, data types, policies, services, and project metadata — to a directory on the filesystem. The export must be run on the primary Impact server; it will fail on a secondary cluster member.

The basic syntax for a single-project export is:

```
nci_export <server_name> --project <project_name> <output_directory>
```

To import, `nci_import` reads from a previously exported directory and loads the artifacts back into the running Impact server:

```
nci_import <server_name> <export_directory>
```

After import, validate and where necessary update any environment-specific data-source settings — such as hostnames, ports, database names, and credentials — before starting services on the target server.

Typical use cases

- **Rollback**: A configuration change to an exported Impact artifact introduced a problem. After stopping or controlling affected services and assessing conflicts, import a verified earlier export to restore the exported object definitions to a known-good state. Revalidate services, data-source connectivity, and policy behavior afterward.
- **Promotion**: Policies and their associated project artifacts developed and validated in a lower environment are exported and imported into the next environment, such as staging to production. Before enabling imported services, review target-specific endpoints, credentials, certificates, drivers, local configuration files, and external dependencies.
- **Bootstrap**: A new or otherwise empty Impact environment receives an initial project import so that the project's exported artifacts are created there. Configure and validate server-local and external dependencies before activating services.
- **Configuration backup**: Run a scheduled or pre-change `nci_export` to preserve a point-in-time copy of exported Impact repository artifacts for recovery, comparison, migration, or rollback.
- **Audit and change control**: Export a project before and after a planned change, retain both versions under controlled storage, and compare or review them as evidence of what changed.

Tasks

1. SSH into the lab server and change to the Impact binary directory:

   ```
   cd $IMPACT_HOME/bin
   ```

2. Create an export destination directory:

   ```
   mkdir -p /tmp/impact-exports
   ```

3. Run the export for `ImpactLabProject`:

   ```
   ./nci_export NCI --project ImpactLabProject /tmp/impact-exports/ImpactLabProject
   ```

   The command connects to the running Impact server instance `NCI` and writes the project artifacts to the specified directory. You should see output confirming the export completed successfully.

4. Verify the export directory is populated:

   ```
   find /tmp/impact-exports/ImpactLabProject -type f | sort
   ```

   The listing will show individual files for each exported artifact — data sources, data types, policies, and services are each written as separate files. Confirm the file count matches the number of artifacts you added to the project.

5. Before importing, simulate a configuration loss by removing two artifacts from the server. In the Impact GUI, select the **Services** tab.

   Right-click `EnrichmentEventReader` and select **Stop**. Wait for the state to change to **Stopped**, then right-click it again and select **Delete**. Confirm the deletion. `EnrichmentEventReader` should no longer appear in the Services list.

6. Select the **Policies** tab. Right-click `AA_EnrichAlert` and select **Delete**. Confirm the deletion. `AA_EnrichAlert` should no longer appear in the Policies list.
7. Confirm both artifacts are gone. Switch to `ImpactLabProject` in the project selector — `AA_EnrichAlert` should be absent from the Policies tab and `EnrichmentEventReader` should be absent from the Services tab. Switch back to **Global** before continuing.
8. Return to the terminal and run the import:

   ```
   ./nci_import NCI /tmp/impact-exports/ImpactLabProject
   ```

   The command restores all exported artifacts. `AA_EnrichAlert` and `EnrichmentEventReader` were absent on the server, so they will be created. Artifacts that were already present will be skipped or overwritten. Review the command output and confirm both deleted artifacts appear in the import log.

   > **If the import reports lock errors**, one or more artifacts are held by a version-control lock. Resolve this before retrying:
   >
   > **1.** Check which items are currently locked:
   >
   > ```
   > cat "$IMPACT_HOME/etc/NCI_versioncontrol.locks"
   > ```
   >
   > **2.** Clear all version-control locks on instance `NCI`:
   >
   > ```
   > "$IMPACT_HOME/bin/nci_version_control" NCI uncoall ""
   > ```
   >
   > **3.** Confirm the lock list is clear or reduced as expected:
   >
   > ```
   > cat "$IMPACT_HOME/etc/NCI_versioncontrol.locks"
   > ```
   >
   > Then re-run the `nci_import` command above.
9. Back in the Impact GUI, confirm the two deleted artifacts have been restored:
   - Select the **Policies** tab and confirm `AA_EnrichAlert` is present.
   - Select the **Services** tab and confirm `EnrichmentEventReader` is present.
10. Start `EnrichmentEventReader`. Right-click it and select **Start**. Confirm it returns to **Running** state.
11. Confirm that all other services that should be running are also in **Running** state. If any were stopped by the import, right-click each and select **Start**:
    - `DeferredEnrichmentActivator`
    - `EscalationActivator`
    - `EvalActionEventReader`
12. Insert a smoke-test event to confirm the enrichment pipeline is intact after the restore:

    ```
    $OMNIHOME/bin/nco_sql -server EVENTS -user root -password netcool << 'EOF'
    INSERT INTO alerts.status (Node, Summary, AlertKey, AlertGroup, Severity, Type, Identifier,
     ↪  Manager, OwnerUID)
    VALUES ('router-lon-01', 'Post-import smoke test', 'import-test-01', 'TrainingAlerts', 3, 1,
     ↪  'router-lon-01:import-test-01:TrainingAlerts', 'ImpactLab', 65534);
    go
    EOF
    ```

13. Check the enrichment policy log:

    ```
    tail -10 $IMPACT_HOME/logs/NCI_policylogger_AA_EnrichAlert.log
    ```

    You should see an entry for `router-lon-01` showing `CUST-1001` and `ImpactFlag = 1`, confirming the restored policy is running correctly.
14. Switch to the OMNIbus Web GUI in your private/incognito browser. Select the **ImpactLab Events** filter and the **SyslogdView** view. Locate the `router-lon-01` smoke-test event and confirm the enrichment fields are populated — `CustomerID`, `CustomerName`, `Service`, and `Location` should all reflect the values from the CMDB, and `ImpactFlag` should be `1`.

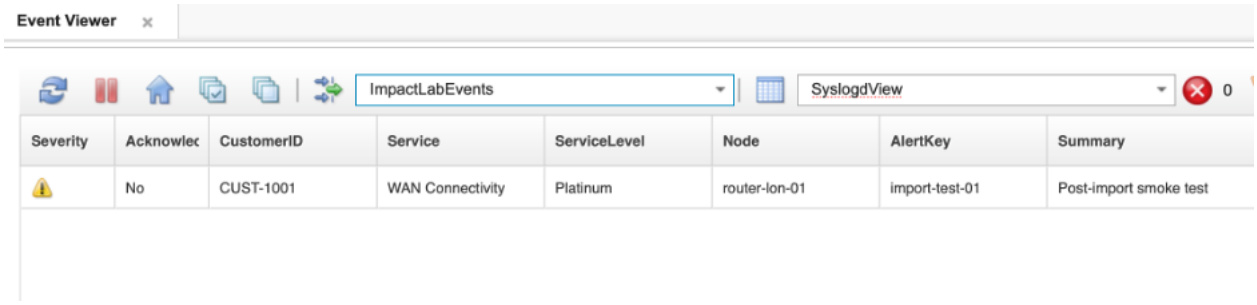


Figure 12: SyslogdView showing the router-lon-01 smoke-test event with enrichment fields populated after the import restore.

Validation The exercise is successful when all of the following are true:

- The `nci_export` command completed without errors and the export directory is populated with artifact files.
- `AA_EnrichAlert` and `EnrichmentEventReader` were successfully deleted before the import.
- The `nci_import` command completed without errors and the import log confirms both deleted artifacts were recreated.
- `EnrichmentEventReader` is running and the smoke-test event was enriched correctly.
- You can describe a real-world scenario for each of the five use cases: rollback, promotion, bootstrap, configuration backup, and audit and change control.
- You can explain what must be validated in the imported data-source definitions before enabling services on a new target server.
- You can state where `nci_export` must be run in a clustered Impact environment and why.

Expected result You have completed the full project lifecycle: create, populate, export, and import. By deliberately removing artifacts and restoring them via `nci_import`, you directly observed how the import behaves when artifacts are missing — exactly the pattern used for rollback and recovery in production. You also understand how the project selector scopes the Policies and Services views to show only the artifacts relevant to the work at hand.

11. Granting users access to Impact

In a fresh Impact installation, the only Impact user is `impactadmin`, which is created during the installation process. All other users must be explicitly granted access before they can do anything in the Impact GUI.

Impact authenticates users against a configured user repository. In this lab, Impact is configured to authenticate against the OMNIbus EVENTS ObjectServer. This means any user defined in the ObjectServer — including `ncoadmin`, which you have been using throughout the lab to log into the OMNIbus Web GUI and manipulate events — can present valid credentials at the Impact login page. However, being able to authenticate is not the same as having access: a user who can log in but has no Impact role mappings is presented with a completely blank workspace.

> **Note:** ObjectServer-based authentication is used in this lab for simplicity. Impact also supports authentication via LDAP and via Keycloak/OIDC for environments with centralized identity management.

Access is controlled through role mappings, managed from the command line using `mapRoles.sh`, located in `/opt/IBM/tivoli/impa`. Each mapping associates a username with one or more Impact roles that determine which parts of the GUI are visible and what actions the user can perform. Roles are additive — a user can hold a narrow functional role and a broader administrative role at the same time. Changes take effect at the next login with no server restart required.

This section walks through the role-mapping workflow using the `ncoadmin` account: first observing the blank experience of an unmapped user, then granting a scoped functional role, and finally granting full administrative access.

11.1 Observe the default experience of an unmapped user

Objective In this exercise, you will log in to the Impact GUI as `ncoadmin` and observe what an unmapped user sees. By the end of the exercise, you will understand the distinction between authentication and access in Impact, and why role mapping is required before a user can work with the product.

Prerequisites

- You are currently logged in to the Impact GUI as `impactadmin` in your private/incognito browser session.

Background `ncoadmin` is an ObjectServer user that you have used throughout this lab to log into the OMNIbus Web GUI. Because Impact in this lab is configured to authenticate against the EVENTS ObjectServer, `ncoadmin` can also log into the Impact GUI — but authentication alone grants no access. The user has no Impact role mappings, so Impact has nothing to display.

This two-layer model — authenticate first, then authorize via role mappings — is standard Impact behavior. It means that adding a new user to Impact does not require creating a separate Impact account; you simply map an existing authenticated identity to the appropriate roles.

Tasks

1. In your private/incognito browser, click the user menu in the top-right corner of the Impact GUI and select **Logout**. Confirm you are returned to the login page.
2. Log in with the following credentials:
   - **Username**: `ncoadmin`
   - **Password**: `netcool`

   If you see a "Connection Failed — Unable to reach the nameserver to get the list of clusters" error, the browser is carrying stale session state from the previous login. Click **Close**, then close the private/incognito browser window entirely and open a fresh one. Navigate back to the Impact GUI URL and log in again. This clears the cached session and the error will not recur. The same applies to any subsequent login in this section.

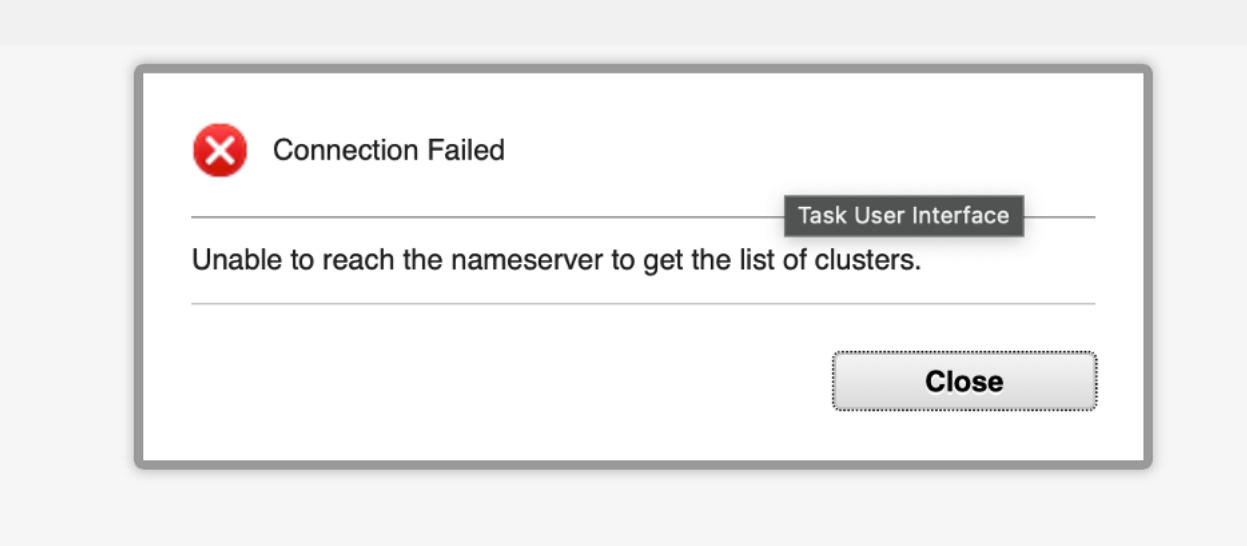


Figure 13: Connection Failed dialog showing "Unable to reach the nameserver to get the list of clusters".

3. Observe the result. The workspace is blank — no tabs, no navigation, no content. Authentication succeeded, but `ncoadmin` has no Impact role mappings, so there is nothing to show.
4. Log out of the Impact GUI.

Validation The exercise is successful when all of the following are true:

- You were able to log in as `ncoadmin` without an authentication error.
- The workspace was blank with no tabs or navigation visible.
- You can explain why `ncoadmin` can authenticate but sees a blank page.
- You can explain why no separate Impact account was needed for `ncoadmin` to log in.

Expected result You have confirmed the default state for an unmapped user and understand the distinction between authentication and access. The next exercise grants `ncoadmin` a scoped functional role and demonstrates the resulting change in access.

11.2 Grant Maintenance Window Manager access

Objective In this exercise, you will use `mapRoles.sh` to grant `ncoadmin` the Impact Maintenance Window Manager roles, then log in to confirm the access. By the end of the exercise, you will understand what the Maintenance Window Manager function does and how scoped role mapping works.

Prerequisites

- Completion of section 11.1.
- SSH access to the lab server.
- You are logged out of the Impact GUI.

Background

Maintenance Window Manager The **Maintenance Window Manager** (MWM) is a built-in Impact feature that lets operators schedule and manage maintenance windows — defined time periods during which event processing rules or escalation policies are suppressed or modified. During a maintenance window, events from a node or group of nodes that would normally trigger policies can be held back, preventing alert storms during planned outages such as patching, reboots, or infrastructure changes.

The MWM has two associated roles:

Role	Purpose
`impactMWMUser`	Allows the user to view maintenance windows and their schedules
`impactMWMAdminUser`	Allows the user to create, edit, and delete maintenance windows

Both roles are typically granted together so that operators can both view and manage windows.

mapRoles.sh `mapRoles.sh` is the command-line tool for adding and removing Impact role mappings. It is located in `/opt/IBM/tivoli/impact/install/security/` and must be run as `root` on the Impact server. The basic syntax for adding roles is:

```
./mapRoles.sh -add -user <username> -roles "<role1>|<role2>"
```

Multiple roles are separated by a pipe character (`|`) within the quoted string. Changes take effect at the next login.

Tasks

1. SSH into the lab server as `root` and navigate to the security directory:

   ```
   cd /opt/IBM/tivoli/impact/install/security
   ```

2. Grant `ncoadmin` the Maintenance Window Manager roles:

   ```
   ./mapRoles.sh -add -user ncoadmin -roles "impactMWMAdminUser|impactMWMUser"
   ```

   You should see the following confirmation:

   ```
   Adding user ncoadmin to role impactMWMAdminUser
   Adding user ncoadmin to role impactMWMUser
   ```

3. Return to your private/incognito browser and log in to the Impact GUI as `ncoadmin` / `netcool`.
4. Observe the result. The **Maintenance Window** tab is now visible in the top navigation bar. `ncoadmin` can view and manage maintenance windows, but no other areas of Impact are accessible.
5. Explore the **Maintenance Window** tab briefly — note that it is the only tab present. The Policies, Services, Data Model, and other administrative tabs are not visible because `ncoadmin` has not been granted the corresponding roles.
6. Log out of the Impact GUI.

Validation The exercise is successful when all of the following are true:

- The `mapRoles.sh` command completed without errors.
- Logging in as `ncoadmin` shows the **Maintenance Window** tab and no other tabs.
- You can explain the difference between `impactMWMUser` and `impactMWMAdminUser`.
- You can explain why the Policies and Services tabs are not visible.

Expected result You have granted a scoped functional role and confirmed the result. `ncoadmin` can now manage maintenance windows and nothing else. The next exercise grants full administrative access.

11.3 Grant full administrative access

Objective In this exercise, you will grant `ncoadmin` the `impactAdminUser` role and confirm that full access is available. By the end of the exercise, you will understand how the administrative role relates to the narrower functional roles already assigned.

Prerequisites

- Completion of section 11.2.
- You are logged out of the Impact GUI.

Background The `impactAdminUser` role grants access to all areas of the Impact GUI — Policies, Services, Data Model, Operator View, Maintenance Window, and administrative functions. It is the equivalent of the built-in `impactadmin` account's access level and is appropriate for Impact administrators and developers who need full control over the environment.

Assigning `impactAdminUser` to a user who already holds narrower roles (such as `impactMWMAdminUser`) is additive — the narrower role mappings remain in place and the user simply gains the additional access that `impactAdminUser` provides.

Tasks

1. In your SSH session, confirm you are still in the `/opt/IBM/tivoli/impact/install/security` directory, then grant `ncoadmin` the administrative role:

   ```
   ./mapRoles.sh -add -user ncoadmin -roles "impactAdminUser"
   ```

   You should see:

   ```
   Adding user ncoadmin to role impactAdminUser
   ```

2. Return to your private/incognito browser and log in to the Impact GUI as `ncoadmin` / `netcool`.
3. Observe the result. The full navigation bar is now visible — **Policies**, **Services**, **Data Model**, **Operator View**, **Maintenance Window**, and **Reports** tabs are all present. `ncoadmin` now has the same level of access as `impactadmin`.
4. Browse the **Policies** tab and confirm the lab policies are visible. Browse the **Services** tab and confirm the lab services are visible.
5. Log out of the Impact GUI in your private/incognito browser and log back in as `impactadmin` / `netcool` to restore the original session.

Validation The exercise is successful when all of the following are true:

- The `mapRoles.sh` command completed without errors.
- Logging in as `ncoadmin` shows the full navigation bar with all tabs.
- You can confirm the lab policies and services are visible to `ncoadmin`.
- You can explain what `impactAdminUser` grants and how it relates to the narrower roles already assigned.
- You can describe a scenario where granting only `impactMWMAdminUser` — rather than `impactAdminUser` — is the correct choice.

Expected result You have granted full administrative access to `ncoadmin` and confirmed the result. You understand the Impact role model: access is controlled entirely by role mappings, mappings are additive, and `/opt/IBM/tivoli/impact/install/securi` is the tool used to manage them from the command line.

2. Impact data source adapters (DSAs)

Data source adapters (DSAs) are the connectors that give Impact policies access to external systems. In most cases, when a policy needs to look up data from a database, query an API, read from LDAP, or interact with an OMNIbus ObjectServer, it does so through a named DSA. There are a few exceptions — for example, policies can make direct HTTP calls or execute inline logic without a DSA — but for the majority of integration scenarios a DSA is the standard mechanism. Without at least one DSA configured, a policy has no way to reach any data outside of the event it received when it was triggered.

DSAs are defined once at the server level and then referenced by name in policy code. This means policies do not need to include connection strings, credentials, or hostname details — those are managed centrally in the DSA configuration, and policies simply call the DSA by name. If the target system changes, only the DSA configuration needs to be updated, rather than every policy that uses it. This also improves security, since credentials are stored and managed in one place rather than embedded in policy code where they could be more easily exposed or overlooked during a rotation.

Impact ships with a set of built-in adapter types that cover the most common integration scenarios. The most frequently used adapter types in OMNIbus environments are:

- **ObjectServer DSA**: Connects to a Netcool/OMNIbus ObjectServer. Used both to receive triggering events and to write enriched values back to `alerts.status`.
- **JDBC DSA**: Connects to any relational database with an available JDBC driver, including MySQL, DB2, Oracle, and Microsoft SQL Server.
- **HTTP/REST DSA**: Connects to REST APIs over HTTP or HTTPS, supporting GET, POST, PUT, and DELETE operations.
- **LDAP DSA**: Connects to an LDAP directory, such as Microsoft Active Directory or IBM Directory Server.
- **File DSA**: Reads data from a delimited flat file, useful for simple lookup tables that do not require a live database connection.

2.1 The ObjectServer DSA

Objective In this exercise, you will review the pre-configured ObjectServer DSA and verify that Impact can connect to the OMNIbus ObjectServer. By the end of the exercise, you will understand what the ObjectServer DSA is used for, how to inspect its configuration, and how to confirm that the connection state is active.

Prerequisites

- Completion of section 1.6.
- The Impact Server is running.
- The OMNIbus ObjectServer (EVENTS) is running.

Background The ObjectServer DSA is one of the most important DSAs in any Impact deployment because it provides the primary link between Impact and the OMNIbus event stream. It serves two distinct purposes:

1. **Event source**: Services that monitor the ObjectServer use the ObjectServer DSA to receive a stream of events that match a defined filter condition. When a new alert arrives or an existing alert is updated in `alerts.status`, the service receives the event data and passes it to the configured policies.
2. **Write-back target**: After a policy has looked up enrichment data or performed automation, it uses the ObjectServer DSA to write results back to `alerts.status`. This is how Impact-enriched fields such as `CustomerID`, `Service`, or `CircuitID` appear on an alert in the OMNIbus event list.

The ObjectServer DSA is typically pre-configured during Impact installation and should already point to the EVENTS ObjectServer used in this lab.

Tasks

1. Open the Impact GUI at `http://<your-server>:17311/ibm/console` and log in as `impactadmin` with password `netcool`.
2. Select the **Data Model** tab from the top navigation bar. In the data source list on the left, locate the `defaultobjectserver` entry and double-click it. The DSA configuration will open as a tabbed window in the workspace on the right.
3. Review the configuration tabs that open in the workspace on the right.

   The ObjectServer Data Source Editor configures how Netcool/Impact connects to the default ObjectServer. It is organized into three main sections: **General Settings**, **Database Failure Policy**, and **Primary/Backup Source** connection details.

   - **Data Source Name**: `defaultobjectserver` — the internal name by which Impact policies refer to this data source.

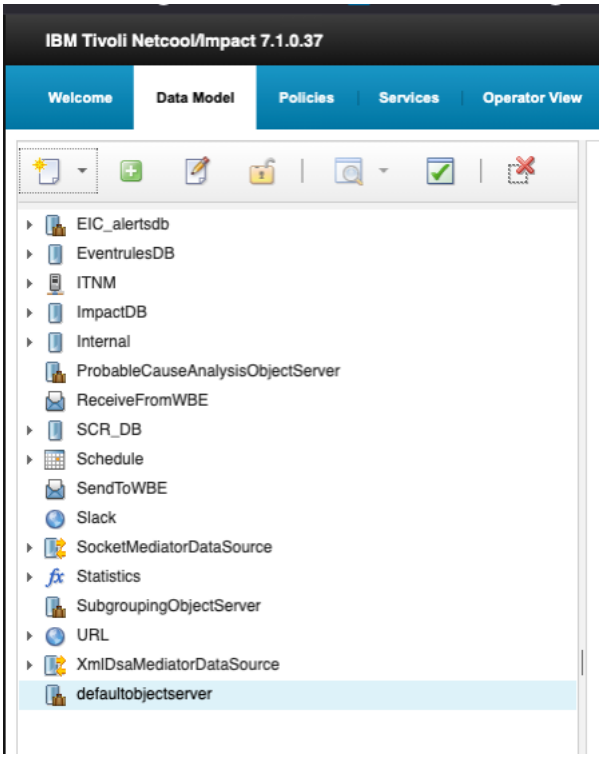


Figure 14: Data Sources panel showing ObjectServer DSA

- **Username and Password**: The database credentials Impact uses to authenticate to the ObjectServer. The configured username is `root`, while the password is masked.
- **Maximum SQL Connections**: Set to `30`, this limits the number of concurrent SQL connections that Impact can open to this data source.
- **Database Failure Policy**: Controls Impact's behavior if it cannot connect to the ObjectServer. **Fail back** is selected, meaning Impact will attempt to return to the primary source after using the backup source.
- **Primary Source**: Defines the main ObjectServer connection. The host is `visa-training-jumphost1.fyre.ibm.com` on port `4100`. SSL Mode is available but not selected. The **Test Connection** button can be used to validate the supplied primary connection settings.
- **Backup Source**: Defines a secondary ObjectServer connection for resilience if the primary source becomes unavailable. In the screenshot, it points to `localhost` on port `4100`.

  Note: Fields marked with an asterisk are required.

4. Click the **Test Connection** button in the **Primary Source** section. Impact uses the username and password configured in the **General Settings** section to attempt a live connection to the ObjectServer at the specified host and port. A popup window will appear with the result of the connection attempt. Confirm that it displays **Connection OK**.

   If the result does not say **Connection OK**, verify that the EVENTS ObjectServer is running on the lab server before retrying.

5. The `defaultobjectserver` data source provides the underlying connection to the ObjectServer database. On its own, it can be used by policies to execute direct SQL calls against any table that the configured user has access to — for example, querying `alerts.status` or updating fields on specific alerts.

   In addition to direct SQL access, Impact also supports **data types**. A data type is defined on top of a data source and targets a specific table or view. It exposes that table to policies through simplified built-in functions such as `GetByFilter`, `GetByKey`, `AddDataItem`, and `DeleteDataItem`, without requiring the policy to write raw SQL. This makes policy code cleaner, easier to read, and less prone to errors.

   The distinction is important:

   - A **data source** defines the connection — the server, credentials, and database to connect to.
   - A **data type** defines a structured view of a specific table within that connection, giving policies a higher-level interface to work with.

   To create a new data type on this data source, right-click on `defaultobjectserver` in the data source list on the left and

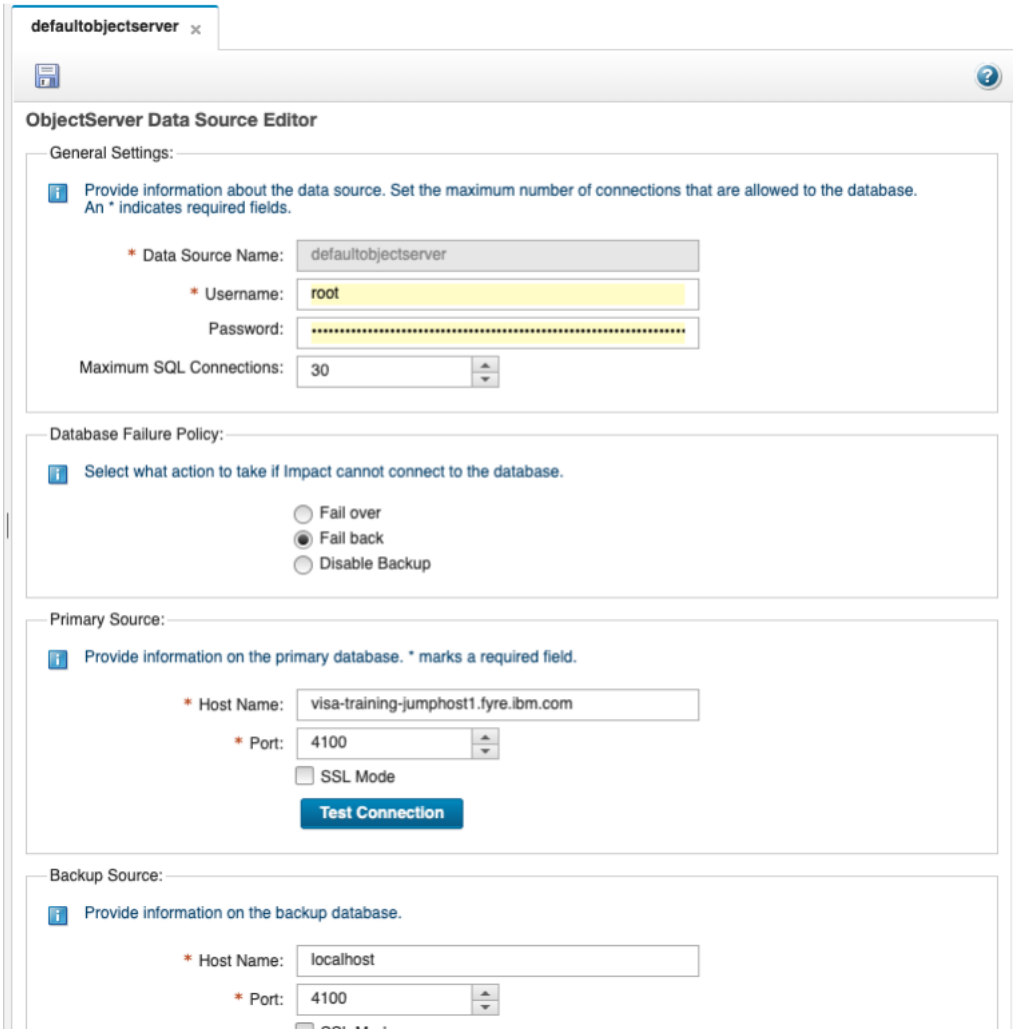


Figure 15: ObjectServer DSA configuration details

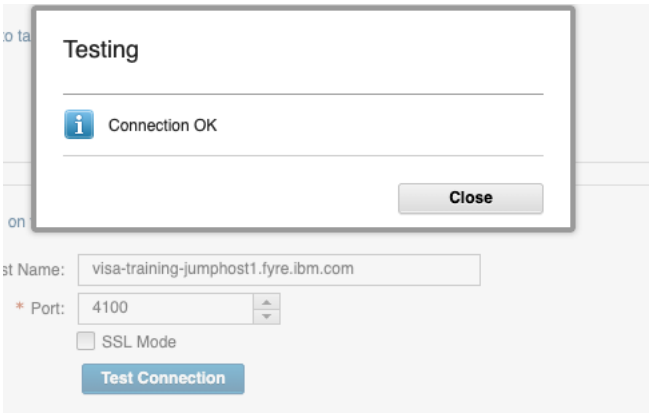


Figure 16: Test Connection result popup

select **New Data Type**.

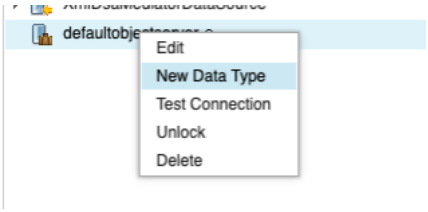


Figure 17: New Data Type context menu

6. A new tab labelled **New Data Type for defaultobjectserver** will open in the workspace on the right, showing the **SQL Data Type Config Page**. Configure the new data type as follows:

   - **Data Type Name**: EVENTS
   - **Data Source Name**: leave as `defaultobjectserver`
   - **Base Table**: `alerts.status`

   Once the base table is entered, click the **Refresh Fields** button. Impact will query the ObjectServer and populate the field list below with the structure of the `alerts.status` table. Each row in the list shows the field **ID**, **Field Name**, **Format**, **Display Name**, **Description**, and a **Key Field** column.

   The **Key Field** setting identifies which field uniquely identifies a record in this data type. Impact uses this field when performing key-based lookups with functions such as `GetByKey`. Scroll through the field list to locate the **Identifier** field and set its **Key Field** column to **Yes**.

   When complete, your configuration should match the following screenshot:

7. Save the data type by clicking the floppy disk icon at the top left of the configuration tab. Once saved, the EVENTS data type will appear nested under `defaultobjectserver` in the data source list on the left.

Validation The exercise is successful when all of the following are true:

- You can navigate to the **Data Model** tab and locate the `defaultobjectserver` entry in the data source list.
- You can open the DSA configuration by double-clicking and identify the **General Settings**, **Database Failure Policy**, and **Primary/Backup Source** sections.
- The **Test Connection** result displays **Connection OK** for the primary source.
- You can explain the difference between a data source and a data type.
- You can explain that the `defaultobjectserver` DSA serves both as an event source and as a write-back target for enrichment results.
- A new EVENTS data type has been created targeting the `alerts.status` table, with `Identifier` set as the key field.
- The EVENTS data type is saved and visible nested under `defaultobjectserver` in the data source list.

Expected result At the end of this exercise, you will understand the role of the ObjectServer DSA in an Impact deployment, know how to inspect and test its configuration, and be able to explain the difference between a data source and a data type. You will also have created the EVENTS data type targeting `alerts.status` with `Identifier` as the key field — a foundation that will be used by policies in later exercises to query and update alerts directly through a structured, simplified interface.

2.2 Creating a JDBC DSA for event enrichment

Objective In this exercise, you will create an enrichment database in DB2, populate it with sample device inventory data, and then connect Impact to that database by creating a JDBC DSA and a corresponding data type. By the end of the exercise, you will have a working enrichment data source that Impact policies can query in later exercises to look up customer and service information for incoming alerts.

Prerequisites

- Completion of section 2.1.
- SSH access to the lab server as `root`.
- DB2 is installed on the lab server and the `db2inst1` instance is available.
- The DB2 JDBC driver (`db2jcc4.jar`) is available to the Impact server.

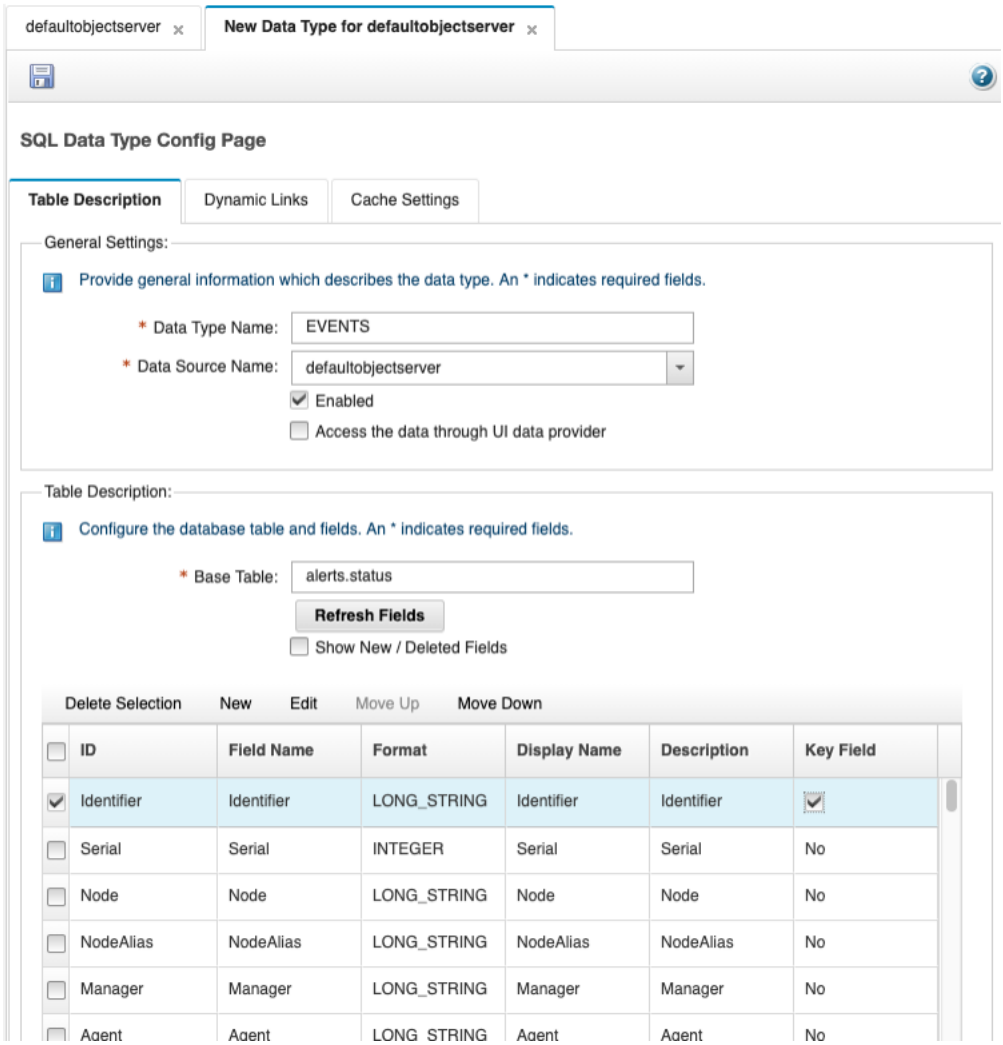


Figure 18: SQL Data Type Config Page for EVENTS data type

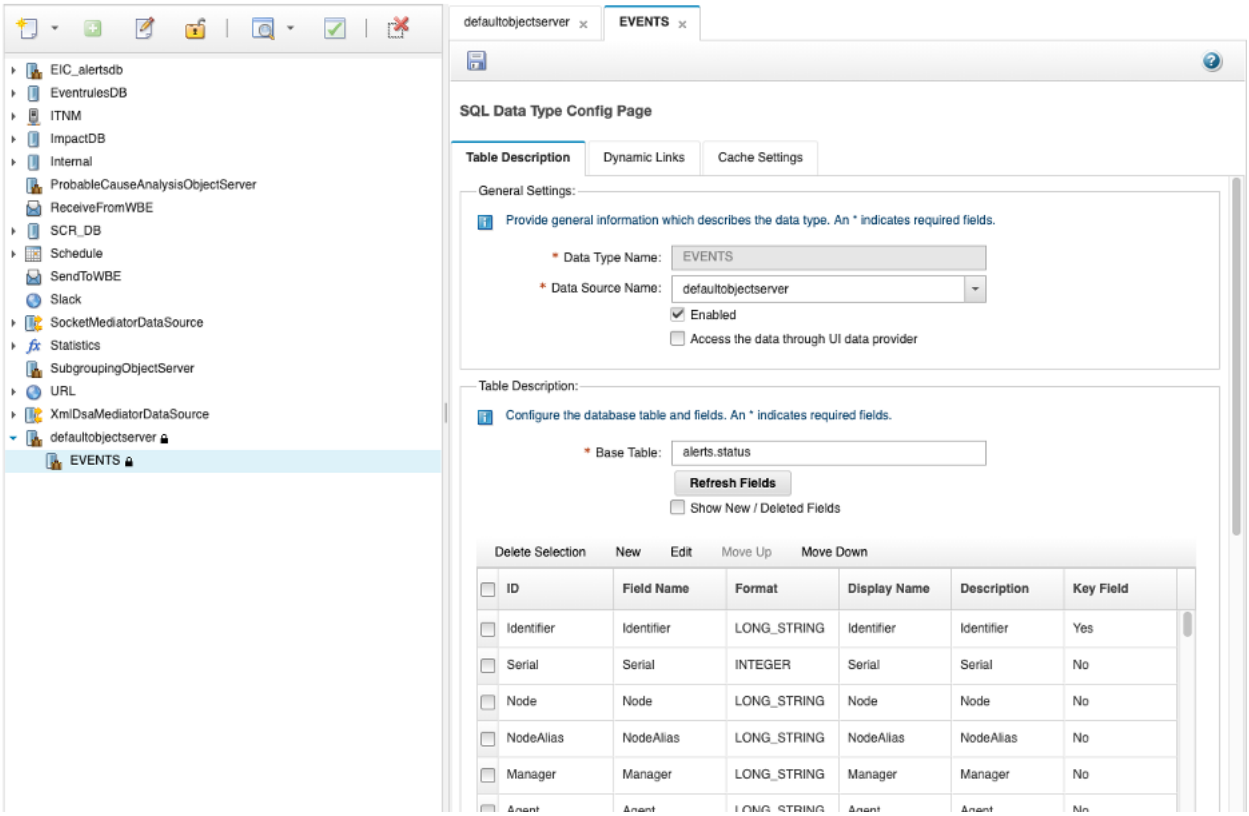


Figure 19: EVENTS data type saved and visible in data source list

Background A common use case for Impact is **event enrichment** — the process of adding contextual information to an alert that the probe could not provide on its own. For example, a probe may insert an alert with a `Node` value of `router-lon-01`, but an operator needs to know which customer owns that device, what service it supports, and who to contact. Rather than building that knowledge into the probe rules, Impact can look it up at runtime from an external database and write the results directly back to the alert.

To support this pattern, the lab environment will use a DB2 database named `CMDB` with a single enrichment table called `DEVICE_INVENTORY`. This table maps device hostnames to customer and service context. Once the table is created and populated, Impact will connect to it through a JDBC DSA, and a data type will be defined on top of that connection so that policies can query the table using simplified built-in functions.

The `DEVICE_INVENTORY` table has the following structure:

Column	Type	Description
`NODE_NAME`	VARCHAR(128)	Device hostname — matches the `Node` field in OMNIbus alerts
`CUSTOMER_ID`	VARCHAR(64)	Customer identifier associated with the device
`CUSTOMER_NAME`	VARCHAR(128)	Full customer name
`SERVICE`	VARCHAR(64)	Business service supported by the device
`LOCATION`	VARCHAR(64)	Physical or logical location of the device
`CONTACT`	VARCHAR(128)	On-call contact name for the device

Tasks

1. SSH into the lab server as `root`. Switch to the DB2 instance user:

   ```
   su - db2inst1
   ```

   Then verify that the DB2 instance is running:

```
db2gcf -s
```

The output should show the instance status as `Available`. If the instance is not running, start it before proceeding:

```
db2start
```

2. Create the enrichment database from the bash shell — do not open the db2 interactive session yet:

```
db2 CREATE DATABASE CMDB
```

Wait for the command to complete. You should see:

```
DB20000I  The CREATE DATABASE command completed successfully.
```

3. Now connect to the new database and create the `DEVICE_INVENTORY` table. Run the following as a single script by saving it to a file and executing it:

```
db2 CONNECT TO CMDB
db2 "CREATE TABLE DEVICE_INVENTORY ( \
    NODE_NAME     VARCHAR(128) NOT NULL, \
    CUSTOMER_ID   VARCHAR(64), \
    CUSTOMER_NAME VARCHAR(128), \
    SERVICE       VARCHAR(64), \
    LOCATION      VARCHAR(64), \
    CONTACT       VARCHAR(128), \
    PRIMARY KEY (NODE_NAME) )"
```

Confirm that the output includes:

```
DB20000I  The SQL command completed successfully.
```

4. Populate the table by writing the INSERT statements to a script file and running it with `db2 -tf`. Create the file with the following command:

```
cat > /tmp/load_cmdb.sql << 'EOF'
CONNECT TO CMDB;
INSERT INTO DEVICE_INVENTORY VALUES ('router-lon-01', 'CUST-1001', 'Acme Corporation',  'WAN
 ↪  Connectivity',   'London DC',    'noc@acme.com');
INSERT INTO DEVICE_INVENTORY VALUES ('router-lon-02', 'CUST-1001', 'Acme Corporation',  'WAN
 ↪  Connectivity',   'London DC',    'noc@acme.com');
INSERT INTO DEVICE_INVENTORY VALUES ('switch-nyc-01', 'CUST-1002', 'Globex Industries', 'LAN
 ↪  Infrastructure', 'New York DC',  'ops@globex.com');
INSERT INTO DEVICE_INVENTORY VALUES ('switch-nyc-02', 'CUST-1002', 'Globex Industries', 'LAN
 ↪  Infrastructure', 'New York DC',  'ops@globex.com');
INSERT INTO DEVICE_INVENTORY VALUES ('server-fra-01', 'CUST-1003', 'Initech Solutions', 'App
 ↪  Hosting',        'Frankfurt DC', 'support@initech.com');
INSERT INTO DEVICE_INVENTORY VALUES ('server-fra-02', 'CUST-1003', 'Initech Solutions', 'App
 ↪  Hosting',        'Frankfurt DC', 'support@initech.com');
INSERT INTO DEVICE_INVENTORY VALUES ('fw-sin-01',     'CUST-1004', 'Pinnacle Networks',
 ↪  'Network Security',   'Singapore DC', 'security@pinnacle.com');
INSERT INTO DEVICE_INVENTORY VALUES ('db-tok-01',     'CUST-1005', 'Soylent Systems',
 ↪  'Database Services',  'Tokyo DC',     'dba@soylent.com');
COMMIT;
EOF
```

Then run the script:

```
db2 -tf /tmp/load_cmdb.sql
```

Each INSERT statement should return `DB20000I  The SQL command completed successfully.`

5. Verify that the data was inserted correctly:

```
db2 CONNECT TO CMDB
db2 "SELECT * FROM DEVICE_INVENTORY"
```

Confirm that eight rows are returned with the expected values.

6. Return to the Impact GUI at `http://<your-server>:17311/ibm/console` and log in as `impactadmin` with password `netcool`. Select the **Data Model** tab from the top navigation bar. Click the **New Data Source** icon (the star/page icon at the top left of the data source toolbar) and from the dropdown menu select **Database (SQL) → DB2**.

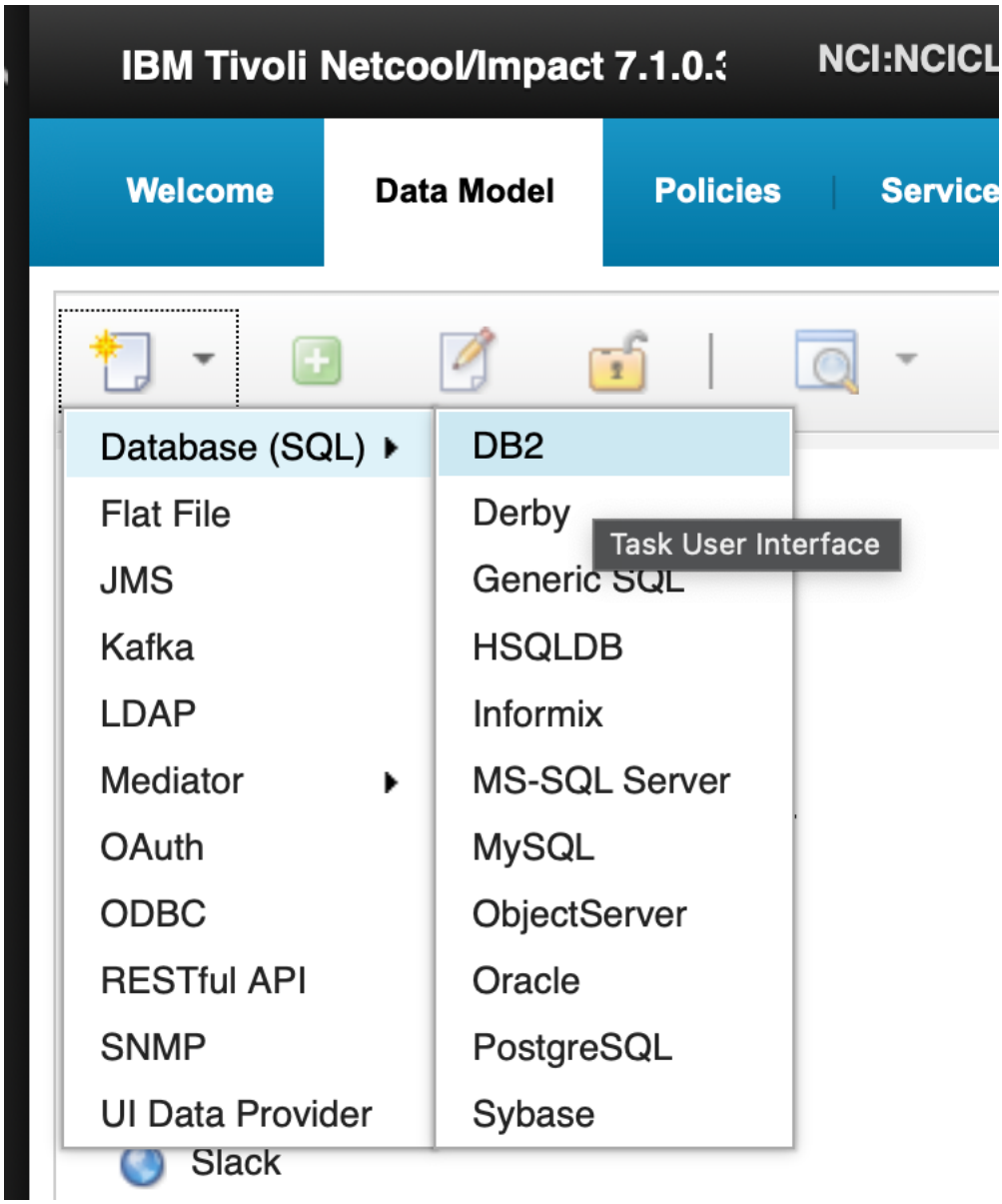


Figure 20: New Data Source menu - Database SQL DB2

7. The **SQL Data Source Config Page** will open as a new tab in the workspace. Configure the DB2 JDBC connection with the following settings:

   - **Data Source Name**: `CMDB`
   - **JDBC URL**: `jdbc:db2://localhost:50000/CMDB`
   - **JDBC Driver**: `com.ibm.db2.jcc.DB2Driver`
   - **Username**: `db2inst1`
   - **Password**: `N3tc00l2024N3tc00l20204!`
   - **Maximum SQL Connections**: `10`

8. Click the **Test Connection** button. Confirm that the result popup displays **Connection OK**. If it does not, verify that the DB2 instance is running and that the database `CMDB` was created successfully in the previous steps.

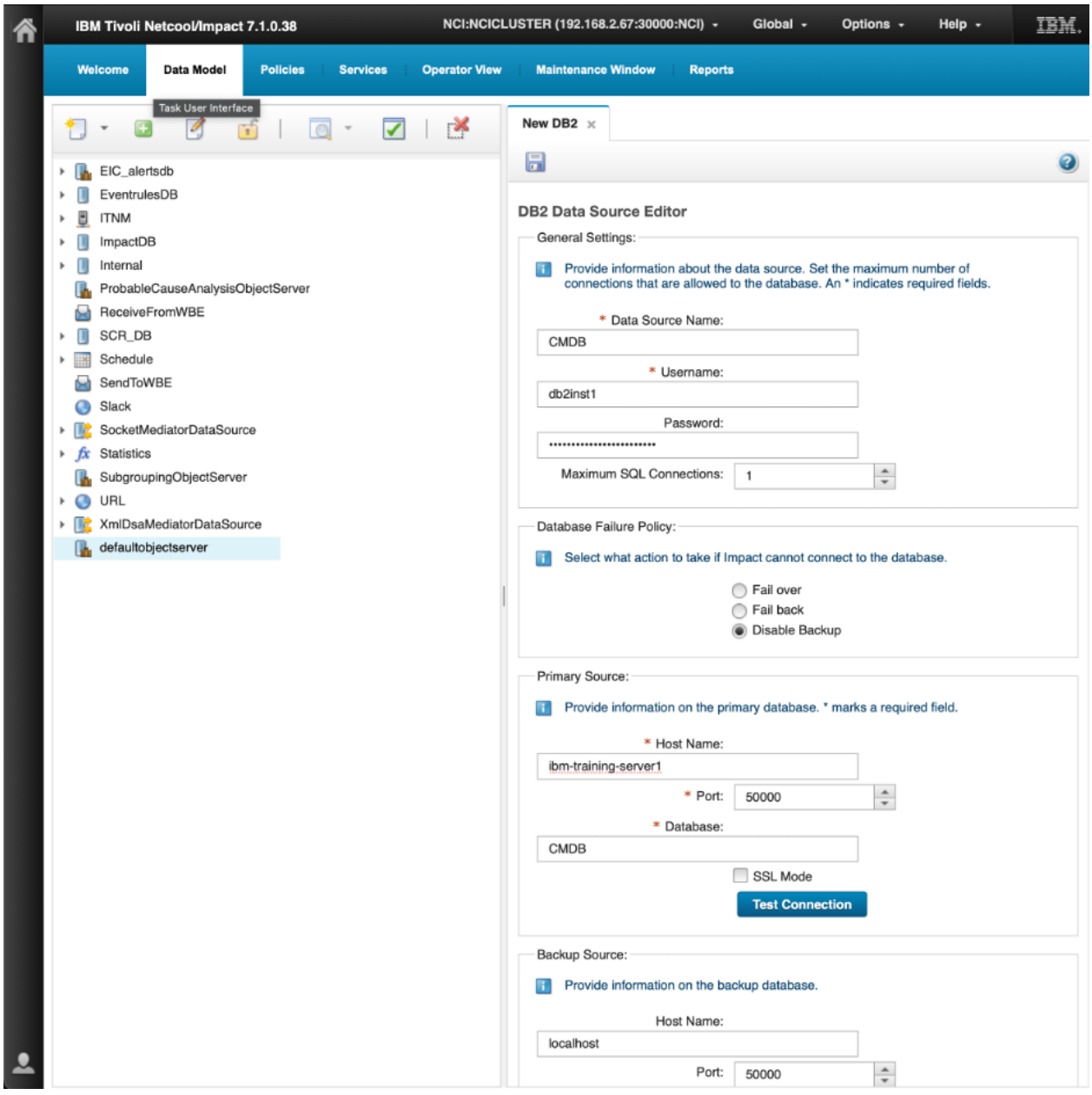


Figure 21: JDBC DSA configuration for CMDB

9. Save the data source by clicking the floppy disk icon at the top left of the configuration tab. Confirm that CMDB now appears in the data source list on the left.
10. Right-click on CMDB in the data source list and select **New Data Type** to create a data type for the enrichment table.

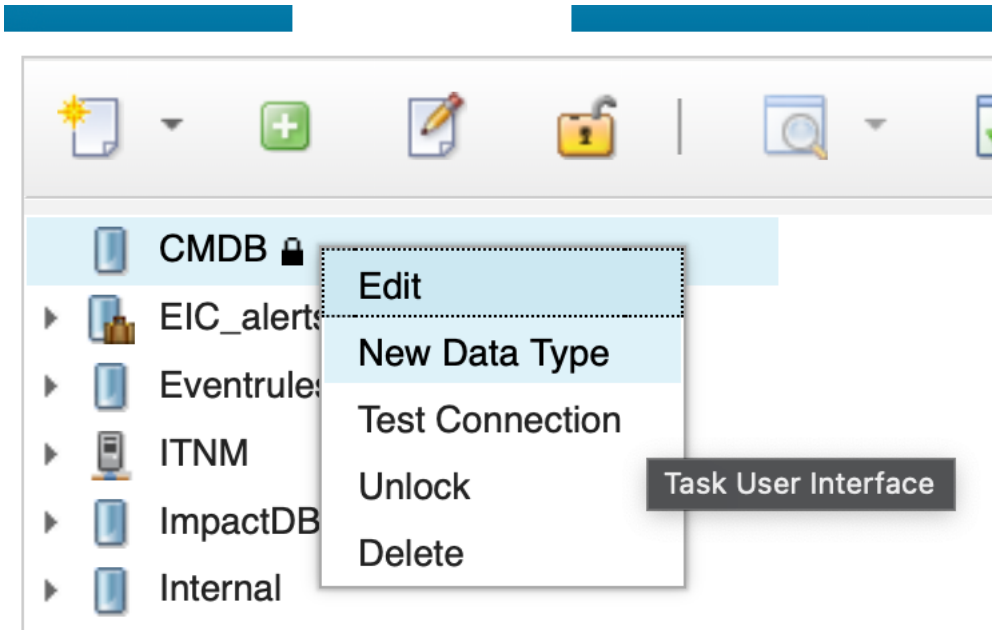


Figure 22: New Data Type for CMDB

11. Configure the new data type on the **SQL Data Type Config Page** as follows:
    - **Data Type Name**: DEVICE_INVENTORY
    - **Data Source Name**: leave as CMDB
    - **Base Table**: DEVICE_INVENTORY

    Click **Refresh Fields** to populate the field list with the table structure. Locate the NODE_NAME field and set its **Key Field** column to **Yes**, since NODE_NAME is the unique identifier that policies will use to look up enrichment data by device hostname.
12. Save the data type by clicking the floppy disk icon. Confirm that DEVICE_INVENTORY appears nested under CMDB in the data source list.

Validation The exercise is successful when all of the following are true:

- The CMDB database exists in DB2 and the DEVICE_INVENTORY table contains eight rows of sample enrichment data.
- The CMDB JDBC DSA is configured and the **Test Connection** result shows **Connection OK**.
- The CMDB data source appears in the Data Model data source list.
- The DEVICE_INVENTORY data type is created with NODE_NAME set as the key field.
- The DEVICE_INVENTORY data type is saved and visible nested under CMDB in the data source list.

Expected result At the end of this exercise, you will have a fully configured enrichment data source in Impact. The CMDB JDBC DSA connects Impact to the DB2 enrichment database, and the DEVICE_INVENTORY data type provides policies with a structured, key-based interface to look up customer and service data by device hostname. This data source and data type will be used directly in the policy enrichment exercises in later sections.

2.3 Validating the CMDB data source and data type

Objective In this exercise, you will validate that the CMDB data source and DEVICE_INVENTORY data type created in section 2.2 are working correctly and are ready for use in policies. You will test the data source connection directly from the Data Model tab, confirm that the data type can return rows from the enrichment table, and review the data type performance report.

Prerequisites

- Completion of section 2.2.

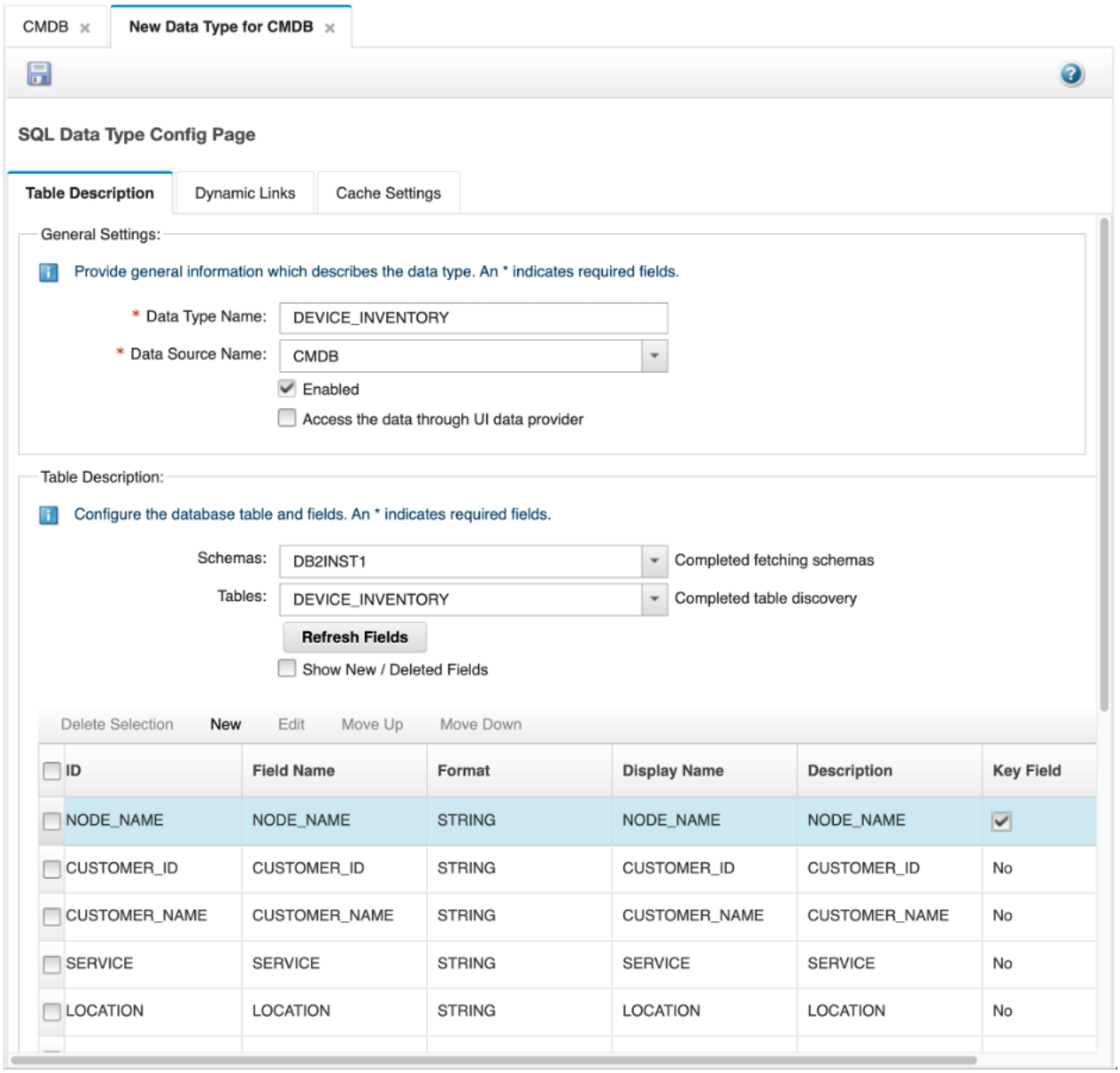


Figure 23: DEVICE_INVENTORY data type configuration

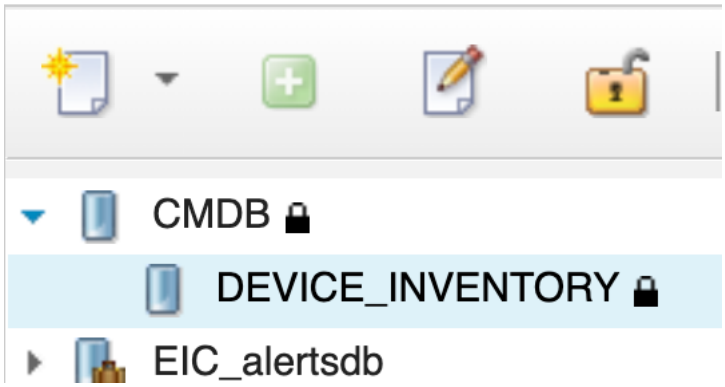


Figure 24: DEVICE_INVENTORY saved under CMDB

- The CMDB JDBC DSA and DEVICE_INVENTORY data type are configured.
- The DB2 instance is running and the CMDB database is available.

Background Before using a data source in a policy, it is good practice to verify that both the data source and its data types are working as expected. In Impact, these are two distinct layers that can be validated independently.

The **data source** represents the physical connection to the external system — in this case, the DB2 CMDB database. Validating the data source confirms that Impact can reach the database using the configured credentials and connection settings. If the data source connection fails, no policy that depends on it will be able to retrieve or write data.

The **data type** sits on top of the data source and defines the structure of a specific table or view. Validating the data type confirms that Impact can query the underlying table and that the field definitions are correct. The **View Data Items** function executes a live query against the table through the data type and returns the current rows, giving you immediate confirmation that the data type is functional and the enrichment data is accessible to policies.

The **Performance Report** provides a view of how the data type has been used over time, including query counts, response times, and error rates. While this is most useful in a production environment where policies are actively querying the data type, reviewing it after initial setup establishes a baseline and confirms that the data type is being tracked by the Impact monitoring framework.

Understanding both layers — and knowing how to validate each one — is an important part of working with Impact data sources. A data source that connects successfully does not guarantee that a data type defined on top of it will return data correctly; the table may not exist, the field mapping may be wrong, or the key field may be misconfigured. Validating each layer separately makes it straightforward to isolate the cause of any issue.

Tasks

1. In the Impact GUI, select the **Data Model** tab from the top navigation bar. In the data source list on the left, locate the CMDB data source entry. To test the connection, either right-click on CMDB and select **Test Connection**, or select CMDB and click the green check icon in the toolbar at the top of the data source list. A popup will appear with the result — confirm that it displays **Connection OK**. If it does not, verify that the DB2 instance is running and that the CMDB database exists before continuing.

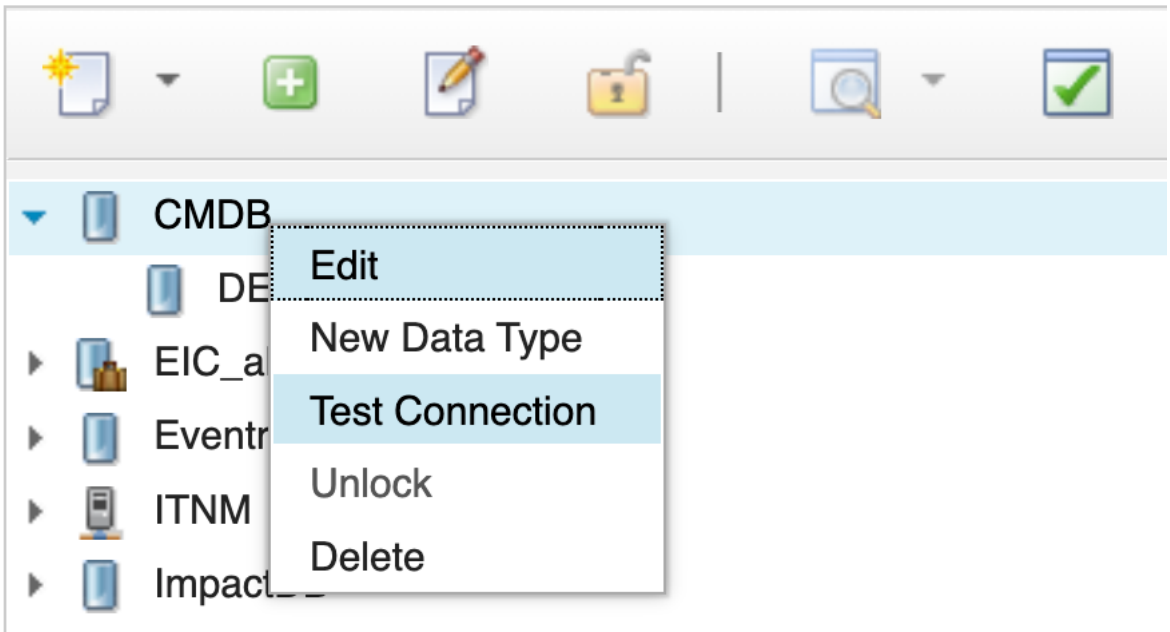


Figure 25: Test Connection result for CMDB

2. Now validate the data type. In the data source list, expand CMDB to reveal the DEVICE_INVENTORY data type nested beneath it. Right-click on DEVICE_INVENTORY and select **View Data Items**.
3. A results panel will open displaying the current rows in the DEVICE_INVENTORY table. Confirm that all eight rows inserted in section 2.2 are returned, with the correct values for NODE_NAME, CUSTOMER_ID, CUSTOMER_NAME, SERVICE, LOCATION, and CONTACT.
4. Right-click on DEVICE_INVENTORY again and select **Show Performance Report**.

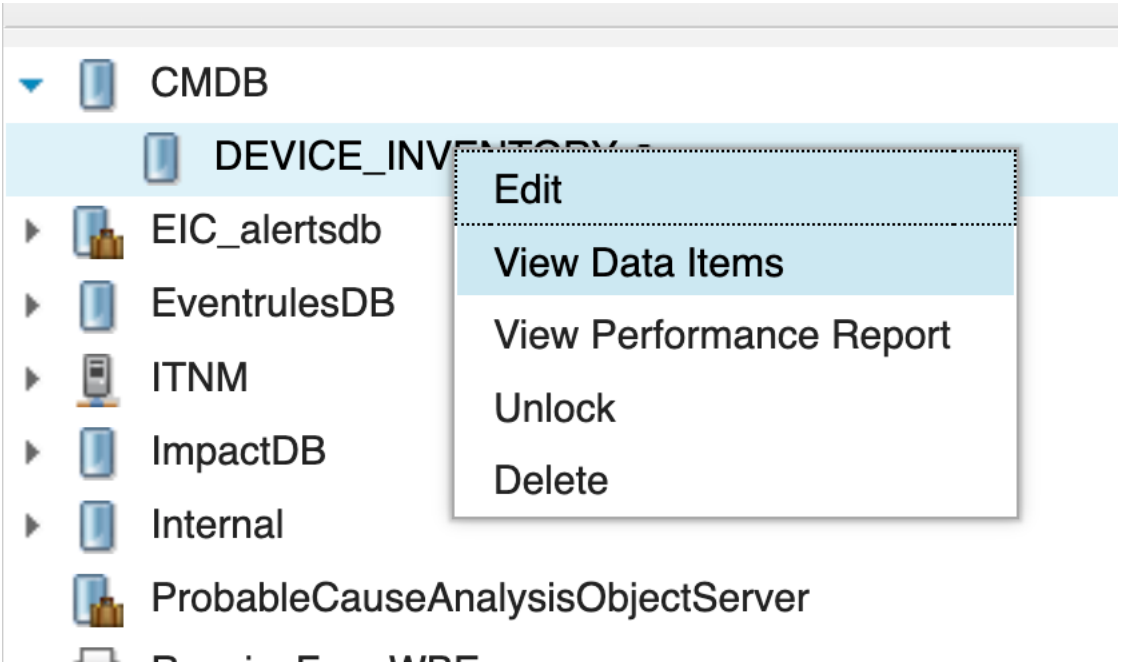


Figure 26: View Data Items option on DEVICE_INVENTORY data type

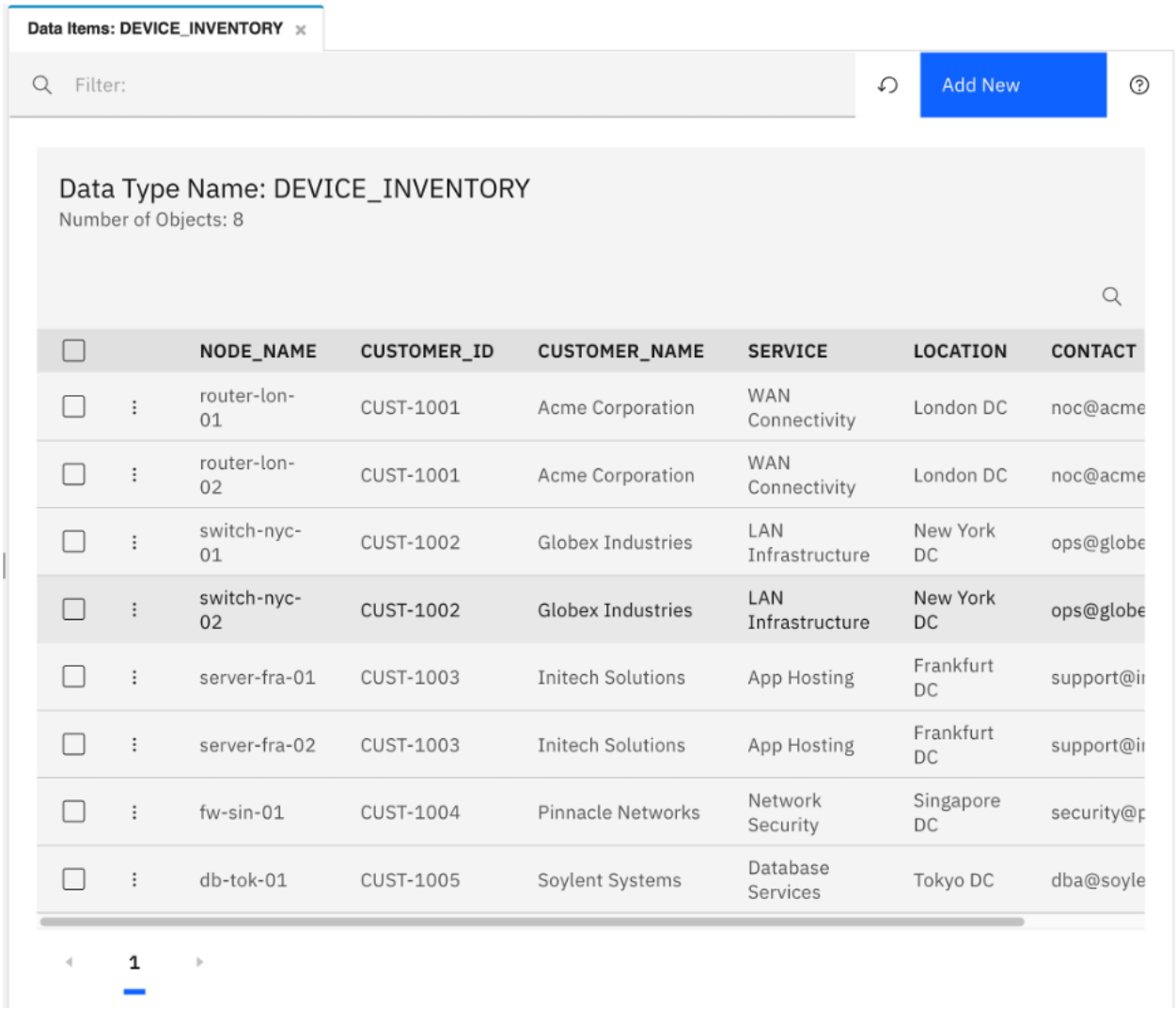


Figure 27: View Data Items results for DEVICE_INVENTORY

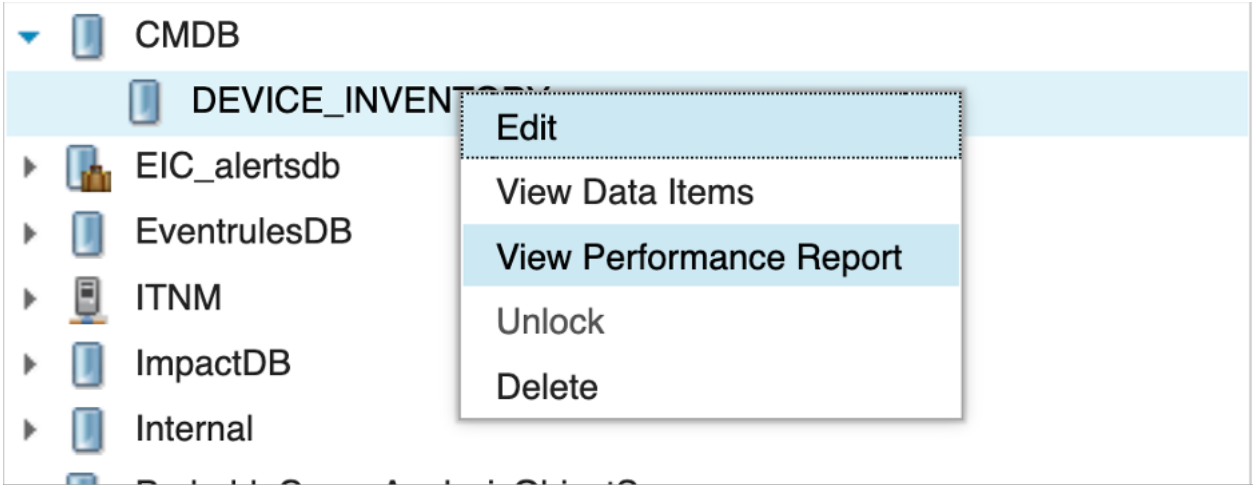


Figure 28: Show Performance Report option on DEVICE_INVENTORY

5. The performance report displays utilization and performance metrics for this data type, including query counts, average response times, and any errors encountered. Review the report and note the available metrics. At this stage the counts will be low since the data type has not yet been used by any policies — this report will become more informative once enrichment policies are actively running in later exercises.

Validation The exercise is successful when all of the following are true:

- Right-clicking `CMDB` and selecting **Test Connection** returns **Connection OK**.
- Right-clicking `DEVICE_INVENTORY` and selecting **View Data Items** returns all eight enrichment rows.
- The **Show Performance Report** option opens the performance report for the data type without error.
- You can explain the difference between validating a data source and validating a data type.

Expected result At the end of this exercise, you will have confirmed that the `CMDB` data source has an active connection to the DB2 database and that the `DEVICE_INVENTORY` data type can successfully retrieve enrichment data. Both layers are now validated and ready to be used by Impact policies in the exercises that follow.

3. Impact policy language (IPL)

The Impact Policy Language (IPL) is the scripting language used to write all Impact automation logic. Every policy in Impact is a block of IPL code that defines what happens when the policy is invoked by a service. IPL is a procedural language — code runs from top to bottom in the order it is written, with conditionals and loops controlling flow.

IPL is purpose-built for event enrichment and automation tasks. It provides direct access to event fields through the triggering event object, built-in functions for querying data sources, string manipulation, date and time handling, logging, and writing results back to the ObjectServer. It does not require knowledge of a general-purpose programming language, but familiarity with basic programming concepts such as variables, conditions, and loops is helpful.

3.1 IPL basics — variables, types, and assignment

Objective In this section, you will be introduced to the fundamental elements of IPL: variables, data types, and assignment syntax. By the end of the section, you will understand how to declare and assign variables, what data types are available, and how to write basic IPL expressions.

Background IPL variables are declared and assigned using the following syntax:

```
variableName = value;
```

There is no explicit type declaration — IPL infers the type from the assigned value. The primary data types used in IPL are:

Type	Description	Example
String	Text value enclosed in double quotes	`"CRITICAL"`
Integer	Whole number	`5`
Float	Decimal number	`3.14`
Boolean	True or false value	`True` or `False`
Array	Ordered list of values	`[1, 2, 3]`
EventContainer	An object holding event field values	Retrieved from the service trigger

Variable names are case-sensitive. A variable assigned in one part of a policy is available for the remainder of that policy's execution. Variables do not persist across separate policy invocations — each time the service triggers the policy, a fresh execution context begins.

Comments are written using the `//` prefix for single-line comments, or using `/* ... */` for block comments that span multiple lines:

```
// This is a single-line comment
nodeName = "router01";   // inline comment after a statement

/*
 * This is a block comment.
 * Use it for policy headers, longer explanations,
 * or to comment out a section of code during testing.
 */
```

Block comments are commonly used at the top of a policy to describe its purpose, prerequisites, and how to run it.

String concatenation Strings are concatenated using the + operator:

```
greeting = "Hello " + "World";
fullKey = nodeValue + ":" + alertKeyValue;
```

Conditional logic IPL uses `if`, `else if`, and `else` for conditional branching:

```
if (severity == 5)
{
    priority = "Critical";
}
elseif (severity >= 3) {
    priority = "High";
}
else
{
    priority = "Normal";
}
```

Comparison operators available in IPL:

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Logical operators:

Operator	Meaning
&&	Logical AND
||	Logical OR
!	Logical NOT

Loops IPL supports `while` loops and `for each` loops for iterating over arrays and result sets:

```
// while loop

counter = 0;

while (counter < 5)
{
    counter = counter + 1;
    log("Counter is " + counter);
}

/// use index and length to iterate over arrays
// arrays are declared in curly braces {}

items = {"a", "b", "c"};

index = 0;
itemCount = Length(items);

while (index < itemCount)
{
    Log("Item: " + item);
    index = index + 1;
}
```

Tasks

1. Open a text editor and write the following IPL variable assignment expressions:

   ```
   // String variable
   hostName = "router01";

   // Integer variable
   retryCount = 3;

   // Boolean variable
   enrichmentRequired = True;

   // String concatenation
   alertKey = hostName + ":interface-down:" + "TrainingAlerts";
   ```

2. Add a conditional block that logs a message based on `retryCount`:

   ```
   if (retryCount > 2)
   {
       Log("Retry threshold reached for host: " + hostName);
   }
   else
   {
       Log("Retry count within normal range: " + retryCount);
   }
   ```

3. Add a `while` loop that iterates over a small array and logs each value:

```
severities = {3, 4, 5};
index = 0;
itemCount = Length(severities);

while (index < itemCount)
{
    Log("Processing severity level: " + sev);
    index = index + 1;
}
```

> **Note:** You do not need to run this code yet. In section 4.1 you will paste policy code directly into the Impact policy editor. These exercises build your familiarity with IPL syntax before you write live policies.

Validation You have completed this section when:

- You can write a valid variable assignment for each of the four primary data types.
- You can write an `if` / `else if` / `else` block using at least two comparison operators.
- You can write a `while` loop that iterates over an array.
- You understand that IPL variables are not explicitly typed — type is inferred from the assigned value.

Expected result You now understand some basic foundational IPL syntax used to create and maintain policies in Impact. The variable, conditional, and loop patterns covered here appear throughout the enrichment and automation policies you will write in section 4.

3.2 Accessing event data in a policy

Objective In this section, you will learn how to access the fields of the triggering alert within a policy. By the end of the section, you will understand how to read event fields, store them in local variables, and use those values in conditional logic.

Background When a service triggers a policy, it passes the current values of the triggering event's fields into the policy execution context. The policy can access these values through the `EventContainer` object, which is available as a special variable that holds all `alerts.status` field values for the event that fired the service.

The standard syntax for reading a field from the triggering event is:

```
localVar = EventContainer.FieldName;
```

For example, to read the `Node`, `Severity`, and `Summary` fields:

```
nodeValue    = EventContainer.Node;
sevValue     = EventContainer.Severity;
summaryValue = EventContainer.Summary;
```

Once stored in local variables, these values can be used in conditions, string operations, data source lookups, and write-back operations.

> **Note:** Field names in `EventContainer` are case-sensitive and must match the exact column names from `alerts.status`. Common fields used in policies include `Node`, `AlertKey`, `AlertGroup`, `Summary`, `Severity`, `Manager`, `Identifier`, and `Service`.

Example policy reading event fields The following is a minimal IPL policy that reads key fields from a triggering event and logs them for inspection:

```
// Read fields from the triggering event
nodeValue    = EventContainer.Node;
sevValue     = EventContainer.Severity;
summaryValue = EventContainer.Summary;
alertKey     = EventContainer.AlertKey;

// Log the values for diagnostic purposes
```

```
Log("Policy triggered for node: " + nodeValue);
Log("Severity: " + sevValue);
Log("Summary: " + summaryValue);
Log("AlertKey: " + alertKey);
```

The Log function writes a message to the Impact policy log. Its syntax is `Log(message)` — pass a single string expression and the message is always written to the policy log file. It is common practice to prepend the policy name to the message string, for example `Log("AA_EnrichAlert: enrichment started for Node=" + nodeValue)`, so that output from different policies is easy to identify when reading the log file or filtering with `grep`.

Tasks

1. Review the field list for the `alerts.status` table used in this lab. The fields you will access in enrichment policies are:

Field name	Type	Description
`Node`	String	Hostname or IP address of the source device
`AlertKey`	String	Unique key for the alert condition
`AlertGroup`	String	Logical grouping for the alert
`Summary`	String	Human-readable description of the alert
`Severity`	Integer	Numeric severity (1=Indeterminate, 2=Warning, 3=Minor, 4=Major, 5=Critical)
`Identifier`	String	Unique identifier combining node, key, and group
`CustomerID`	String	Custom field — populated by the enrichment policy
`Service`	String	Custom field — populated by the enrichment policy

2. Review the IPL code below that reads `Node`, `Severity`, `AlertKey`, and `Summary` from the triggering event and stores each in a local variable:

```
nodeValue    = EventContainer.Node;
sevValue     = EventContainer.Severity;
alertKey     = EventContainer.AlertKey;
summaryValue = EventContainer.Summary;
```

3. The following adds a conditional check that logs different messages depending on severity:

```
if (sevValue >= 4)
{
    Log(2, "High severity event on node: " + nodeValue + ", Key: " + alertKey);
}
else
{
    Log(2, "Event received on node: " + nodeValue + ", Severity: " + sevValue);
}
```

Validation You have completed this section when:

- You can write a valid `EventContainer.FieldName` expression for at least four `alerts.status` fields.
- You understand that field names in `EventContainer` are case-sensitive.
- You can explain what happens if you attempt to read a field that does not exist in `alerts.status`.
- You can use a severity value read from `EventContainer` in a conditional expression.

Expected result You now know how to read any field from the triggering event inside a policy. This is the first step in every enrichment policy: read the event fields you need, then use those values to drive the lookup and write-back logic that follows.

3.3 Querying data sources from IPL

Objective In this section, you will learn about the `GetByFilter` IPL function to query Impact data types. By the end of the section, you will understand how to call the function, process the results, and handle the case where no matching rows are found.

Background IPL provides the `GetByFilter` function to retrieve data items from a configured Impact data type. The function queries the data source associated with the named data type and returns an **array** of matching data items.

`[Array =] GetByFilter(DataType, Filter, [CountOnly]);`

The arguments are:

Argument	Type	Description
`dataTypeName`	String	The Impact data type name as configured in the Data Model tab (not the DSA name or table name)
`filterCondition`	String	A filter expression used to select matching data items. For SQL data sources, this is typically an SQL-style condition such as Location = 'New York'; do not include the WHERE keyword.
`countOnly`	Boolean	Optional. Retained for compatibility with earlier releases. Specify False; it does not return a matching-row count.

For example:

```
DataType = "DEVICE_INVENTORY";
Filter = "Location = 'New York' AND Facility = 'Manhattan'";
MyNodes = GetByFilter(DataType, Filter, False);
```

> **Note:** The first argument to `GetByFilter` is the **Impact data type name** — the name you gave the data type when you created it in section 2.2 (`DEVICE_INVENTORY`). It is not the DSA name and not the underlying database table name. Impact uses the data type configuration to resolve which DSA and table to query.

Checking for empty results Always check the result array length before accessing elements. Accessing index `[0]` on an empty array will cause the policy to throw an error:

```
filterCondition = "NODE_NAME = '" + nodeValue + "'";
results = GetByFilter("DEVICE_INVENTORY", filterCondition, False);

if (Length(results) > 0) {
    firstRow = results[0];
    customerID = firstRow.CUSTOMER_ID;
    service = firstRow.SERVICE;

    Log("Found inventory record for node: " + nodeValue
        + ", customer: " + customerID
        + ", service: " + service);
} else {
    Log("No inventory record found for node: " + nodeValue);
}
```

Building filter strings safely When constructing a filter expression from a string value, ensure that embedded single quotes do not terminate the quoted value prematurely. For SQL-backed data types, a single quote within a string literal is represented by two single quotes.

For example, if nodeValue is it's-router-01, the value must become it''s-router-01 before it is included in the filter expression:

```
nodeFilterValue = Replace(nodeValue, "'", "''", 0);
filterCondition = "NODE_NAME = '" + nodeFilterValue + "'";
results = GetByFilter("DEVICE_INVENTORY", filterCondition, False);
```

The final filter expression then becomes:

```
NODE_NAME = 'it''s-router-01'
```

The final 0 argument tells Replace() to replace every matching occurrence. Replace() uses regular expressions for its pattern argument and requires the expression, pattern, replacement value, and maximum number of replacements.

Tasks

1. Review the following IPL code along with its comments. Note the `GetByFilter` IPL function call that looks up `nodeValue` in the `DEVICE_INVENTORY` data type. Store the result and guard against an empty response:

```
// Read the Node value from the event that invoked this policy.
// EventContainer is the policy context object containing the incoming event fields.
nodeValue = EventContainer.Node;


// Replace embedded single quotes in the node name with two single quotes.
// For an SQL-backed data type, this prevents a single quote from prematurely
// ending the quoted value in the filter expression.
nodeFilterValue = Replace(nodeValue, "'", "''", 9999);


// Build the GetByFilter expression used to locate an inventory row whose
// NODE_NAME field exactly matches the event's Node value.
//
// Example:
// nodeValue = "router01"
// filter = "NODE_NAME = 'router01'"
//
// If nodeValue = "it's-router-01", the escaped filter becomes:
// "NODE_NAME = 'it''s-router-01'"
filter = "NODE_NAME = '" + nodeFilterValue + "'";


// Query the configured Impact data type named DEVICE_INVENTORY.
// The result is an array containing zero or more matching data items.
// False is the optional CountOnly argument and should normally be specified
// as False; it is retained for backward compatibility.
results = GetByFilter("DEVICE_INVENTORY", filter, False);


// Check that at least one inventory record was returned before attempting
// to access the first element of the result array.
if (Length(results) > 0)
{
    // Retrieve the first matching data item from the returned array.
    // If duplicate NODE_NAME values exist, this selects only the first row.
    inventoryRow = results[0];

    // Copy the CUSTOMER_ID field from the matching inventory record
    // into a policy variable for later use.
    customerID = inventoryRow.CUSTOMER_ID;

    // Copy the SERVICE field from the matching inventory record
    // into a policy variable for later use.
    serviceName = inventoryRow.SERVICE;
```

```
    // Write a diagnostic message at log level 2. This message is recorded only
    // when the Policy Logger service is configured with a highest log level
    // of 2 or greater.
    Log("Inventory record found for node: " + nodeValue);
}
else
{
    // Write a level-2 diagnostic message when no DEVICE_INVENTORY record
    // matches the event Node value.
    Log("No inventory record for node: " + nodeValue);
}
```

2. Review this `GetByFilter` call that retrieves all events from the ObjectServer data type EVENTS where `Severity` equals 5. Log the count of matching rows:

```
criticalEvents = GetByFilter("defaultobjectserver", "Severity = 5", False);

Log("Critical events found: " + Length(criticalEvents));
```

3. Extend the example from task 2 to iterate over the results and log the `Node` and `Summary` for each row:

```
criticalEvents = GetByFilter("defaultobjectserver", "Severity = 5", False);

if (Length(criticalEvents) > 0)
{
    for each event in criticalEvents
    {
        Log("Critical: Node=" + event.Node + ", Summary=" + event.Summary);
    }
}
else
{
    Log("No critical events found.");
}
```

> **Note:** You do not need to run these snippets yet. You will use these patterns directly in the policies you build in section 4.

Validation You have completed this section when:

- You can write a `GetByFilter` call with the correct three-argument signature.
- You understand that the first argument is the Impact data type name, not the DSA or table name.
- You can check the length of a `GetByFilter` result before accessing its elements.
- You can explain why single quotes in field values must be escaped before building a filter string.
- You can iterate over a `GetByFilter` result array using a `for each` loop.

Expected result You now understand the data retrieval function and the empty-check pattern required to use it safely. These patterns form the lookup step in every enrichment policy — reading event fields is step one; looking up matching inventory data is step two.

3.4 Writing values back to the ObjectServer

Objective In this section, you will learn how to write values from a policy back to the OMNIbus ObjectServer by updating fields on the triggering alert. By the end of the section, you will understand how to assign new values directly to `EventContainer` fields and use `ReturnEvent` to commit those changes back to the ObjectServer.

Background The primary purpose of most Impact enrichment policies is to update alert fields in `alerts.status` with data retrieved from an external source. This write-back operation is performed in two steps:

1. **Assign** the new value directly to the `EventContainer` field using standard assignment syntax.
2. **Commit** the changes back to the ObjectServer by calling `ReturnEvent(EventContainer)`.

The assignment syntax mirrors the read syntax from section 3.2:

```
EventContainer.FieldName = newValue;
```

For example, after looking up customer data from the inventory database, the following code writes the customer ID and service name back to the alert:

```
EventContainer.CustomerID = customerID;
EventContainer.Service    = serviceName;
```

After all assignments are complete, call `ReturnEvent` to push the updated `EventContainer` back to the ObjectServer:

```
ReturnEvent(EventContainer);
```

Without `ReturnEvent`, the changes exist only in the local execution context — the ObjectServer alert is not updated.

Important considerations

- **Field names are case-sensitive**: `CustomerID` is not the same as `customerid`. The field name must exactly match the column name in `alerts.status`.
- **Custom fields must exist**: Fields written to `EventContainer` must already exist as columns in `alerts.status`. Writing to a non-existent field will not create the column. The `ImpactFlag`, `CustomerID`, and `Service` fields added in earlier exercises are examples of custom fields that must be created before Impact can write to them.
- **Only the triggering event is updated**: `ReturnEvent(EventContainer)` updates the specific alert that triggered the policy. To update other alerts, a separate `GetByFilter` against the ObjectServer data type followed by explicit assignments on each returned row is required.
- **Call `ReturnEvent` once, after all assignments**: Assign all the field values you need first, then call `ReturnEvent` a single time to commit the complete set of changes in one operation.

```
// Read the node from the triggering event
nodeValue = EventContainer.Node;

// Look up customer information in the inventory database
nodeFilterValue = Replace(nodeValue, "'", "''");
filter = "NODE_NAME = '" + nodeFilterValue + "'";
results = GetByFilter("DEVICE_INVENTORY", filter, False);

if (Length(results) > 0) {
    inventoryRow = results[0];

    // Assign enrichment fields directly to EventContainer
    EventContainer.CustomerID  = inventoryRow.CUSTOMER_ID;
    EventContainer.Service     = inventoryRow.SERVICE;
    EventContainer.ImpactFlag  = 1;

    Log(2, "Enriched alert for node: " + nodeValue
        + " with customer: " + inventoryRow.CUSTOMER_ID);
} else {
    EventContainer.CustomerID  = "NOT_FOUND";
    EventContainer.Service     = "NOT_FOUND";
    EventContainer.ImpactFlag  = 2;

    Log(2, "No inventory record found for node: " + nodeValue
        + " — enrichment skipped");
}

// Commit all changes back to the ObjectServer
ReturnEvent(EventContainer);
```

Example write-back snippet

Tasks

1. Review the important considerations listed in the background section. In your own words, answer the following:
   - What happens if you assign a value to an `EventContainer` field that does not exist as a column in `alerts.status`?
   - Can you use `ReturnEvent` to update a different alert (not the one that triggered the policy)? If not, what would you need to do instead?
   - Why should all field assignments be made before calling `ReturnEvent` rather than calling `ReturnEvent` after each individual assignment?

Validation You have completed this section when:

- You can write a complete enrichment write-back block using `EventContainer.Field = value` assignment syntax.
- You understand that `ReturnEvent(EventContainer)` is required to commit the changes to the ObjectServer.
- You understand that field names in `EventContainer` are case-sensitive and must match existing `alerts.status` columns.
- You can explain the difference between updating the triggering event and updating another alert.
- You can describe the read → look up → assign → `ReturnEvent` pattern that all enrichment policies follow.

Expected result You now know how to write data back to the ObjectServer from a policy. Combined with reading event fields (section 3.2) and querying data sources (section 3.3), you have all three building blocks of a complete enrichment policy. Section 4 puts these blocks together into policies that run against live event data.

3.5 Built-in IPL functions reference

The following table summarizes some commonly used IPL built-in functions. These functions will be used throughout the policy-building exercises in section 4.

Function	Signature	Description
`Log`	`Log(message)`	Writes a message to the Impact policy log. Pass a string expression; the message is always written to the policy log file.
`GetByFilter`	`GetByFilter(dataTypeName, filter, [countOnly])`	Returns an array of data items from the named Impact data type that match the filter expression. The optional countOnly parameter is retained for compatibility; specify False for normal retrieval. The filter is interpreted by the DSA associated with the data type.
`ReturnEvent`	`ReturnEvent(EventContainer)`	Returns an event container to its event source. For an incoming ObjectServer event, this commits updates made to event fields; it can also be used to add a journal entry or delete an event when the appropriate EventContainer state variables are set.
`Replace`	`Replace(string, pattern, replacement, maxReplacements)`	Returns a new string after replacing occurrences that match the specified regular-expression pattern. The final argument controls the maximum number of replacemednts.
`Length`	`Length(array)`	Returns the number of elements in an array. Use to check `GetByFilter` results before accessing elements.
`ToUpper`	`ToUpper(str)`	Returns the input string converted to uppercase.
`ToLower`	`ToLower(str)`	Returns the input string converted to lowercase.
`Trim`	`Trim(str)`	Returns the input string with leading and trailing whitespace removed.

Function	Signature	Description
Contains	Contains(str, substr)	Returns True if str contains substr, otherwise False.
GetDate	GetDate()	Returns the current date/time value as seconds since the UNIX epoch. It is commonly passed to LocalTime() for formatting.
LocalTime	LocalTime(timestamp, [pattern]])	Converts an epoch time value to a formatted local date/time string.
IsNull	IsNull(value)	Returns True if the value is NULL, otherwise False.
ToString	ToString(value)	Converts a non-string value to its string representation.
ToInteger	ToInteger(str)	Converts a string to an integer value.

This is only a subset of all functions available in Impact Policy Language. For a complete listing, consult the Netcool/Impact functions list available at the following link: https://www.ibm.com/docs/en/tivoli-netcoolimpact/7.1.0?topic=policies-working-functions

4. Event enrichment

> **Before you begin:** This section involves writing IPL policy code. If you have not already done so, review section 3 (Impact Policy Language) before starting — in particular sections 3.2 (accessing event data), 3.3 (querying data sources), and 3.4 (writing values back to the ObjectServer).

Event enrichment is the most common use case for Impact in OMNIbus environments. When a probe inserts a new alert into the ObjectServer, the alert typically contains only what the probe could detect — the node name, a summary message, and a severity. It has no knowledge of which customer owns that node, which business service it supports, or who is responsible for it. Enrichment solves this by automatically looking up that context from an inventory or CMDB source and writing it back to the alert, so operators see a fully populated event the moment it arrives.

The enrichment pattern has three steps:

1. **Read** — the policy reads key fields from the triggering alert (for example, Node).
2. **Look up** — the policy queries an inventory data source to find the record matching that node.
3. **Write back** — the policy writes the returned fields (CustomerID, Service, etc.) back to the alert in the ObjectServer.

Workflow control with flag fields

A key design question in any enrichment implementation is: how does the service know which events still need to be processed? Filtering on Severity > 0 would match every active alert — including ones that have already been enriched — causing the policy to run repeatedly on the same events.

The solution is a dedicated **workflow flag field** on alerts.status. This pattern is common across Impact deployments and is not limited to enrichment:

- An enrichment flag controls which events have been looked up in the CMDB.
- A ticketing flag tracks whether a ticket has been raised for an alert.
- A notification flag records whether an on-call notification has been sent.
- A maintenance flag marks alerts that were suppressed during a planned outage window.

In each case, the flag starts at a default value (typically 0) for every new event, and the policy updates it when it acts on the alert. The service filter targets only events where the flag is still at its default value, ensuring each event is processed exactly once.

In this lab, you will add an ImpactFlag integer column to alerts.status with a default value of 0. The enrichment policy will set it to 1 when enrichment succeeds, and to 2 when enrichment is attempted but no inventory record is found. You will then create a service — the component in Impact that watches the ObjectServer and triggers policies — configured to match only alerts where ImpactFlag = 0, so that each event is passed to the enrichment policy exactly once.

This section walks through the complete end-to-end implementation: add the flag field, write the policy, configure the service, connect them, and validate the result with live test events.

4.1 Write the enrichment policy

Objective In this exercise, you will create the enrichment policy in the Impact GUI. By the end of the exercise, you will have a saved, enabled policy that reads the Node field from the triggering alert, looks up the matching record in the DEVICE_INVENTORY table, and writes CustomerID and Service back to the alert.

Prerequisites

- Completion of sections 1, 2, and 3.
- The defaultobjectserver ObjectServer DSA is configured and connected (section 2.1).
- The EVENTS data type is configured in the Data Model, using the defaultobjectserver DSA against the alerts.status table (section 2.1).
- The CMDB DB2 DSA is configured and connected (section 2.2).
- The OMNIbus ObjectServer is running.

Tasks

1. SSH into the lab server and add the ImpactFlag column to the alerts.status table. This column will be used to track whether each alert has been processed by Impact:

```
$OMNIHOME/bin/nco_sql -server EVENTS -user root -password netcool << 'EOF'
ALTER TABLE alerts.status ADD COLUMN ImpactFlag INTEGER;
go
EOF
```

   Note: Integer fields that are not explicitly set by a probe rule are automatically populated with 0 when an event is inserted into the ObjectServer. This means any new alert will have ImpactFlag = 0 without requiring any changes to probes or gateways.

2. Verify the column was added:

```
$OMNIHOME/bin/nco_sql -server EVENTS -user root -password netcool << 'EOF'
DESCRIBE alerts.status;
go
EOF
```

   Confirm that ImpactFlag appears in the column list with type integer.

3. Open a **private/incognito browser window** and navigate to the Impact GUI at http://<your-server>:17311/ibm/console. Log in as impactadmin with password netcool.

   Note: A private window is used throughout this lab to keep the Impact GUI session separate from other browser sessions. Keep this window open for all remaining tasks in this section.

4. Select the **Policies** tab from the top navigation bar. The policies list will appear in the left pane. In the toolbar at the top of the list, click the **dropdown arrow** next to the New Policy icon and select **IPL Policy**.

5. A blank IPL code editor opens titled **New IPL Policy**. Before entering any code, click the **Save** icon (floppy disk, top-left). A **Save** dialog appears with a single **Policy Name** field. Enter AA_EnrichAlert and click **Save**.

   Note: The AA_ prefix causes the policy to sort to the top of the alphabetical policy list, making it easy to find during this lab. This is a common convention when working in shared Impact environments that contain many pre-existing policies.

6. The editor tab updates to show AA_EnrichAlert. Notice that the policy now appears at the top of the policy list in the left pane with a **lock icon** (🔒) next to its name. The lock indicates that the policy is currently open for editing by you. Other users cannot edit it while it is locked. The lock is released when you close the editor tab.

   Now enter the following IPL code in the editor:

```
// Policy: AA_EnrichAlert
//
// Prerequisites:
//   - CustomerID, Service, and ImpactFlag exist in the ObjectServer table.
//   - Those fields are included in the OMNIbusEventReader event mapping.
```

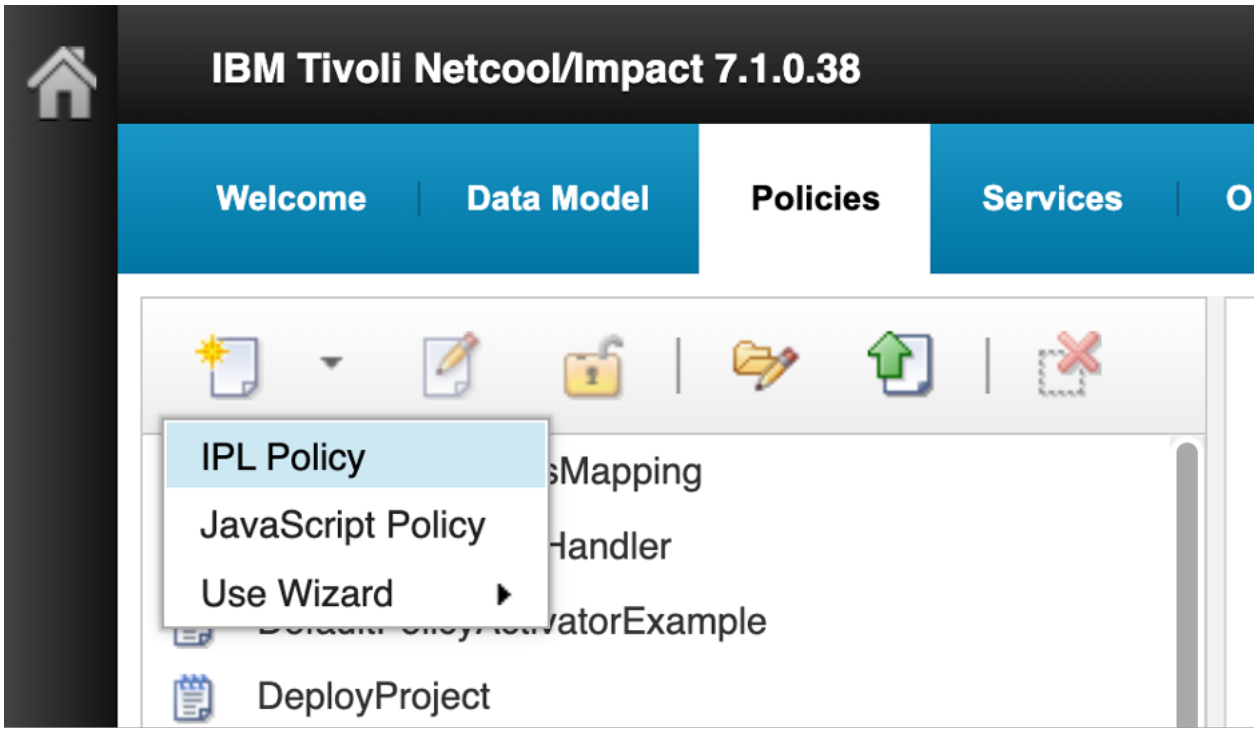


Figure 29: New Policy dropdown with IPL Policy selected

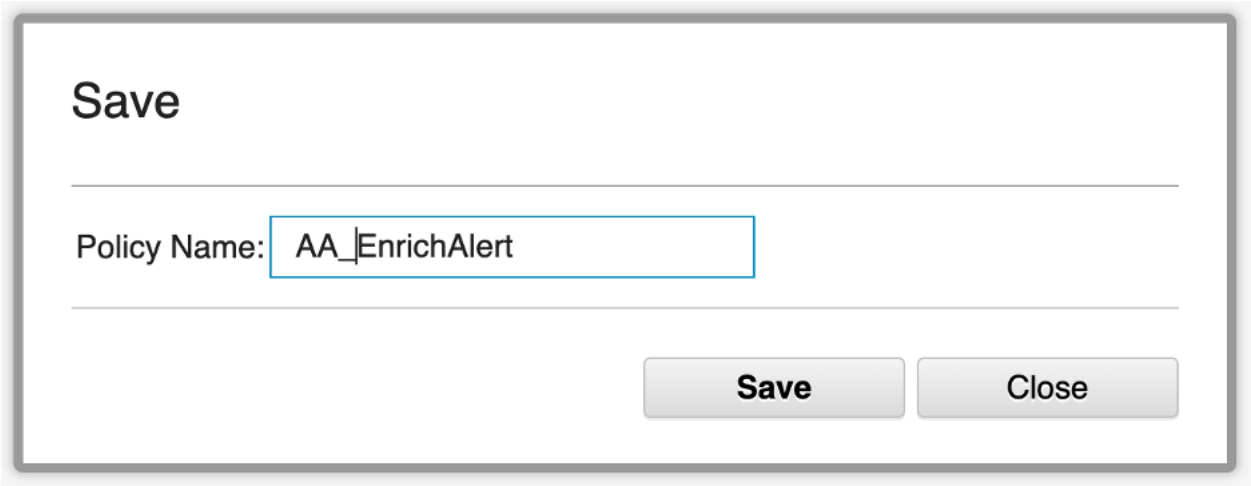


Figure 30: Save dialog with Policy Name AA_EnrichAlert

```
//   - DEVICE_INVENTORY is an Impact data type using the CMDB datasource.
//   - DEVICE_INVENTORY exposes NODE_NAME, CUSTOMER_ID, and SERVICE.

if (EventContainer.ImpactFlag != 0) {
    Log("AA_EnrichAlert: Serial=" + EventContainer.Serial
        + " already processed; ImpactFlag=" + EventContainer.ImpactFlag);
} else {

    nodeValue = EventContainer.Node;

    Log("AA_EnrichAlert: enrichment started for Serial="+ EventContainer.Serial + ", Node="
 ↪  + nodeValue);

    // Escape embedded single quotes before building the SQL filter string.
    nodeFilterValue = Replace(nodeValue, "'", "''");
    filter = "NODE_NAME = '" + nodeFilterValue + "'";

    // GetByFilter(DataType, Filter, CountOnly)
    inventoryRows = GetByFilter("DEVICE_INVENTORY", filter, False);

    if (Length(inventoryRows) > 0) {
        inventoryRow = inventoryRows[0];

        EventContainer.CustomerID = inventoryRow.CUSTOMER_ID;
        EventContainer.Service = inventoryRow.SERVICE;
        EventContainer.ImpactFlag = 1;

        Log("AA_EnrichAlert: enrichment applied for Serial=" + EventContainer.Serial
            + ", CustomerID=" + EventContainer.CustomerID
            + ", Service=" + EventContainer.Service);
    } else {
        EventContainer.CustomerID = "NOT_FOUND";
        EventContainer.Service = "NOT_FOUND";
        EventContainer.ImpactFlag = 2;

        Log("AA_EnrichAlert: no inventory record found for Serial="
            + EventContainer.Serial + ", Node=" + nodeValue
            + "; ImpactFlag set to 2");
    }

    ReturnEvent(EventContainer);
}
```

7. Before saving, click the **Syntax Check** icon in the toolbar (the green checkmark icon). The syntax checker parses the IPL code and reports any errors — mismatched braces, invalid function calls, or malformed expressions — without executing the policy. A successful check confirms the code is syntactically valid and ready to run. An **Information** dialog will appear with the message **Syntax check successful. No errors found.** Click **Close**.
8. Click the **Save** icon to save the policy code. Confirm that `AA_EnrichAlert` appears at the top of the policies list in the left pane.

Validation The exercise is successful when all of the following are true:

- The `ImpactFlag` column exists in `alerts.status` with type integer.
- The `AA_EnrichAlert` policy is saved and visible at the top of the policies list.
- You can explain the purpose of the `if (@ImpactFlag != 0)` guard at the top of the policy.
- You can describe the difference between `@FieldName` (direct field assignment) and the `GetByFilter` / `ReturnEvent` pattern used here.
- You can explain what `ImpactFlag` is set to in each outcome and why both non-zero values prevent re-processing.

AA_EventEnrichment

```
// Policy: AA_EnrichAlert
//
// Prerequisites:
//   - CustomerID, Service, and ImpactFlag exist in the ObjectServer table.
//   - Those fields are included in the OMNIbusEventReader event mapping.
//   - DEVICE_INVENTORY is an Impact data type using the CMDB datasource.
//   - DEVICE_INVENTORY exposes NODE_NAME, CUSTOMER_ID, and SERVICE.

if (EventContainer.ImpactFlag != 0) {
    Log(2, "AA_EnrichAlert: Serial=" + EventContainer.Serial
        + " already processed; ImpactFlag=" + EventContainer.ImpactFlag);
} else {

    nodeValue = EventContainer.Node;

    Log(2, "AA_EnrichAlert: enrichment started for Serial="
        + EventContainer.Serial + ", Node=" + nodeValue);

    // Escape embedded single quotes before building the SQL filter string.
    nodeFilterValue = Replace(nodeValue, "'", "''");
    filter = "NODE_NAME = '" + nodeFilterValue + "'";

    // GetByFilter(DataType, Filter, CountOnly)
    inventoryRows = GetByFilter("DEVICE_INVENTORY", filter, False);

    if (Length(inventoryRows) > 0) {
        inventoryRow = inventoryRows[0];

        EventContainer.CustomerID = inventoryRow.CUSTOMER_ID;
        EventContainer.Service = invent                    E;
        EventContainer.ImpactFlag = 1;

        Log(2, "AA_EnrichAlert: enrichment applied for Serial="
            + EventContainer.Serial
            + ", CustomerID=" + EventContainer.CustomerID
            + ", Service=" + EventContainer.Service);
    } else {
        EventContainer.CustomerID = "NOT_FOUND";
        EventContainer.Service = "NOT_FOUND";
        EventContainer.ImpactFlag = 2;

        Log(2, "AA_EnrichAlert: no inventory record found for Serial="
            + EventContainer.Serial + ", Node=" + nodeValue
            + "; ImpactFlag set to 2");
    }

    ReturnEvent(EventContainer);
}
```

Task User Interface

Figure 31: EnrichAlert policy code in the editor

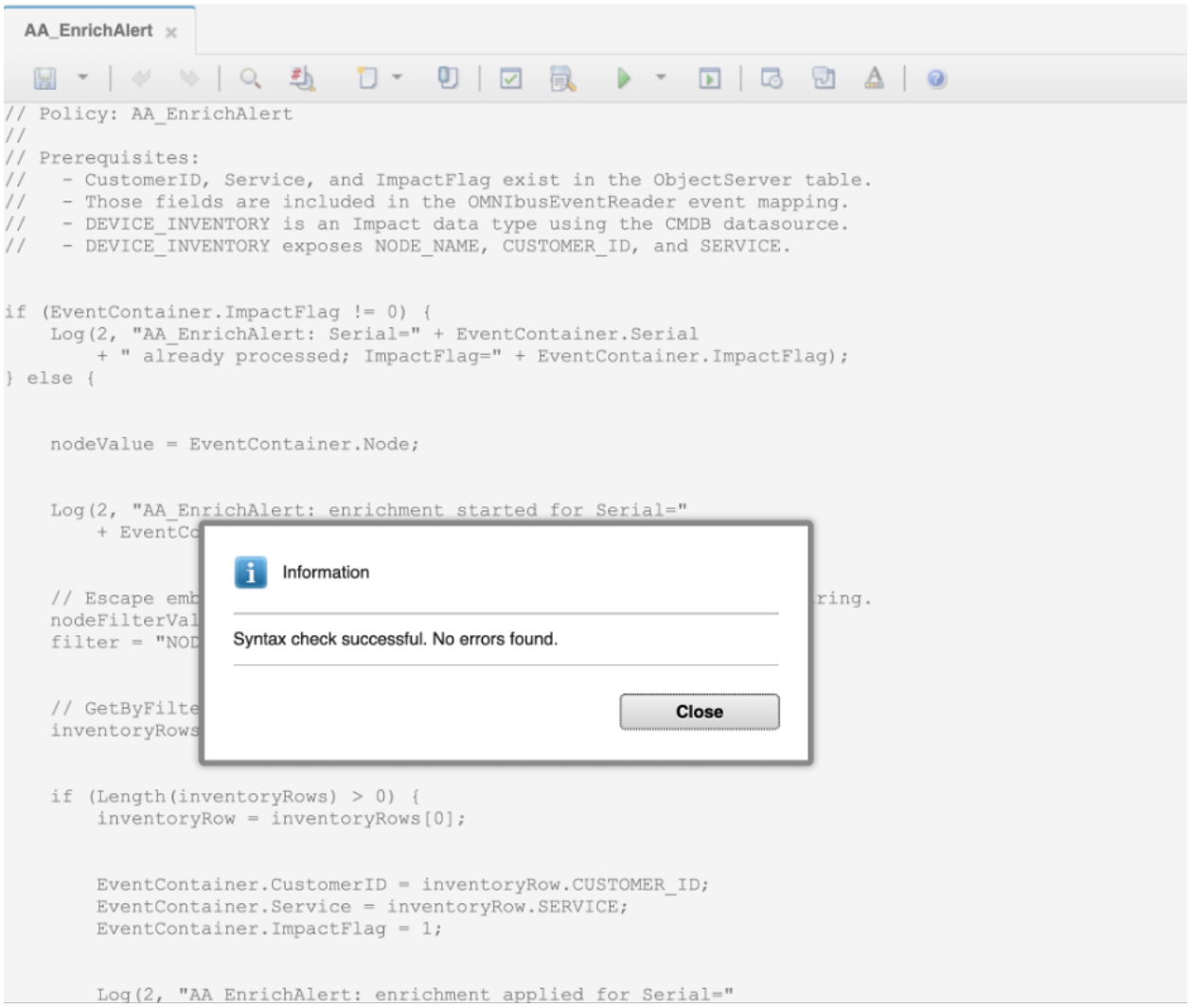


Figure 32: Syntax check successful dialog

Expected result You now have a complete enrichment policy saved in Impact. It is not yet connected to a service, so it will not run automatically. The next exercise creates the OMNIbusEventReader service and wires it to this policy.

4.2 Create and configure the OMNIbusEventReader service

Objective In this exercise, you will create an OMNIbusEventReader service, configure it to poll the ObjectServer, and assign the `AA_EnrichAlert` policy to it. By the end of the exercise, the service will be running and ready to trigger the policy when new alerts arrive.

Prerequisites

- Completion of section 4.1.
- The `AA_EnrichAlert` policy is saved and enabled.

Background A policy on its own does nothing — it needs a service to trigger it. Impact supports several service types, each designed for a different trigger source or integration pattern:

Service type	Trigger source	Typical use
OMNIbusEventReader	OMNIbus ObjectServer (`alerts.status`)	Event enrichment, correlation, escalation, ticket automation, suppression, and lifecycle processing. The standard service type for OMNIbus-to-Impact integrations.
DatabaseEventReader	External SQL database table or view	Polling a CMDB, inventory, or OSS table; importing workflow items; or reacting to externally staged events.
PolicyActivator	Time / schedule	Periodic housekeeping, aged-event escalation checks, cache refresh, CMDB synchronization, and reconciliation jobs.
KafkaMessageListener	Apache Kafka topics	Modern streaming and event-bus integration — telemetry, application events, or a CP4AIOps-adjacent event pipeline.
JMSMessageListener	JMS topic or queue	IBM MQ / WebSphere enterprise messaging integration.
EmailReader	POP3 or IMAP mailbox	Email-based acknowledgements, approval workflows, or ingestion from legacy systems that only emit email.
EventListener	Impact event-listener endpoint	Custom integrations where an external application submits an Impact-compatible event directly.
OMNIbusEventListener	OMNIbus AEN fast-track channels	Specialized low-latency handling for explicitly fast-tracked ObjectServer events. Legacy or special-purpose use.

For enrichment, the **OMNIbusEventReader** is the right choice. When an alert appears or changes in `alerts.status` and matches the service's filter, the service passes all fields of that alert to the assigned policy through the `EventContainer` object. The service configuration has two tabs:

- **General Settings**: sets the service name, the ObjectServer data source to poll, and the polling interval.
- **Event Mapping**: defines one or more filter conditions that determine which events are matched, and maps each filter to the policy that should run when it matches. All fields of a matching event are automatically passed to the assigned policy.

Tasks

1. Select the **Services** tab from the top navigation bar. The services list will appear in the left pane.
2. In the toolbar at the top of the services list, click the **dropdown arrow** next to the New Service icon and select **OMNIbusEventReader**. The **New OMNIbusEventReader Service** configuration page opens showing the **General Settings** tab.

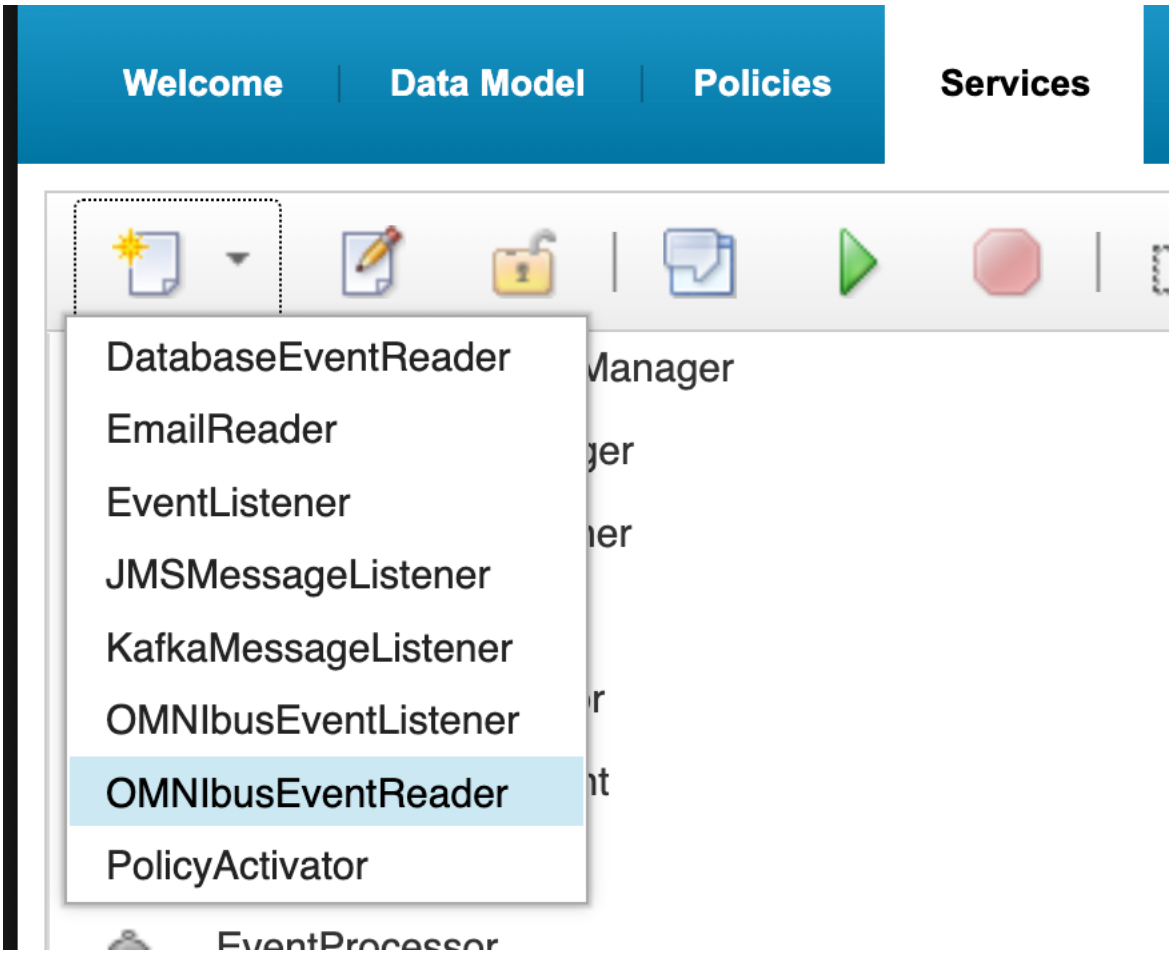


Figure 33: New Service dropdown with OMNIbusEventReader selected

3. On the **General Settings** tab, configure the following:
   - **Service Name**: `EnrichmentEventReader`
   - **Data Source Name**: `defaultobjectserver`
   - **Polling Interval (milliseconds)**: `3000` (default)
   - Leave **Starts automatically when server starts** unchecked.

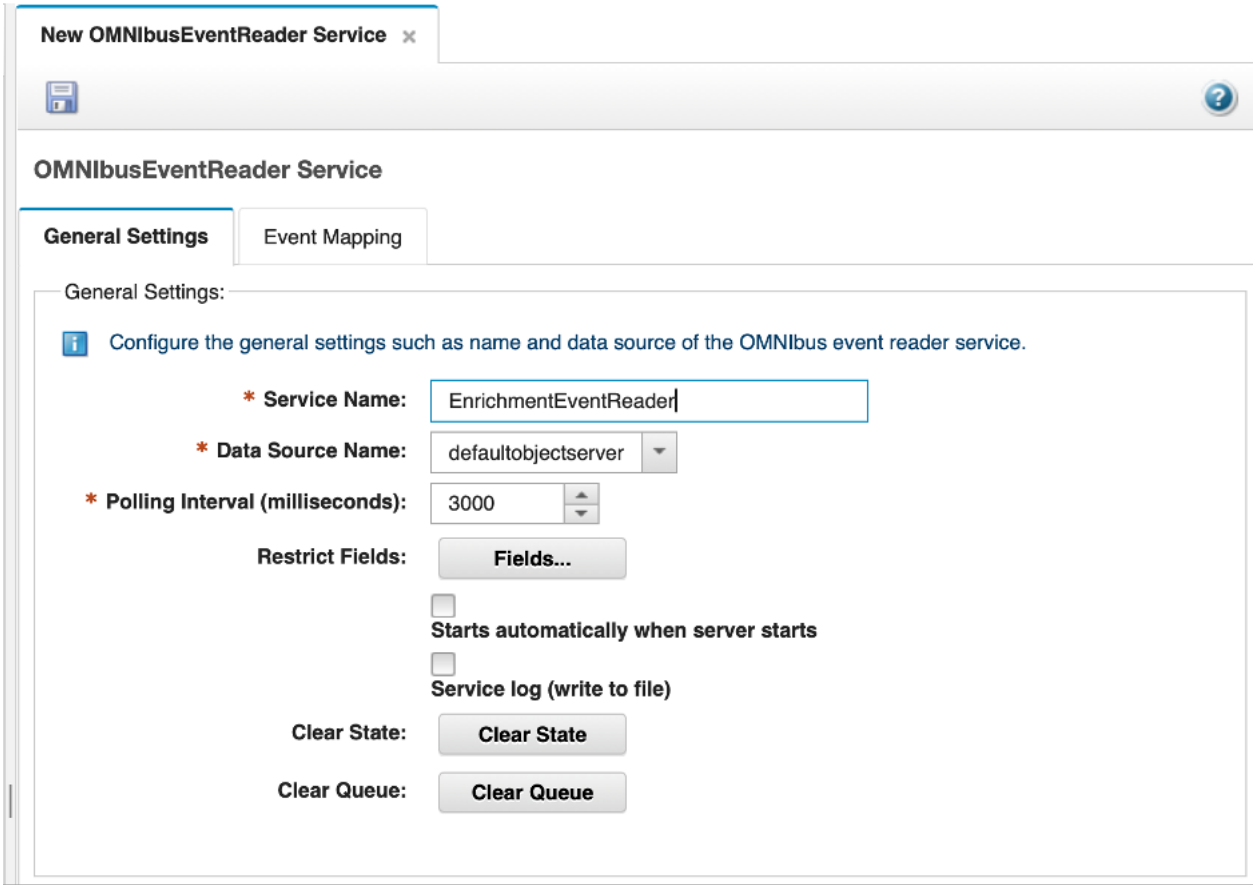


Figure 34: General Settings tab configured

4. Click the **Event Mapping** tab. Review the sections:
   - **Event Matching**: leave set to **Stop testing after first match**.
   - **Action**: leave **Get updated events** and **Get Status Events** unchecked — the service will process newly created events only.
   - **Deletion** and **Event Locking**: leave at defaults.
5. In the **Event Mapping Table**, click **New** to add a mapping row. Configure the row as follows:
   - **Restriction Filter**: `ImpactFlag = 0`
   - **Policy Name**: select `AA_EnrichAlert` from the dropdown
   - **Active**: ensure this is checked

   Click **OK** to confirm the row.

   > **Note:** The filter `ImpactFlag = 0` targets only alerts that have not yet been processed by Impact. Once the policy runs and sets `ImpactFlag` to 1 or 2, the alert no longer matches the filter and will not be re-processed on subsequent polling cycles.

   Confirm that the new row appears in the **Event Mapping Table** at the bottom of the page, showing `ImpactFlag = 0` in the Restriction Filter column and `AA_EnrichAlert` in the Policy Name column.
6. Click the **Save** icon (floppy disk, top-left) to save the service.
7. With the `EnrichmentEventReader` service selected in the left pane, click the **Start** button (green triangle in the toolbar) to start the service. Confirm the service state changes to **Running**.

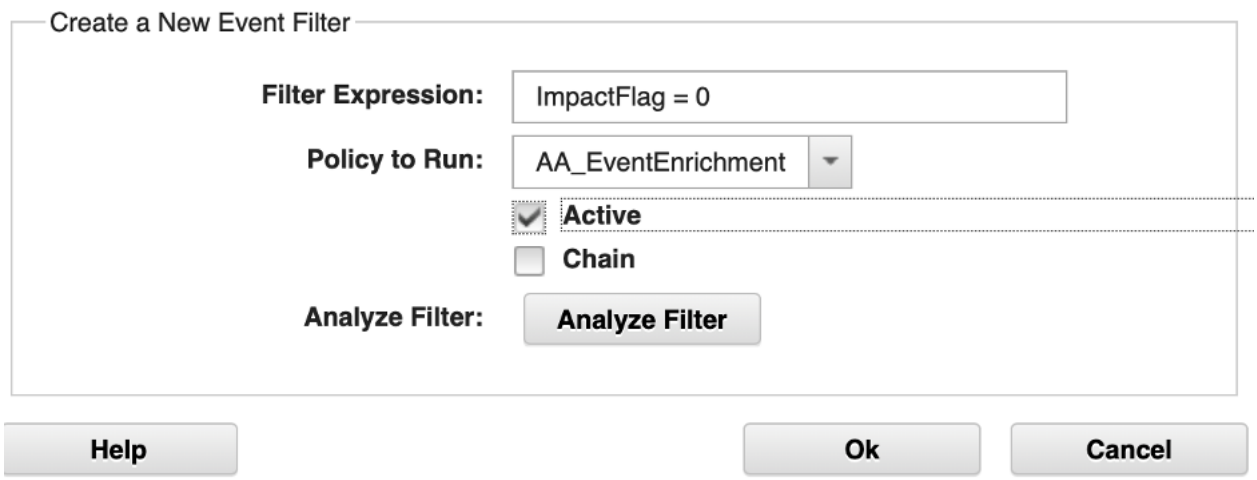


Figure 35: Event Mapping row configured before clicking OK

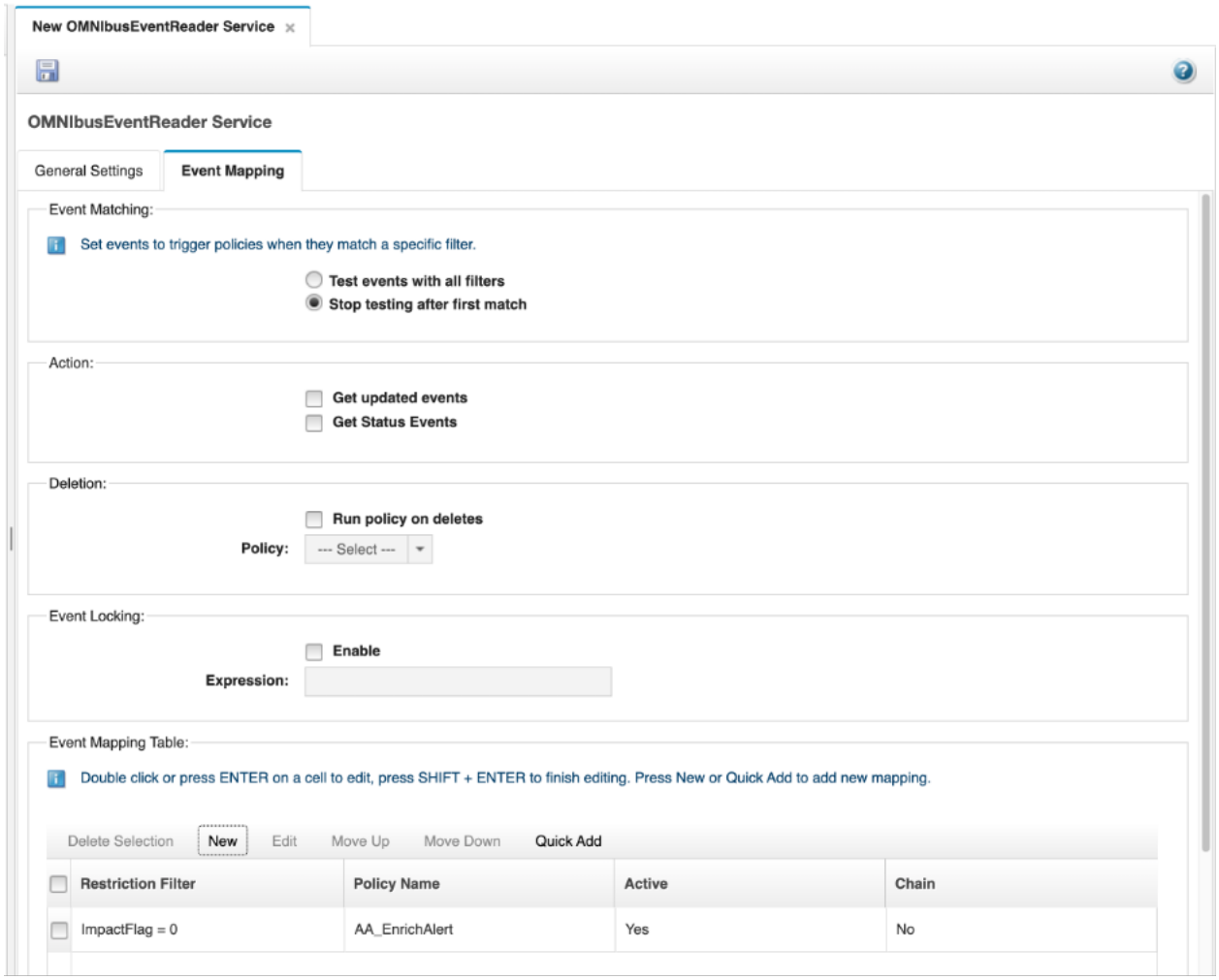


Figure 36: Event Mapping Table with EnrichAlert policy assigned

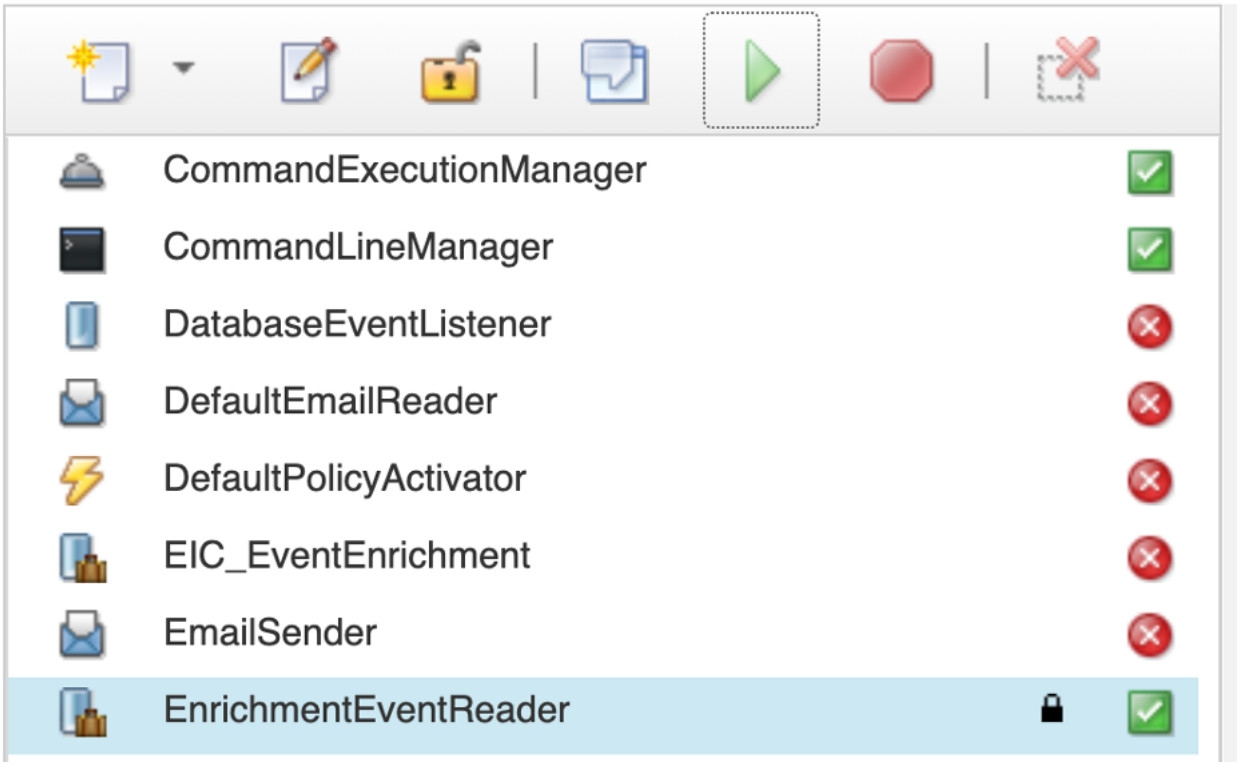


Figure 37: EnrichmentEventReader service in Running state

Validation The exercise is successful when all of the following are true:

- The `EnrichmentEventReader` service is saved with data source `defaultobjectserver`.
- The Event Mapping Table contains one row: filter `ImpactFlag = 0` mapped to `AA_EnrichAlert`, Active.
- The service state is **Running**.
- You can explain why `ImpactFlag = 0` is a better filter than `Severity > 0` for an enrichment service.

Expected result The enrichment pipeline is now active. Any new alert that enters the ObjectServer will have `ImpactFlag = 0` by default, which will be picked up by the service and passed to the `AA_EnrichAlert` policy. Once processed, `ImpactFlag` is updated and the alert is not re-processed.

4.3 Configure the policy logger

Objective In this exercise, you will configure the Impact PolicyLogger service to write each policy's log output to its own dedicated log file. By the end of the exercise, any `Log()` call in a policy will write to a dedicated file under `$IMPACT_HOME/logs/`, making it straightforward to isolate and review the output from a specific policy without wading through a shared log.

Background By default, all `Log()` calls from all policies write to the shared `ImpactServer.log` file. When several policies are running simultaneously, their output is interleaved and can be difficult to follow. The PolicyLogger service has an option — **Append policy name to log file name** — that causes each policy to write to its own file instead. With this option enabled, a policy named `AA_EnrichAlert` writes to `$IMPACT_HOME/logs/NCI_policylogger_AA_EnrichAlert.log`, a policy named `AA_EscalatePlatinum` writes to `$IMPACT_HOME/logs/NCI_policylogger_AA_EscalatePlatinum.log`, and so on. You configure this once and it applies to all policies on the server.

Prerequisites

- Completion of section 4.2.
- You are logged in to the Impact GUI in your private browser window.

Tasks

1. Select the **Services** tab from the top navigation bar.

2. In the services list, locate the **PolicyLogger** service. It is a built-in service that is present on every Impact server by default.

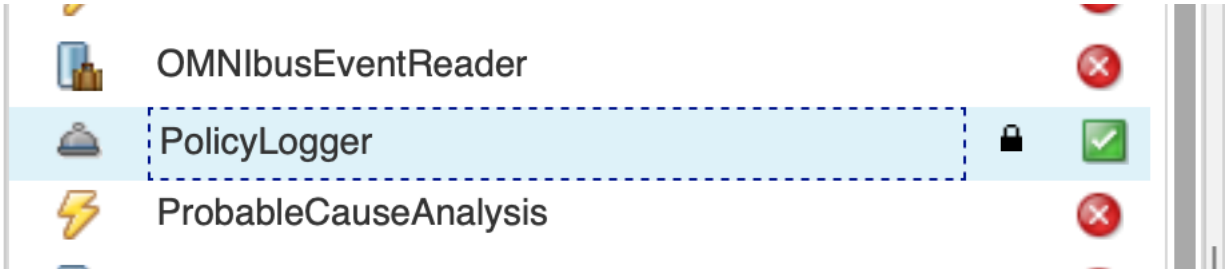


Figure 38: PolicyLogger service in the services list

3. Double-click the **PolicyLogger** service to open its configuration page.
4. Check the **Append policy name to log file name** checkbox.

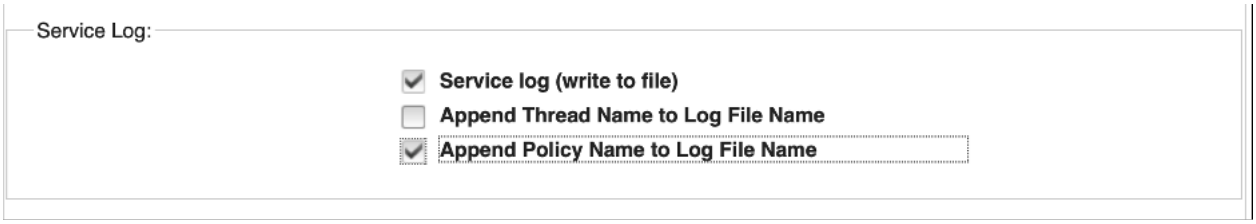


Figure 39: PolicyLogger configuration with Append policy name checked

5. Click the **Save** icon (floppy disk, top-left) to save the configuration.
6. SSH into the lab server as root and stop the Impact server:

```
$IMPACT_HOME/bin/stopImpactServer.sh
```

   Wait for the command to return. The Impact GUI session in your browser will become unresponsive — this is expected.

7. Start the Impact server:

```
$IMPACT_HOME/bin/startImpactServer.sh
```

   You should see:

```
Starting server NCI.
Server NCI started with process ID 526361.
```

   Note: The process ID in your output will differ from the example above.

   Wait approximately 60 seconds, then refresh the Impact GUI in your private browser window and log back in as `impactadmin` / `netcool`. Confirm that the **Services** tab shows the `PolicyLogger` and `EnrichmentEventReader` services in a **Running** state.

Validation The exercise is successful when all of the following are true:

- The **Append policy name to log file name** checkbox is checked and saved.
- The Impact server has been restarted and the GUI is accessible.
- The `PolicyLogger` and `EnrichmentEventReader` services are both in a **Running** state after the restart.
- You can explain why per-policy log files are more useful than a single shared log when multiple policies are running.

Expected result From this point on, every `Log()` call in any policy will write to a dedicated file in `$IMPACT_HOME/logs/`. When you run test events in the next exercise, you will be able to check `$IMPACT_HOME/logs/NCI_policylogger_AA_EnrichAlert.log` directly to see the output from the enrichment policy.

4.4 Run test events and validate enrichment

Objective In this exercise, you will insert test events into the ObjectServer and confirm that the AA_EnrichAlert policy correctly enriches matching alerts and handles nodes with no inventory record.

Prerequisites

- Completion of section 4.3.
- The EnrichmentEventReader service is running.
- The DEVICE_INVENTORY table in the CMDB database is populated (section 2.2).

Tasks

1. Return to your original OMNIbus Web GUI browser window (non-private) at https://<your-server>:16311/ibm/console. Log in as ncoadmin with password netcool if your login has timed out.
2. Open the event viewer again, and create an event filter named ImpactLabEvents with the condition Manager = 'ImpactLab' and apply it.
3. Create an event view named EnrichmentView with the following columns in order: Severity, Acknowledged, Customer, Location, Service, Node, Summary, FirstOccurrence, Contact. Apply it.
4. SSH into the lab server. Switch to the db2inst1 user and confirm that db-tok-01 exists in the DEVICE_INVENTORY table:

   ```
   su - db2inst1
   ```

   Then run:

   ```
   db2 "CONNECT TO CMDB"
   db2 "SELECT NODE_NAME, CUSTOMER_ID, SERVICE FROM DEVICE_INVENTORY WHERE NODE_NAME =
    ↪  'db-tok-01'"
   ```

   You should see one row with CUSTOMER_ID = CUST-1005 and SERVICE = Database Services. This record was inserted in section 2.2.
5. Type exit to return to your root shell, then use the nco_sql tool to insert a test alert for db-tok-01 into the ObjectServer:

   ```
   $OMNIHOME/bin/nco_sql -server EVENTS -user root -password netcool << 'EOF'
   INSERT INTO alerts.status (Node, Summary, AlertKey, AlertGroup, Severity, Type, Identifier,
    ↪  Manager, OwnerUID)
   VALUES ('db-tok-01', 'Enrichment test event', 'enrich-key-01', 'TrainingAlerts', 4, 1,
    ↪  'db-tok-01:enrich-key-01:TrainingAlerts', 'ImpactLab', 65534);
   go
   EOF
   ```

   Confirm you see that one row has been affected (your insert).
6. Switch to the OMNIbus Web GUI. The db-tok-01 alert should appear in the ImpactLabEvents filter view. Within a few seconds the enrichment fields should populate. Confirm that the Enrichment View columns show:

   - Customer shows CUST-1005
   - Service shows Database Services

   Double-click the alert to open the full event details and confirm ImpactFlag shows 1.
7. Check the policy log to confirm the enrichment message was written:

   ```
   tail -30 $IMPACT_HOME/logs/NCI_policylogger_AA_EnrichAlert.log
   ```

   Look for: AA_EnrichAlert: enrichment applied for Serial=..., CustomerID=CUST-1005, Service=Database Services
8. Now test the not-found path. Insert a test alert for a node that does not exist in the inventory:

   ```
   $OMNIHOME/bin/nco_sql -server EVENTS -user root -password netcool << 'EOF'
   INSERT INTO alerts.status (Node, Summary, AlertKey, AlertGroup, Severity, Type, Identifier,
    ↪  Manager, OwnerUID)
   ```

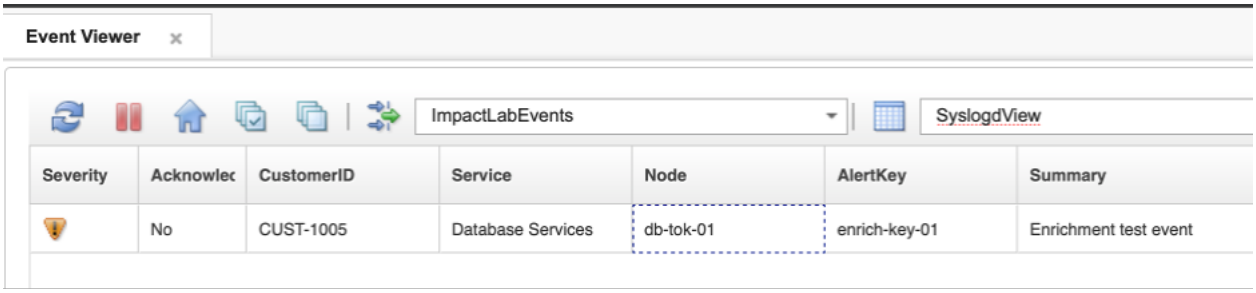


Figure 40: Enriched db-tok-01 alert in the event list

```
VALUES ('unknownhost99', 'Event from unknown node', 'unknown-key-01', 'TrainingAlerts', 3,
 ↪  1, 'unknownhost99:unknown-key-01:TrainingAlerts', 'ImpactLab', 65534);
go
EOF
```

9. In the event list, locate the `unknownhost99` alert and confirm the `Enrichment View` columns show:

   - `Customer` shows `NOT_FOUND`
   - `Service` shows `NOT_FOUND`

   Double-click the alert and confirm `ImpactFlag` shows 2.

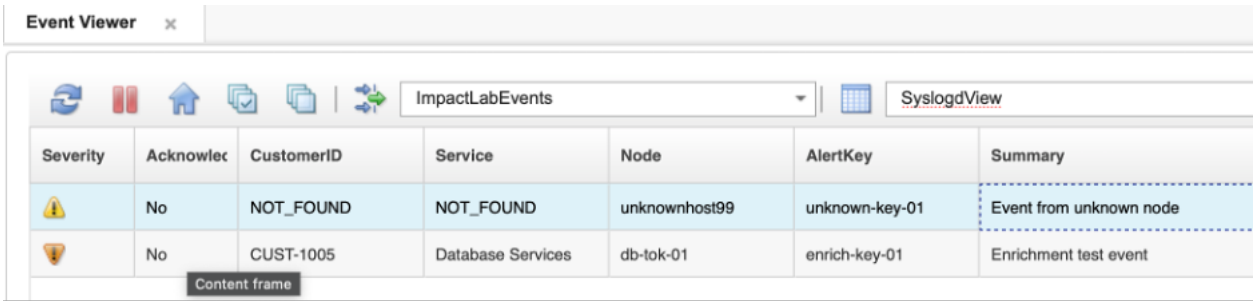


Figure 41: unknownhost99 alert showing NOT_FOUND enrichment and ImpactFlag=2

10. Confirm in the policy log that the warning message was written:

```
tail -30 $IMPACT_HOME/logs/NCI_policylogger_AA_EnrichAlert.log
```

    Look for: `AA_EnrichAlert: no inventory record found for Serial=..., Node=unknownhost99; ImpactFlag set to 2`

Validation The exercise is successful when all of the following are true:

- A test alert for `db-tok-01` results in `CustomerID = CUST-1005`, `Service = Database Services`, and `ImpactFlag = 1` on the alert.
- The Impact server log shows the enrichment confirmation message for `db-tok-01`.
- A test alert for `unknownhost99` results in `CustomerID = NOT_FOUND`, `Service = NOT_FOUND`, and `ImpactFlag = 2` on the alert.
- The Impact server log shows the warning message for `unknownhost99`.
- You can describe the complete path an event takes from ObjectServer insertion to enriched alert: probe → `alerts.status` → OMNIbusEventReader service → `AA_EnrichAlert` policy → `ReturnEvent` → updated alert.
- You can explain what the three `ImpactFlag` values mean and why this pattern prevents re-processing.

Expected result You now have a working end-to-end event enrichment pipeline. New alerts that arrive in the ObjectServer are automatically enriched with customer and service context within seconds of insertion, without any manual intervention. This is the core automation pattern that Impact is built around.

5. Scheduled automation

Some automation tasks need to run on a schedule rather than in response to a specific event. Impact's **PolicyActivator** service fills this role. It runs an assigned policy at a defined interval, independently of any inbound event, making it the Impact equivalent of a scheduled task or cron job.

Common use cases include:

- **Deferred or retry enrichment**: Reprocess unresolved events whose initial lookup failed because a CMDB, DNS, REST endpoint, or data source was temporarily unavailable at insert time.
- **Aging and escalation**: Query active events based on `FirstOccurrence`, `LastOccurrence`, severity, ownership, or acknowledgment state, then add journal context, change severity, send a notification, or create and update a ticket.
- **Periodic reconciliation**: Compare ObjectServer event state with an external ITSM, inventory, maintenance window, or CMDB source and repair drift.
- **Maintenance and retention**: Purge stale Impact reporting data or perform controlled cleanup. IBM specifically documents attaching a recurring reporting purge policy to a PolicyActivator for this purpose.
- **Batch state evaluation**: Run a periodic health or compliance calculation across a population of devices or services, then update event fields or trigger downstream automation.

Unlike the OMNIbusEventReader, the PolicyActivator has a **1:1 relationship** with its policy — each PolicyActivator service runs exactly one policy. The policy is selected from a dropdown on the service configuration page and there is no event filter or mapping table.

In this section, you will implement **deferred enrichment** — a scheduled policy that retries CMDB lookups for alerts that previously failed enrichment because the data source was unavailable. This is one of the most common practical uses of the PolicyActivator in production OMNIbus environments.

5.1 Write the deferred enrichment policy

Objective In this exercise, you will create a policy that scans the ObjectServer for alerts where enrichment previously failed and retries the CMDB lookup for each one. By the end of the exercise, you will have a policy that uses `GetByFilter` to find a specific population of alerts and acts on each result.

Prerequisites

- Completion of section 4.
- The `defaultobjectserver` ObjectServer DSA is configured and connected.
- The `EVENTS` data type is configured against the `alerts.status` table (section 2.1).
- The `CMDB` DB2 DSA and `DEVICE_INVENTORY` data type are configured (section 2.2).
- The `ImpactFlag`, `CustomerID`, and `Service` columns exist in `alerts.status` (section 4.1).

Background Recall from section 4 that the enrichment policy sets `ImpactFlag = 2` when no inventory record is found for an alert's `Node` value. There are two reasons an alert might reach this state:

1. The node genuinely does not exist in the CMDB — it is unknown or out of scope.
2. The CMDB was temporarily unavailable at the time the alert arrived — a database restart, a network blip, or a DSA reconnection in progress.

In the second case, the alert was not enriched through no fault of its own. A scheduled deferred enrichment policy handles this by periodically retrying the CMDB lookup for all `ImpactFlag = 2` alerts. If the CMDB is now available and the record is found, enrichment is applied and `ImpactFlag` is updated to 1. If the record still cannot be found, the alert remains at `ImpactFlag = 2` for the next cycle.

A PolicyActivator policy does not receive a triggering event — there is no `EventContainer` passed in by the service. Instead, the policy queries the ObjectServer directly using `GetByFilter` to build the list of alerts to process. Write-back to each alert is done using `ReturnEvent` on the individual row objects returned from the query.

Tasks

1. In your private browser window, ensure you are logged in to the Impact GUI. If your session has expired, navigate to `http://<your-server>:17311/ibm/console` and log in as `impactadmin` with password `netcool`.
2. Select the **Policies** tab from the top navigation bar. In the toolbar at the top of the policy list, click the **dropdown arrow** next to the New Policy icon and select **IPL Policy**. A blank IPL code editor opens titled **New IPL Policy**.

3. Before entering any code, click the **Save** icon (floppy disk, top-left). A **Save** dialog appears with a single **Policy Name** field. Enter `AA_DeferredEnrichment` and click **Save**.
4. In the IPL code editor, enter the following policy code:

```
/*
* Policy: AA_DeferredEnrichment
* Purpose: Retry CMDB enrichment for alerts where ImpactFlag = 2
*          (previous enrichment attempt failed — no record found).
* Trigger: PolicyActivator (scheduled, not event-driven).
*/

// Find all alerts where enrichment previously found no record
pendingAlerts = GetByFilter("EVENTS", "ImpactFlag = 2", False);

//Get the number of rows returned
alertCount = Length(pendingAlerts);

Log("AA_DeferredEnrichment: found " + alertCount + " alert(s) pending re-enrichment");

if (alertCount > 0)
{
   counter = 0; // set a counter as an index to loop through the returned alerts
   while(counter < alertCount)
   {
      nodeValue = pendingAlerts[counter].Node;

      nodeFilterValue = Replace(nodeValue, "'", "''");
      filter = "NODE_NAME = '" + nodeFilterValue + "'";
      inventoryRows = GetByFilter("DEVICE_INVENTORY", filter, False);

      if (Length(inventoryRows) > 0)
      {
         inventoryRow = inventoryRows[0]; // for simplicty only assume 1 return

         Log("AA_DeferredEnrichment: enrichment applied for Node=" + nodeValue
            + ", CustomerID=" + inventoryRow.CustomerID );

         // Update this alert using the Identifier as our where clause:

         DirectSQL("defaultobjectserver", "update alerts.status set CustomerID='"
            + inventoryRow.CUSTOMER_ID + "', Service='"
            + inventoryRow.SERVICE + "', ImpactFlag=1 where Identifier='"
            + pendingAlerts[counter].Identifier + "'", false);

      }
      else
      {

         Log("AA_DeferredEnrichment: still no record for Node=" + nodeValue
            + "; leaving ImpactFlag=2");
      }

      counter = counter + 1;

   }
}
else
{
   Log("AA_DeferredEnrichment: no alerts pending re-enrichment");
}
```

Note: In a PolicyActivator policy there is no `EventContainer` — the service does not pass a triggering event. Instead, each row returned by `GetByFilter` is its own object. Field assignments are made directly on the referenced row (`pendingAlerts[counter].CustomerID = ...`) and we use `DirectSQL` function against the **data source** for our ObjectServer (`defaultobjectserver`), not against the data type EVENTS. This is purely to illustrate that you can run queries, inserts, and updates using Impact functions such as GetByFilter(), AddDataItem(), and BatchUpdate(), but you can also construct SQL queries and run those queries against the data source itself.

5. Click the **Syntax Check** icon in the toolbar. Confirm the **Syntax check successful** dialog appears, then click **Close**.
6. Click the **Save** icon to save the policy code. Confirm that `AA_DeferredEnrichment` appears at the top of the policy list in the left pane.

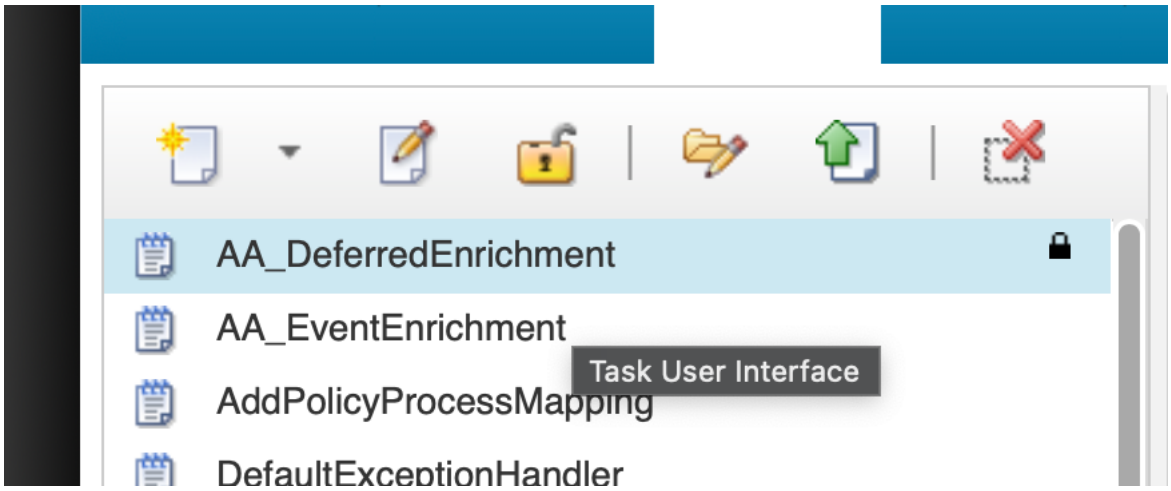


Figure 42: Saved AA_DeferredEnrichment in policy list

Validation The exercise is successful when all of the following are true:

- The `AA_DeferredEnrichment` policy is saved and appears at the top of the policy list.
- The syntax check passes with no errors.
- You can explain why this policy uses `GetByFilter` rather than `EventContainer`.
- You can describe what happens when a retry succeeds versus when no record is still found.
- You can explain the difference between `ReturnEvent(EventContainer)` in the enrichment policy and `ReturnEvent(alert)` here.

Expected result You now have a deferred enrichment policy that can recover alerts missed by the real-time enrichment pipeline. The next exercise creates the PolicyActivator service and connects it to this policy.

5.2 Create and configure the PolicyActivator service

Objective In this exercise, you will create a PolicyActivator service and configure it to run the `AA_DeferredEnrichment` policy every 60 seconds.

Prerequisites

- Completion of section 5.1.
- The `AA_DeferredEnrichment` policy is saved.

Tasks

1. Select the **Services** tab from the top navigation bar.
2. In the toolbar at the top of the services list, click the **dropdown arrow** next to the New Service icon and select **PolicyActivator**. The **New PolicyActivator Service** configuration page opens.
3. Configure the following fields:
   - **Service Name**: `DeferredEnrichmentActivator`

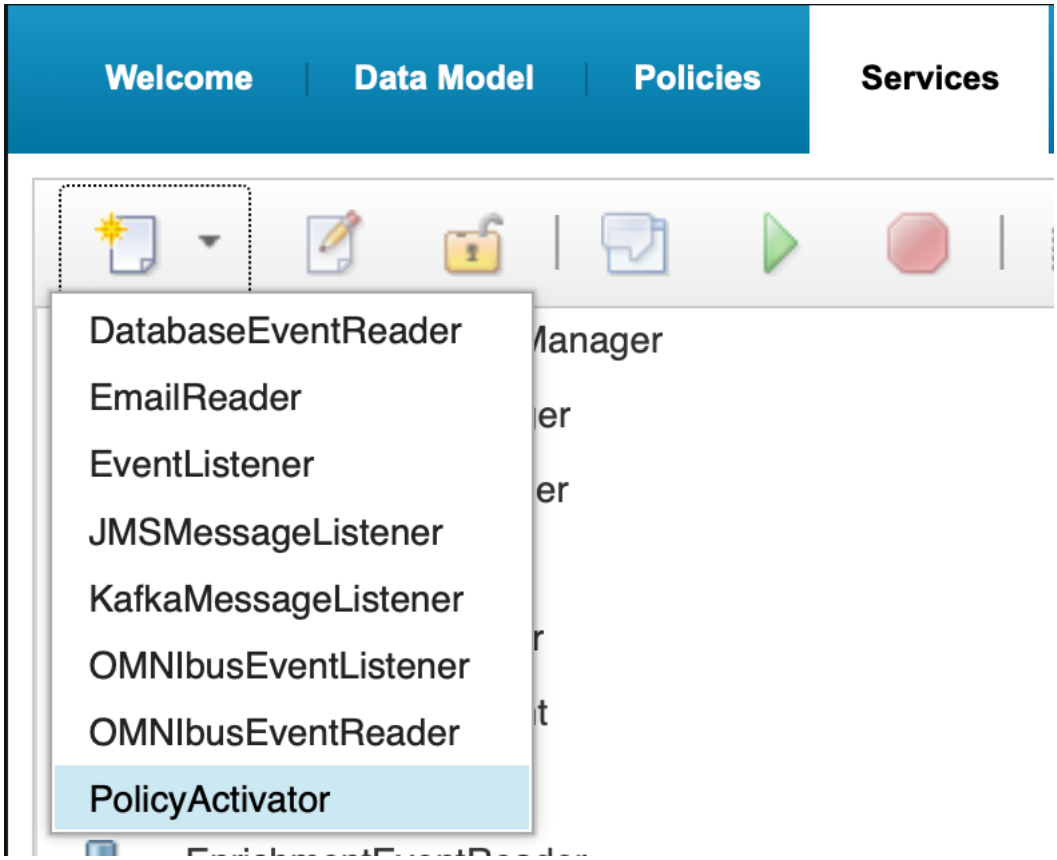


Figure 43: New Service dropdown with PolicyActivator selected

- **Activation Interval (seconds)**: `60`
- **Policy**: select `AA_DeferredEnrichment` from the dropdown
- Leave **Starts automatically when server starts** unchecked.

4. Click the **Save** icon (floppy disk, top-left) to save the service.
5. Select your new service in the service, and click the **Start** button (green triangle in the toolbar) to start the service. Alternatively, right-click on the service and select "Confirm the state changes to **Running**.

Validation The exercise is successful when all of the following are true:

- The `DeferredEnrichmentActivator` service is saved with activation interval `60` seconds.
- The policy is set to `AA_DeferredEnrichment`.
- The service state is **Running**.
- You can explain the difference between this service and the `EnrichmentEventReader` in terms of what triggers the policy.

Expected result The deferred enrichment pipeline is now active. Every 60 seconds, the PolicyActivator will invoke `AA_DeferredEnrichment`, which scans for `ImpactFlag = 2` alerts and retries the CMDB lookup for each one.

5.3 Validate deferred enrichment

Objective In this exercise, you will simulate a failed enrichment by inserting a test alert for an unknown node, then update the CMDB to add that node, and confirm that the deferred enrichment policy picks it up on its next cycle.

Prerequisites

- Completion of section 5.2.
- The `DeferredEnrichmentActivator` service is running.
- The `EnrichmentEventReader` service from section 4 is also running.

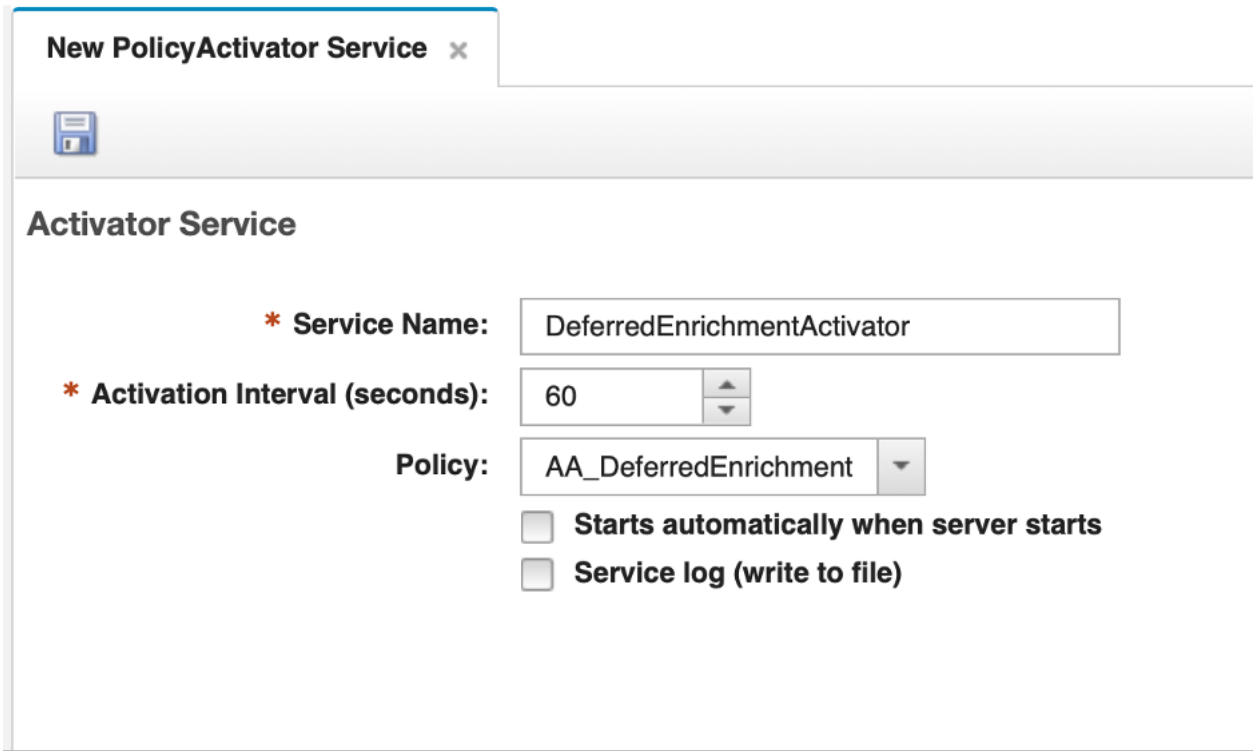


Figure 44: PolicyActivator configuration page

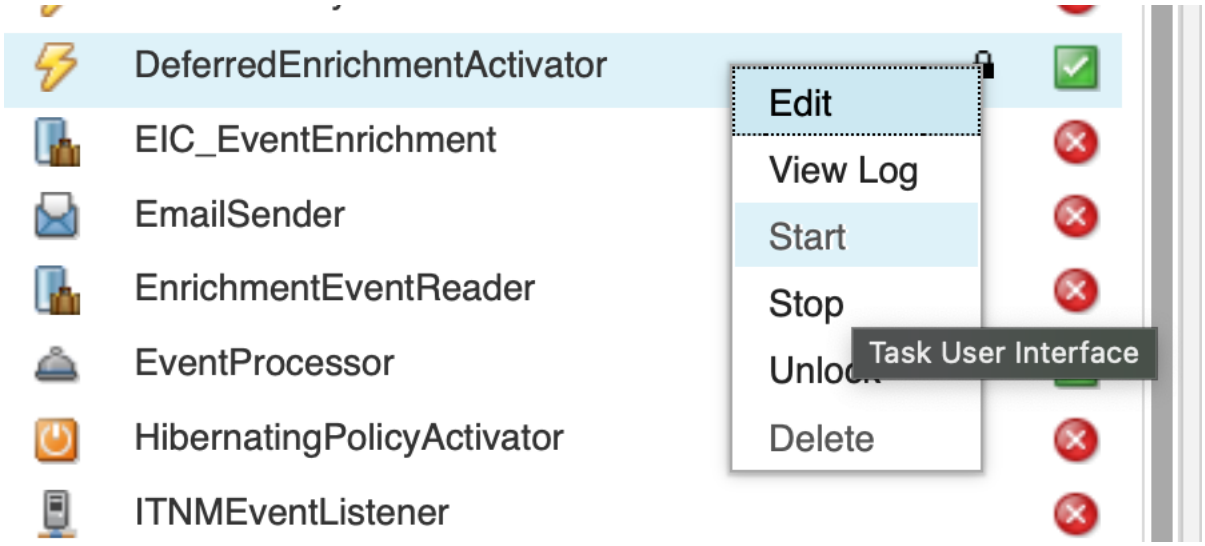


Figure 45: DeferredEnrichmentActivator service in Running state

Tasks

1. Insert a test alert for a node that does not exist in the CMDB. The enrichment policy will process it, fail to find a record, and set `ImpactFlag = 2`:

```
$OMNIHOME/bin/nco_sql -server EVENTS -user root -password netcool << 'EOF'
INSERT INTO alerts.status (Node, Summary, AlertKey, AlertGroup, Severity, Type, Identifier,
 ↪  Manager, OwnerUID)
VALUES ('newnode-ams-01', 'Deferred enrichment test', 'deferred-key-01', 'TrainingAlerts',
 ↪  3, 1, 'newnode-ams-01:deferred-key-01:TrainingAlerts', 'ImpactLab', 65534);
go
EOF
```

2. In the OMNIbus Web GUI, locate the `newnode-ams-01` alert in your `ImpactLabEvents` filter, and switch to your `SyslogdView`. Within a few seconds the `EnrichmentEventReader` should process it. Confirm that `Customer` and `Service` both show `NOT_FOUND` and that `ImpactFlag` shows 2 in the event details.

3. Now add `newnode-ams-01` to the CMDB so the next deferred enrichment cycle can find it. The DB2 commands must be run as the `db2inst1` user:

```
su - db2inst1
```

   Then run:

```
db2 "CONNECT TO CMDB" && \
db2 "INSERT INTO DEVICE_INVENTORY (NODE_NAME, CUSTOMER_ID, CUSTOMER_NAME, SERVICE, LOCATION,
 ↪  CONTACT) \
     VALUES ('newnode-ams-01', 'CUST-1001', 'Acme Corporation', 'WAN Connectivity',
 ↪  'Amsterdam DC', 'noc@acme.com')"
```

   You should see:

```
DB20000I  The SQL command completed successfully.
```

   Then type `exit` to return to your root shell.

4. Wait up to 120 seconds for the `DeferredEnrichmentActivator` to fire. Then refresh the event in the OMNIbus Web GUI and confirm:

   - `Customer` now shows `CUST-1001`
   - `Service` now shows `WAN Connectivity`
   - `ImpactFlag` now shows `1`

5. Check the policy log to confirm the retry succeeded:

```
tail -30 $IMPACT_HOME/logs/NCI_policylogger_AA_DeferredEnrichment.log
```

   Look for: `AA_DeferredEnrichment: enrichment applied for Node=newnode-ams-01, CustomerID=CUST-1001`

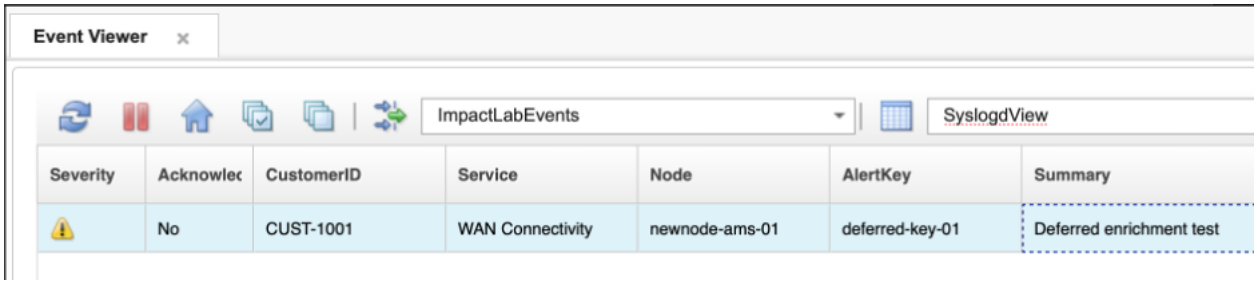


Figure 46: Deferred enrichment showing successful retry

Validation The exercise is successful when all of the following are true:

- The `newnode-ams-01` alert initially shows `NOT_FOUND` and `ImpactFlag = 2`.

- After adding the node to the CMDB, the deferred enrichment policy updates the alert to `ImpactFlag = 1` with correct customer and service values.
- The policy log confirms the enrichment retry message.
- You can explain the difference between `ImpactFlag = 1` (enriched at arrival) and `ImpactFlag = 1` after a deferred retry — from the alert's perspective, the outcome is identical.
- You can describe a real-world scenario where deferred enrichment would recover alerts that the real-time policy could not process.

Expected result You have now seen how the PolicyActivator complements the OMNIbusEventReader. The real-time enrichment pipeline handles the normal path; the scheduled deferred pipeline handles recovery. Together they ensure that all alerts eventually carry the context operators need, regardless of whether the CMDB was available at the moment each alert arrived.

5.4 Add a retry limit to the deferred enrichment policy

Objective In this exercise, you will update the `AA_DeferredEnrichment` policy to limit the number of times it will retry a failed enrichment. By the end of the exercise, the policy will use the standard `Poll` field as a retry counter, and alerts that have been attempted three times will be automatically excluded from further processing.

Background The policy as written will retry every `ImpactFlag = 2` alert on every cycle, indefinitely. In most cases this is fine — a transient CMDB outage resolves quickly and the alert is enriched on the next attempt. But for alerts whose `Node` value will never exist in the CMDB, this creates unnecessary processing overhead that compounds over time as more unknown-node alerts accumulate.

The solution is to use the `Poll` field — a standard integer field already present on every `alerts.status` row — as a retry counter. On each failed lookup, the policy increments `Poll` by 1 and uses `DirectSQL` to write that value back to the ObjectServer immediately. The `GetByFilter` query is updated to include `AND Poll < 3`, which means alerts that have already been attempted three times are never retrieved in the first place. Filtering at the query level is more efficient than pulling all `ImpactFlag = 2` alerts and checking `Poll` inside the loop — alerts beyond the limit simply never enter the processing path.

Prerequisites

- Completion of section 5.3.
- The `DeferredEnrichmentActivator` service is running.

Tasks

1. In the Impact GUI, select the **Policies** tab and open `AA_DeferredEnrichment` for editing. Replace the entire policy body with the following:

```
/*
* Policy: AA_DeferredEnrichment
* Purpose: Retry CMDB enrichment for alerts where ImpactFlag = 2
*          (previous enrichment attempt failed — no record found).
* Trigger: PolicyActivator (scheduled, not event-driven).
*/

// Find all alerts where enrichment previously found no record
pendingAlerts = GetByFilter("EVENTS", "ImpactFlag = 2 AND Poll < 3", False);

//Get the number of rows returned
alertCount = Length(pendingAlerts);

Log("AA_DeferredEnrichment: found " + alertCount + " alert(s) pending re-enrichment");

if (alertCount > 0)
{
   counter = 0; // set a counter as an index to loop through the returned alerts
   while(counter < alertCount)
   {
      nodeValue = pendingAlerts[counter].Node;
```

```
      nodeFilterValue = Replace(nodeValue, "'", "''");
      filter = "NODE_NAME = '" + nodeFilterValue + "'";
      inventoryRows = GetByFilter("DEVICE_INVENTORY", filter, False);

      if (Length(inventoryRows) > 0)
      {
         inventoryRow = inventoryRows[0]; // for simplicty only assume 1 return

         Log("AA_DeferredEnrichment: enrichment applied for Node=" + nodeValue
            + ", CustomerID=" + inventoryRow.CustomerID );

         // Update this alert using the Identifier as our where clause:

         DirectSQL("defaultobjectserver", "update alerts.status set CustomerID='"
            + inventoryRow.CUSTOMER_ID + "', Service='"
            + inventoryRow.SERVICE + "', ImpactFlag=1 where Identifier='"
            + pendingAlerts[counter].Identifier + "'", false);

      }
      else
      {

         currentPollValue = pendingAlerts[counter].Poll;
         newPollValue = currentPollValue + 1;

         DirectSQL("defaultobjectserver", "update alerts.status set Poll="
            + newPollValue + " where Identifier='"
            + pendingAlerts[counter].Identifier + "'", false);

            Log("AA_DeferredEnrichment: still no record for Node=" + nodeValue
               + "; Poll=" + pendingAlerts[counter].Poll);


      }


      counter = counter + 1;

   }
}
else
{
   Log("AA_DeferredEnrichment: no alerts pending re-enrichment");
}
```

2. **Save** your updated policy, then review the changes in the updated policy against the original. For each of the following, identify where it appears in the code and explain its purpose:

Change	Where	Why
`"ImpactFlag = 2 AND Poll < 3"` added to `GetByFilter`	The `GetByFilter` filter string at the top of the policy	Excludes alerts that have already been retried three times before they are even retrieved. Filtering at the query level means those alerts never enter the loop — no wasted iterations.
`Poll` increment in the failure branch	`currentPollValue` / `newPollValue` calculation in the `else` block	Tracks how many times enrichment has been attempted for this alert. Each failed lookup advances the counter by 1.

Change	Where	Why
`DirectSQL` in the failure branch	`DirectSQL("defaultobjectserver", "update alerts.status set Poll=...")`	Writes the incremented `Poll` value directly back to the ObjectServer using a SQL UPDATE statement.
`Poll` is only incremented in the failure branch	Success branch updates `CustomerID`, `Service`, and `ImpactFlag = 1` via `DirectSQL` — `Poll` is left untouched	Once an alert is successfully enriched, `ImpactFlag` changes to 1 and it will never be retrieved by this policy again (`ImpactFlag = 2 AND Poll < 3` will not match). There is no need to increment `Poll` on the success path.

3. Click the **Syntax Check** icon. Confirm the **Syntax check successful** dialog appears, then click **Close**.
4. Click the **Save** icon to save the updated policy (in case you forgot to save it as directed in task 2).

Validation The exercise is successful when all of the following are true:

- The `GetByFilter` filter string reads `"ImpactFlag = 2 AND Poll < 3"`.
- `Poll` is incremented only on the failure path.
- `DirectSQL` is used on both paths to write changes back to the ObjectServer.
- The syntax check passes with no errors.
- You can explain why `DirectSQL` is used instead of `ReturnEvent` in a PolicyActivator context.
- You can explain why filtering `Poll < 3` at the query level is more efficient than checking it inside the loop.

Expected result The policy now limits itself to three attempts per alert. Alerts that cannot be enriched are retried up to three times and then silently excluded from further processing, keeping the policy focused on alerts that still have a realistic chance of recovery.

5.5 Validate the retry limit

Objective In this exercise, you will confirm that the updated policy correctly limits retries to three attempts for an alert that will never have a CMDB entry.

Prerequisites

- Completion of section 5.4.
- The `DeferredEnrichmentActivator` service is running.
- The `EnrichmentEventReader` service from section 4 is running.

Tasks

1. Insert a test alert for a node that does not exist in the CMDB and will not be added — this simulates a genuinely unknown node:

```
$OMNIHOME/bin/nco_sql -server EVENTS -user root -password netcool << 'EOF'
INSERT INTO alerts.status (Node, Summary, AlertKey, AlertGroup, Severity, Type, Identifier,
 ↪  Manager, OwnerUID)
VALUES ('ghost-node-99', 'Retry limit test', 'retry-limit-01', 'TrainingAlerts', 3, 1,
 ↪  'ghost-node-99:retry-limit-01:TrainingAlerts', 'ImpactLab', 65534);
go
EOF
```

2. In the OMNIbus Web GUI, locate the `ghost-node-99` alert in the `ImpactLabEvents` filter. Confirm that `EnrichmentEventReader` processes it and sets `Customer = NOT_FOUND`, `Service = NOT_FOUND`, and `ImpactFlag = 2`. Note that `Poll` starts at `0`.
3. Watch the policy log over the next three minutes. Each time the `DeferredEnrichmentActivator` fires you should see a log entry with the `Poll` counter incrementing:

```
tail -f $IMPACT_HOME/logs/NCI_policylogger_AA_DeferredEnrichment.log
```

After the first attempt: `Poll=1`
After the second attempt: `Poll=2`
After the third attempt: `Poll=3`

4. After the third attempt, open the `ghost-node-99` alert in the Web GUI and confirm `Poll` shows 3 and `ImpactFlag` still shows 2.
5. Wait for at least one more `DeferredEnrichmentActivator` cycle (60 seconds). Confirm that `ghost-node-99` no longer appears in the policy log output — it is now excluded by the `Poll < 3` filter.

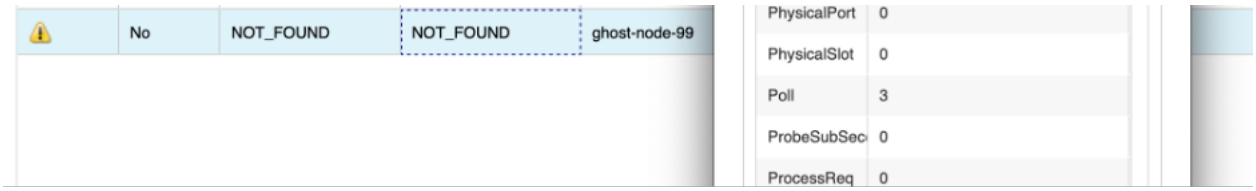


Figure 47: ghost-node-99 excluded after Poll reaches 3

Validation The exercise is successful when all of the following are true:

- `ghost-node-99` is retried exactly three times, with `Poll` incrementing from `0` to 3.
- After `Poll` reaches 3, the alert no longer appears in the policy log on subsequent cycles.
- `ImpactFlag` remains 2 — the alert is preserved as unenriched for operator visibility.
- You can explain why filtering at the query level is more efficient than retrieving all `ImpactFlag = 2` alerts and checking `Poll` inside the loop.

Expected result The retry limit ensures that genuinely unknown nodes do not generate processing overhead indefinitely. Alerts with `Poll = 3` and `ImpactFlag = 2` remain visible in the event list so operators can investigate, but they are silently excluded from further automated retry attempts.

6. Event escalation

Not every alert that arrives in the ObjectServer demands immediate action. But an alert that has been sitting open and unacknowledged for an extended period is a signal that something has been missed. Impact can automate the response to that condition — detecting aged, unacknowledged events and escalating them by raising their severity or setting a visual flag to draw operator attention.

This section extends the enrichment pipeline from sections 4 and 5 with a service-level dimension: a `SERVICE_LEVEL` field is added to both the CMDB and the ObjectServer, the enrichment policy is updated to populate it, and a new scheduled escalation policy targets only `Platinum`-tier events that have aged past a threshold without acknowledgment.

This section introduces two important ideas that extend what you have already learned:

- **Scheduled escalation with a PolicyActivator**: Alerts that age past a threshold do not generate an ObjectServer update event — time simply passes. A PolicyActivator polls on a fixed interval and queries for qualifying alerts, making it the right tool for age-based escalation.
- **Severity modification and the `Flash` flag**: Writing back to `Severity` and `Flash` using `DirectSQL` to change how an alert appears to operators.

In production, a reasonable escalation threshold might be 10 minutes or more. For this lab, you will use **60 seconds** so the behavior is immediately observable.

6.1 Preparing the scenario

Objective In this exercise, you will extend the existing enrichment data model and pipeline to carry a service level classification. You will add a `SERVICE_LEVEL` field to the DB2 CMDB table and the ObjectServer schema, re-discover the affected data types in Impact, populate the CMDB with test data, and update the enrichment policy to write the new field to each alert.

Prerequisites

- Completion of sections 4 and 5.
- The `defaultobjectserver` ObjectServer DSA is configured and connected.
- The CMDB DB2 DSA is configured and connected.
- The `AA_EnrichAlert` policy exists and is working correctly.

Tasks

1. SSH into the lab server and switch to the DB2 instance user:

```
su - db2inst1
```

2. Add the `SERVICE_LEVEL` column to the `DEVICE_INVENTORY` table in DB2:

```
db2 connect to CMDB
db2 "ALTER TABLE DEVICE_INVENTORY ADD COLUMN SERVICE_LEVEL VARCHAR(32)"
db2 connect reset
exit
```

   You should see `DB20000I  The SQL command completed successfully.` after the `ALTER TABLE` statement.

3. Add the `ServiceLevel` field to the `alerts.status` table in the ObjectServer:

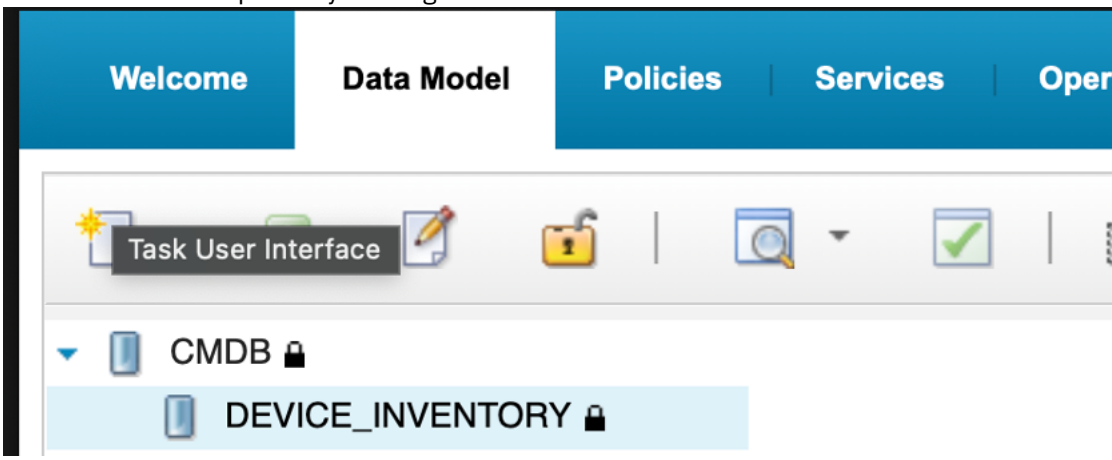
```
$OMNIHOME/bin/nco_sql -server EVENTS -user root -password netcool << 'EOF'
ALTER TABLE alerts.status ADD COLUMN ServiceLevel VARCHAR(32);
go
EOF
```

4. Verify that the `ServiceLevel` column was added successfully:

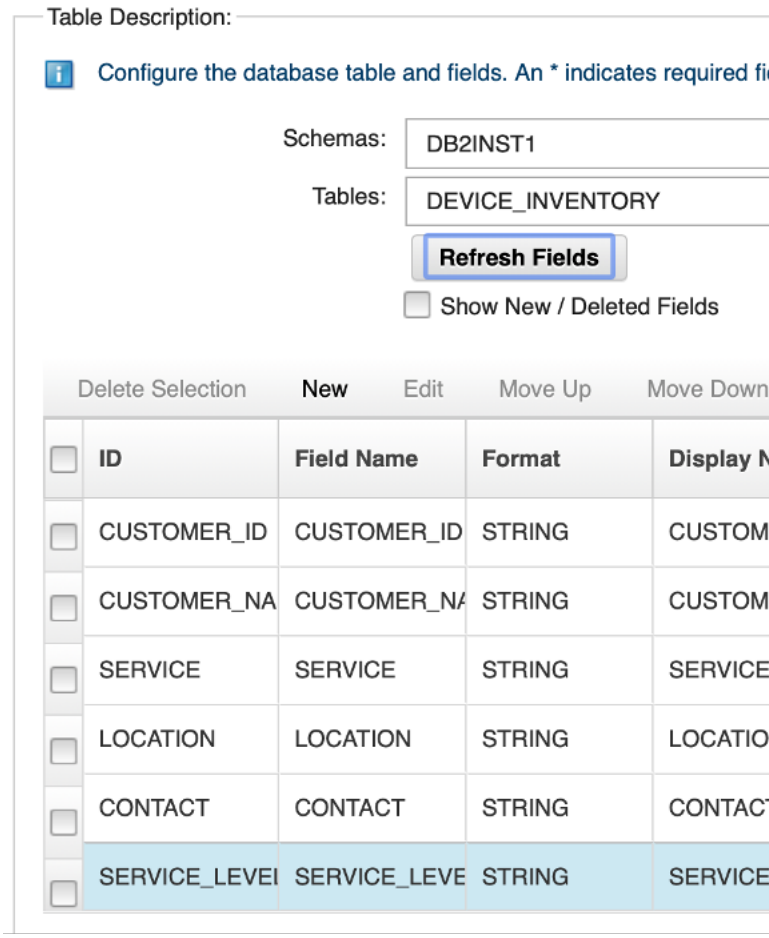
```
$OMNIHOME/bin/nco_sql -server EVENTS -user root -password netcool << 'EOF'
describe alerts.status;
go
EOF
```

   Confirm that `ServiceLevel` appears in the output as a `VARCHAR(32)` column.

5. In the Impact GUI, re-discover the `DEVICE_INVENTORY` data type so Impact picks up the new `SERVICE_LEVEL` column.
   - Select the **Data Model** tab from the top navigation bar.
   - Locate the CMDB data source in the DSA list and open it by clicking on the arrow to the left of the titel. Double-click `DEVICE_INVENTORY` to open it.



   - Click the **Refresh Fields** button in the `Table Description` section of the data type configuration window.



- Confirm that SERVICE_LEVEL now appears in the field list.
- Click **Save** to save the updated data type using the disk icon at the top-left of the data type config window.

6. Next, in the same Data Model tab, re-discover the EVENTS data type to pick up the new ServiceLevel column from alerts.status.
   - Locate the defaultobjectserver DSA, open the tree view and double-click on the EVENTS data type to open it.
   - Click the **Refresh Fields** button in the Table Description section of the data type configuration window.
   - Confirm that ServiceLevel now appears in the field list.
   - Click **Save** by clicking on the disk icon.
7. Populate the SERVICE_LEVEL field in the CMDB for **Acme Corporation** nodes. In the lab, Acme Corporation holds nodes router-lon-01 and router-lon-02 (customer CUST-1001):

   First, switch to the db2inst1 user:

   ```
   su - db2inst1
   ```

   Next, populate the **Acme Corporation** nodes with the **Platinum** service level:

   ```
   db2 connect to CMDB
   db2 "UPDATE DEVICE_INVENTORY SET SERVICE_LEVEL = 'Platinum' WHERE CUSTOMER_ID = 'CUST-1001'"
   db2 connect reset
   ```

   You should see DB20000I The SQL command completed successfully.

   Verify the update:

   ```
   db2 connect to CMDB
   db2 "SELECT NODE_NAME, CUSTOMER_NAME, SERVICE_LEVEL FROM DEVICE_INVENTORY"
   ```

   Confirm that router-lon-01 and router-lon-02 show Platinum in the SERVICE_LEVEL column. All other rows should show a null value.

   Exit the db2inst1 user to return to the root user:

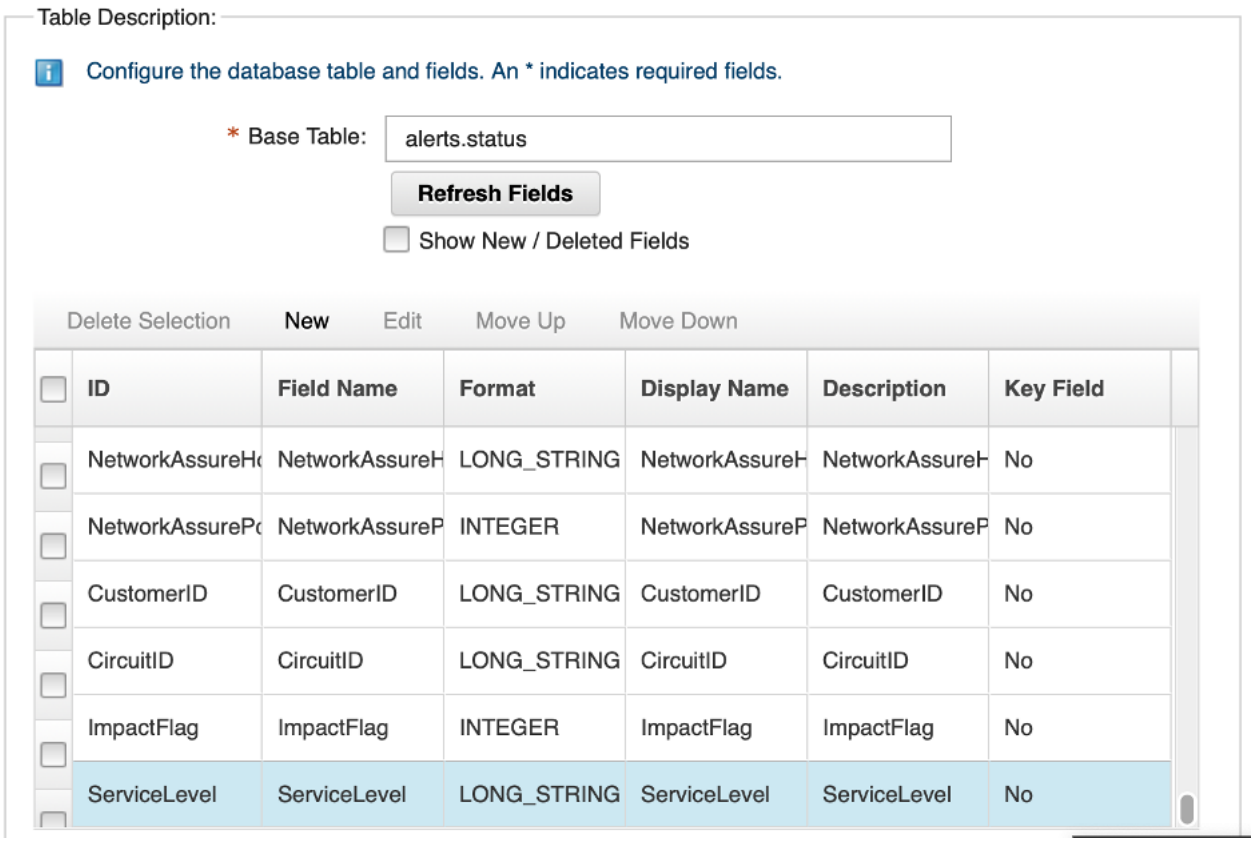


Figure 48: EVENTS data type after re-discovery showing ServiceLevel field

```
exit
```

8. Open the `AA_EnrichAlert` policy in the Impact GUI and add `ServiceLevel` to the enrichment write-back.
   - Select the **Policies** tab and click `AA_EnrichAlert` in the policy list to open it.
   - In the `if (Length(inventoryRows) > 0)` branch, add the following line immediately after the existing `EventContainer.Service` assignment:

     ```
     EventContainer.ServiceLevel = inventoryRow.SERVICE_LEVEL;
     ```

   - In the not-found `else` branch, add:

     ```
     EventContainer.ServiceLevel = "NOT_FOUND";
     ```

   - Click **Syntax Check**. Confirm the check passes, then click **Save**.

Validation The exercise is successful when all of the following are true:

- `SERVICE_LEVEL VARCHAR(32)` exists in the `DEVICE_INVENTORY` DB2 table.
- `ServiceLevel VARCHAR(32)` exists in `alerts.status`.
- Both `DEVICE_INVENTORY` and `EVENTS` data types in the Impact Data Model show the new field.
- `router-lon-01` and `router-lon-02` have `SERVICE_LEVEL = 'Platinum'` in the CMDB.
- The `AA_EnrichAlert` policy includes `ServiceLevel` in both the found and not-found branches, and the syntax check passes.

Expected result The enrichment pipeline now carries service level classification. Any alert from a Platinum-tier node will have `ServiceLevel = 'Platinum'` set at enrichment time. The next exercise uses this field to build a targeted escalation policy.

```
if (Length(inventoryRows) > 0) {
    inventoryRow = inventoryRows[0];

    EventContainer.CustomerID = inventoryRow.CUSTOMER_ID;
    EventContainer.Service = inventoryRow.SERVICE;
    EventContainer.ServiceLevel = inventoryRow.SERVICE_LEVEL;
    EventContainer.ImpactFlag = 1;

    Log("AA_EnrichAlert: enrichment applied for Serial=" + EventContainer.Serial
        + ", CustomerID=" + EventContainer.CustomerID
        + ", Service=" + EventContainer.Service);
} else {
    EventContainer.CustomerID = "NOT_FOUND";
    EventContainer.Service = "NOT_FOUND";
    EventContainer.ServiceLevel = "NOT FOUND";
    EventContainer.ImpactFlag = 2;

    Log("AA_EnrichAlert: no inventory record found for Serial="
        + EventContainer.Serial + ", Node=" + nodeValue
        + "; ImpactFlag set to 2");
}
```

Task User Interface

Figure 49: AA_EnrichAlert policy updated with ServiceLevel assignment

6.2 Write the escalation policy

Objective In this exercise, you will create a policy that runs on a schedule, queries the ObjectServer for unacknowledged Platinum-tier alerts that have been open longer than the age threshold, and escalates each one — either raising its severity by one level or, if it is already at severity 5, setting `Flash = 1`. By the end of the exercise, you will understand how to apply the `GetByFilter` and `DirectSQL` pattern to targeted, time-based escalation.

Prerequisites

- Completion of section 6.1.
- The OMNIbus ObjectServer is running.
- You are logged in to the Impact GUI.

Background

Time calculations in IPL IPL provides several functions for working with time. One that is particularly useful in working with events is:

- **`GetDate()`** — returns the current server time as a Unix epoch integer (seconds since 1 January 1970).

ObjectServer timestamp fields such as `FirstOccurrence` and `LastOccurrence` are also stored as Unix epoch integers, so you can compare them directly against `GetDate()` using plain arithmetic. For example, to calculate how many seconds an alert has been open:

```
ageSeconds = GetDate() - alert.FirstOccurrence;
```

This arithmetic is used directly in the `GetByFilter` filter to select only alerts that have been open long enough to qualify for escalation.

Severity and Flash `Severity` in `alerts.status` is an integer field with values from 1 (Indeterminate) to 5 (Critical). The policy checks the current value before acting — if it is below 5, severity is raised by one; if it is already at 5, the policy sets `Flash = 1` instead. Write-back is done with `DirectSQL`:

```
if (alert.Severity < 5)
{
    DirectSQL("defaultobjectserver", "update alerts.status set Severity=" + (alert.Severity + 1)
        + " where Identifier='" + alert.Identifier + "'", false);
}
```

```
else
{
    DirectSQL("defaultobjectserver", "update alerts.status set Flash=1"
        + " where Identifier='" + alert.Identifier + "'", false);
}
```

`Flash` is a standard integer field in `alerts.status`. Setting it to 1 signals to compatible event consoles that the alert row should flash to attract attention. It is also a useful filter condition — once `Flash = 1` is set on a severity 5 alert, the `GetByFilter` filter excludes it from future cycles.

Why a PolicyActivator, not an OMNIbusEventReader An OMNIbusEventReader fires when an alert is inserted or updated. An alert that is simply aging — sitting unacknowledged with no field changes — will not trigger an update event, so an OMNIbusEventReader would never see it cross the time threshold.

A PolicyActivator runs on a fixed schedule regardless of event activity. On each cycle it queries the ObjectServer using `GetByFilter` with a filter that includes the age condition, finds all currently qualifying alerts, and processes each one. This is the correct pattern for any escalation logic where the trigger is the passage of time rather than a field change.

Tasks

1. Select the **Policies** tab from the top navigation bar. In the toolbar, click the **dropdown arrow** next to the New Policy icon and select **IPL Policy**.
2. Click the **Save** icon, enter `AA_EscalatePlatinum` in the **Policy Name** field, and click **Save**.
3. Enter the following policy code in the editor:

```
/*
 * Policy: AA_EscalatePlatinum
 * Purpose: Escalate unacknowledged Platinum-tier alerts open longer than
 *          the age threshold.
 *          - Severity < 5: raise severity by 1.
 *          - Severity = 5: set Flash = 1 (already at maximum severity).
 * Trigger: PolicyActivator (scheduled — not event-driven).
 */

ageThreshold = 60;

filter = "ServiceLevel = 'Platinum'"
    + " AND Acknowledged = 0"
    + " AND Severity >= 3"
    + " AND FirstOccurrence < getdate() - " + ageThreshold
    + " AND NOT (Severity = 5 AND Flash = 1)";

alerts = GetByFilter("EVENTS", filter, False);
alertCount = Length(alerts);

Log("AA_EscalatePlatinum: " + alertCount + " Platinum alert(s) qualify for escalation");

if (alertCount > 0) {

    counter = 0;
    while (counter < alertCount) {

        alert = alerts[counter];
        ageSeconds = GetDate() - alert.FirstOccurrence;

        if (alert.Severity < 5) {

            DirectSQL("defaultobjectserver",
                "update alerts.status set Severity=" + (alert.Severity + 1)
```

```
                + " where Identifier='" + alert.Identifier + "'", false);

            Log("AA_EscalatePlatinum: severity raised to " + (alert.Severity + 1)
                + " for Node=" + alert.Node
                + " (age=" + ageSeconds + "s)");

        } else {

            DirectSQL("defaultobjectserver",
                "update alerts.status set Flash=1"
                + " where Identifier='" + alert.Identifier + "'", false);

            Log("AA_EscalatePlatinum: Flash set for Node=" + alert.Node
                + " (Severity=5, age=" + ageSeconds + "s)");
        }

        counter = counter + 1;
    }

} else {
    Log("AA_EscalatePlatinum: no Platinum alerts qualify — nothing to do");
}
```

Note: The `ageThreshold` variable is defined at the top of the policy so it is easy to find and change. In a production deployment you would set this to `600` (10 minutes) or higher. For this lab, `60` seconds allows you to observe the escalation without waiting. To escalate different service tiers at different rates, create separate policies — one per tier — each with its own `GetByFilter` filter and `ageThreshold`, and bind each to its own PolicyActivator service with an independent polling interval.

4. Click the **Syntax Check** icon. Confirm the **Syntax check successful** dialog appears, then click **Close**.
5. Click the **Save** icon to save the policy. Confirm that `AA_EscalatePlatinum` appears near the top of the policy list.

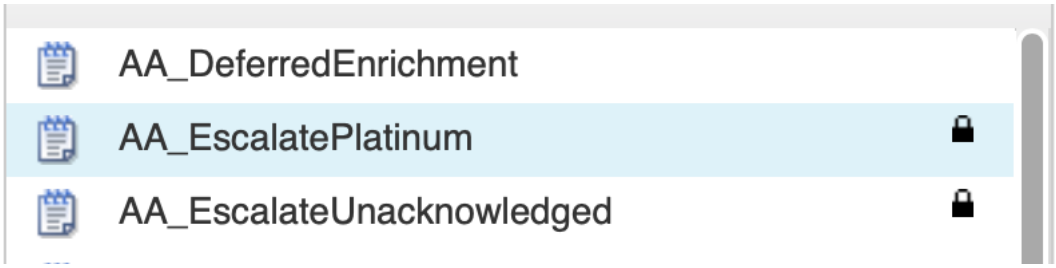


Figure 50: AA_EscalatePlatinum policy in policy list

Validation The exercise is successful when all of the following are true:

- The `AA_EscalatePlatinum` policy is saved and the syntax check passes.
- You can describe the two escalation paths — severity bump for alerts below 5, Flash for alerts already at 5.
- You can explain why a PolicyActivator is used instead of an OMNIbusEventReader for age-based escalation.
- You can explain why `DirectSQL` is used for write-back rather than `ReturnEvent`.
- You can explain what each clause of the `GetByFilter` filter does.

Expected result You have a scheduled escalation policy that queries for qualifying Platinum-tier alerts on each cycle and escalates each one. The next exercise creates the PolicyActivator service that runs it.

6.3 Create and configure the escalation service

Objective In this exercise, you will create a PolicyActivator service that runs the `AA_EscalatePlatinum` policy on a 60-second interval.

Prerequisites

- Completion of section 6.2.
- The `AA_EscalatePlatinum` policy is saved.

Background The PolicyActivator service is deliberately simple: it invokes one assigned policy when the service starts, if configured to do so, and then at a fixed activation interval. Unlike an OMNIbusEventReader, it does not process an inbound event stream and therefore has no event filter or event-to-policy mapping table. The policy receives no triggering event, so it must select the records it will process. In this lab, the escalation policy uses `GetByFilter` to identify Platinum alerts that are eligible for escalation.

This lab uses a 60-second activation interval so you can observe the escalation workflow promptly. The activation interval determines the policy's evaluation cadence, not the escalation threshold itself. In production, select an interval based on the acceptable delay after an alert becomes eligible: with a 300-second interval, an alert that crosses the threshold immediately after a run may not be evaluated until nearly five minutes later. IBM notes that smaller PolicyActivator intervals provide more accurate timing for scheduled work.

Tasks

1. Select the **Services** tab from the top navigation bar.
2. In the toolbar, click the **dropdown arrow** next to the New Service icon and select **PolicyActivator**.

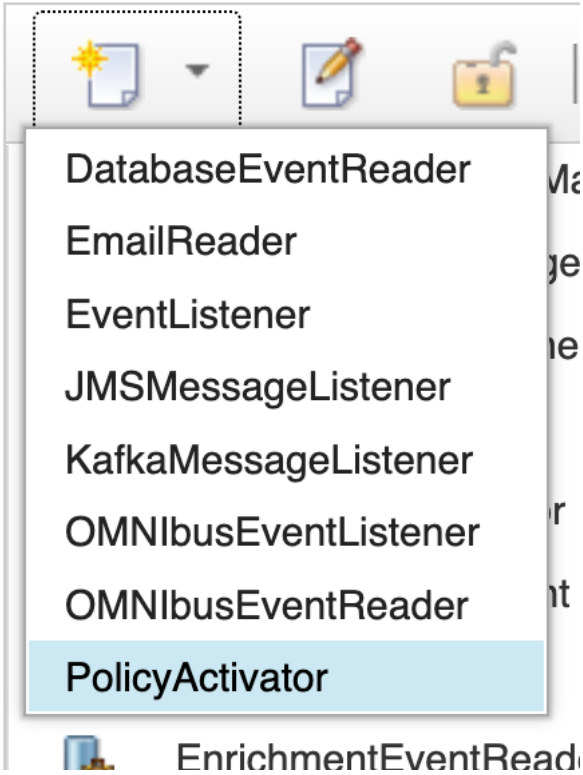


Figure 51: New Service dropdown with PolicyActivator selected

3. Configure the following fields:
   - **Service Name**: `EscalationActivator`
   - **Activation Interval (seconds)**: `60`
   - **Policy**: select `AA_EscalatePlatinum` from the dropdown
   - Leave **Starts automatically when server starts** unchecked.
4. Click the **Save** icon to save the service.
5. Right-click on your new `EscalationActivator` serice and select the **Start** menu item to start the service. Confirm the state changes to **Running**.

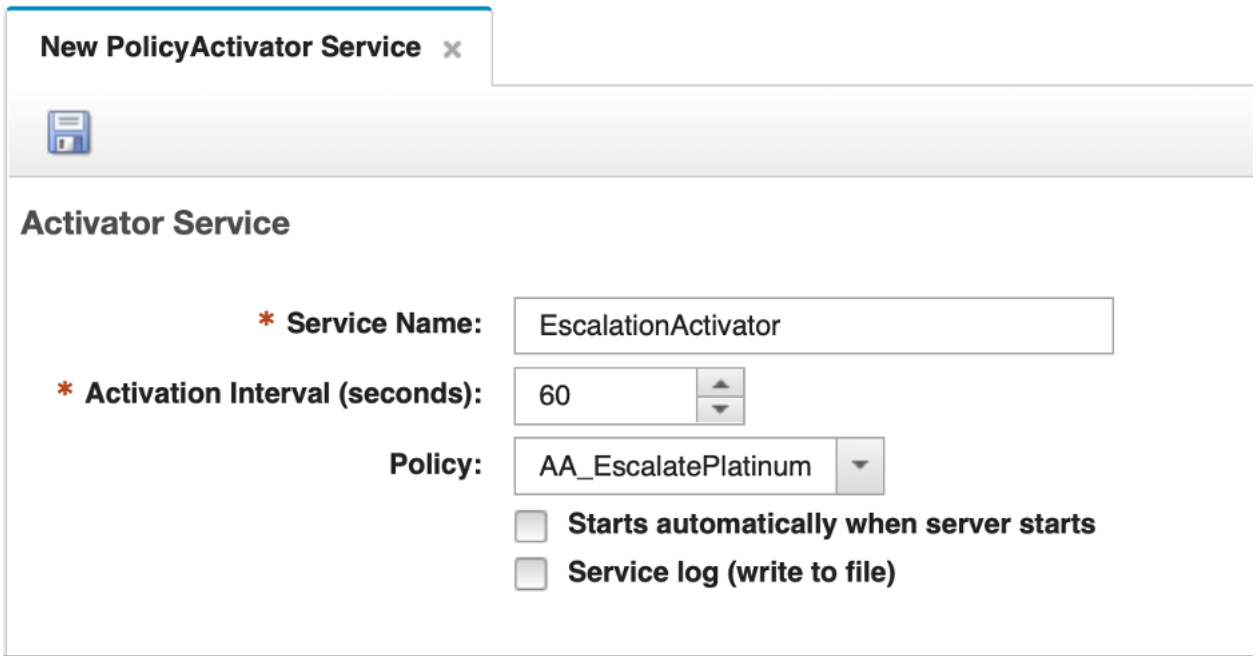


Figure 52: EscalationActivator PolicyActivator configuration

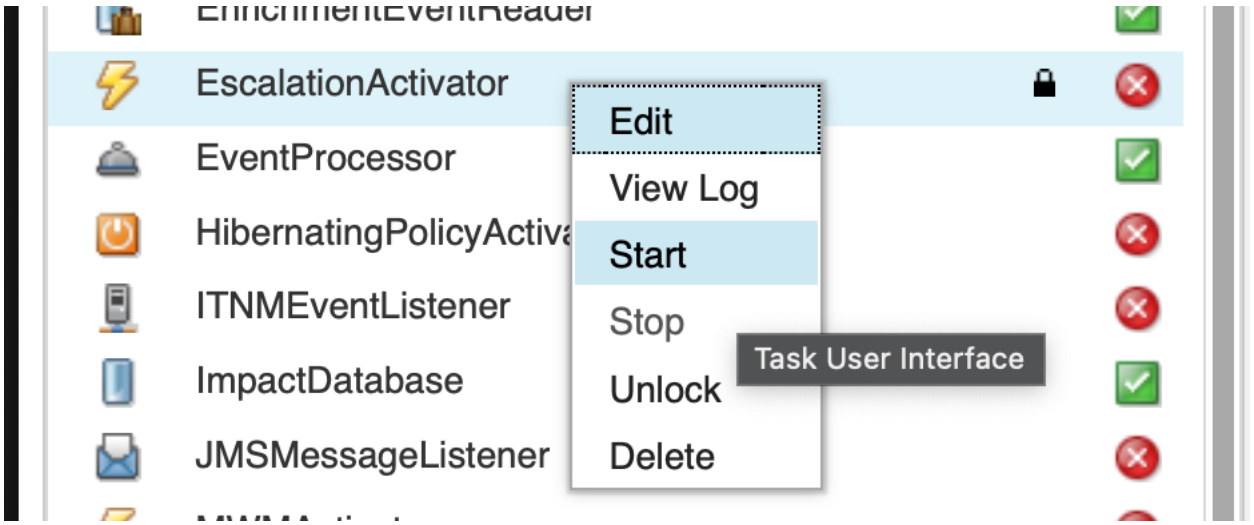


Figure 53: EscalationActivator service in Running state

6. The `EnrichmentEventReader` service caches the `alerts.status` schema at startup. Because you added the `ServiceLevel` column after it was started, it needs to be restarted to pick up the new field — otherwise it will not populate `ServiceLevel` when writing events back to the ObjectServer.

   In the services list, right-click `EnrichmentEventReader` and select **Stop**. Wait for the state to change to **Stopped**, then right-click again and select **Start**. Confirm the state returns to **Running**.

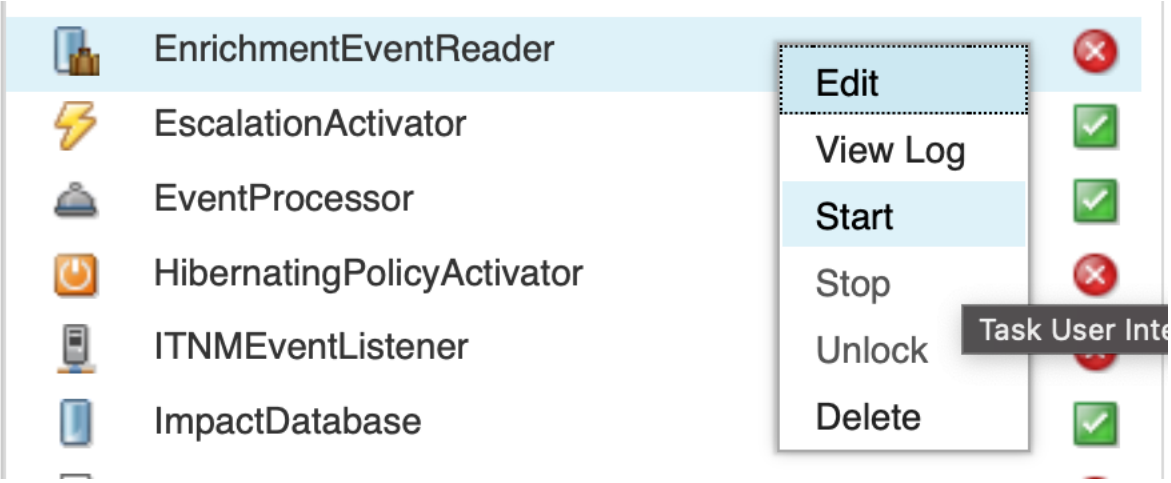


Figure 54: EnrichmentEventReader restarted and Running

Validation The exercise is successful when all of the following are true:

- The `EscalationActivator` service is saved with a 60-second activation interval and `AA_EscalatePlatinum` assigned.
- Both `EscalationActivator` and `EnrichmentEventReader` are in **Running** state.
- You can explain why the PolicyActivator has no filter or event mapping, and where the targeting logic lives instead.
- You can explain why the `EnrichmentEventReader` needed to be restarted.

Expected result The escalation pipeline is now active. Every 60 seconds the `EscalationActivator` wakes up and runs `AA_EscalatePlatinum`, which queries for qualifying Platinum-tier alerts and escalates each one found.

6.4 Validate event escalation

Objective In this exercise, you will insert test alerts for a Platinum-tier node and confirm that the escalation policy correctly raises severity for aged alerts and sets `Flash` for severity 5 alerts. You will also confirm that alerts from non-Platinum nodes are not escalated.

Prerequisites

- Completion of section 6.3.
- The `EscalationActivator` service is running.

Tasks

1. In the OMNIbus Web GUI, enable Flash notifications in your event viewer so you will see the visual effect when `Flash = 1` is set on an alert:
   - Click the **Workspace Preferences** icon in the event viewer toolbar (the slider/equaliser icon to the left of the quick filter text box).
   - Click the **Notifications** tab.
   - Check **Enable Notifications** and **Flashing**.
   - Click **Apply and Save** button at the bottom of the **Workspace Preferences** window.
2. In the OMNIbus Web GUI, set up your event view to show the `ServiceLevel` field:
   - Navigate to the **Event Viewer** and select the **ImpactLab Events** filter.
   - Select the **SyslogdView** view.

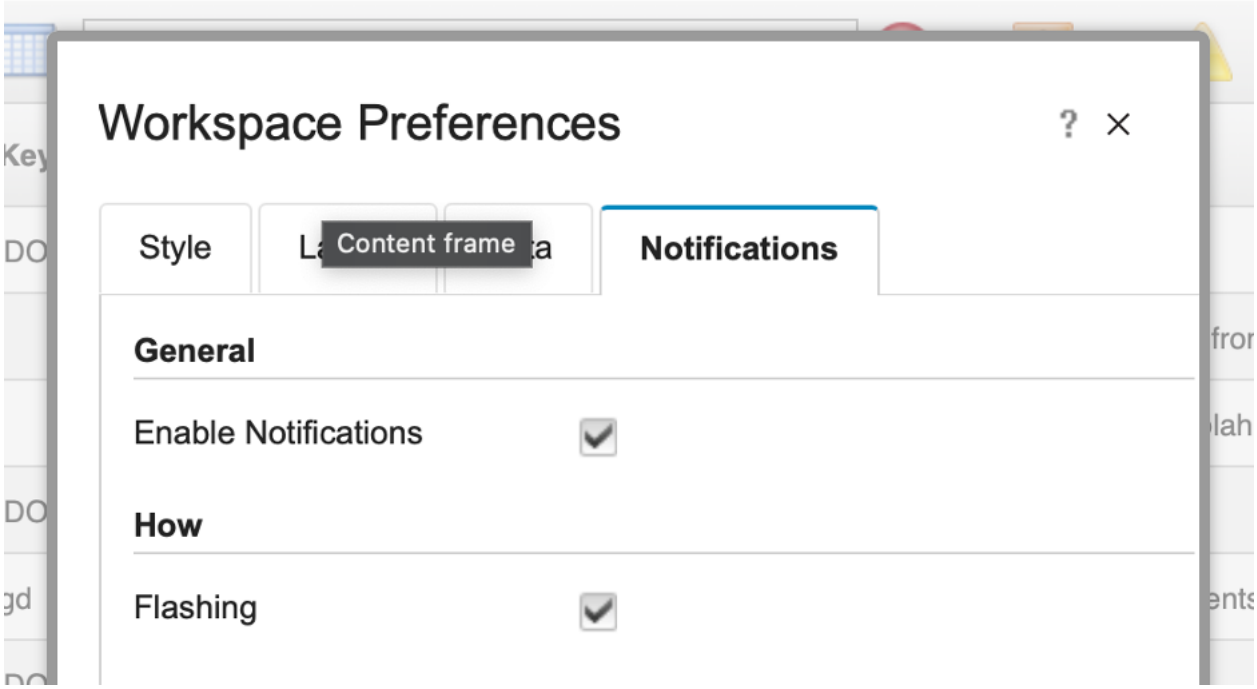


Figure 55: Workspace Preferences — Notifications tab with Enable Notifications and Flashing checked

- Click the **Edit View** icon to open the view editor.
- In the column list, locate the `Service` field. Add a new column for `ServiceLevel` immediately after it.
- Set the column width to `10` characters.
- Save and close the view editor.

3. Insert a severity 3 test alert for `router-lon-01` — a Platinum-tier node:

```
$OMNIHOME/bin/nco_sql -server EVENTS -user root -password netcool << 'EOF'
INSERT INTO alerts.status (Node, Summary, AlertKey, AlertGroup, Severity, Type, Identifier,
 ↪  Manager, OwnerUID)
VALUES ('router-lon-01', 'Escalation test — Platinum node', 'escalate-platinum-01',
 ↪  'TrainingAlerts', 3, 1, 'router-lon-01:escalate-platinum-01:TrainingAlerts',
 ↪  'ImpactLab', 65534);
go
EOF
```

4. Insert a severity 3 test alert for `switch-nyc-01` — a non-Platinum node — to confirm it is not escalated:

```
$OMNIHOME/bin/nco_sql -server EVENTS -user root -password netcool << 'EOF'
INSERT INTO alerts.status (Node, Summary, AlertKey, AlertGroup, Severity, Type, Identifier,
 ↪  Manager, OwnerUID)
VALUES ('switch-nyc-01', 'Escalation test — non-Platinum node', 'escalate-nonplatinum-01',
 ↪  'TrainingAlerts', 3, 1, 'switch-nyc-01:escalate-nonplatinum-01:TrainingAlerts',
 ↪  'ImpactLab', 65534);
go
EOF
```

5. Insert a severity 3 test alert for `router-lon-02` — the second Platinum-tier node:

```
$OMNIHOME/bin/nco_sql -server EVENTS -user root -password netcool << 'EOF'
INSERT INTO alerts.status (Node, Summary, AlertKey, AlertGroup, Severity, Type, Identifier,
 ↪  Manager, OwnerUID)
VALUES ('router-lon-02', 'Escalation test — Platinum node', 'escalate-platinum-02',
 ↪  'TrainingAlerts', 3, 1, 'router-lon-02:escalate-platinum-02:TrainingAlerts',
 ↪  'ImpactLab', 65534);
go
```

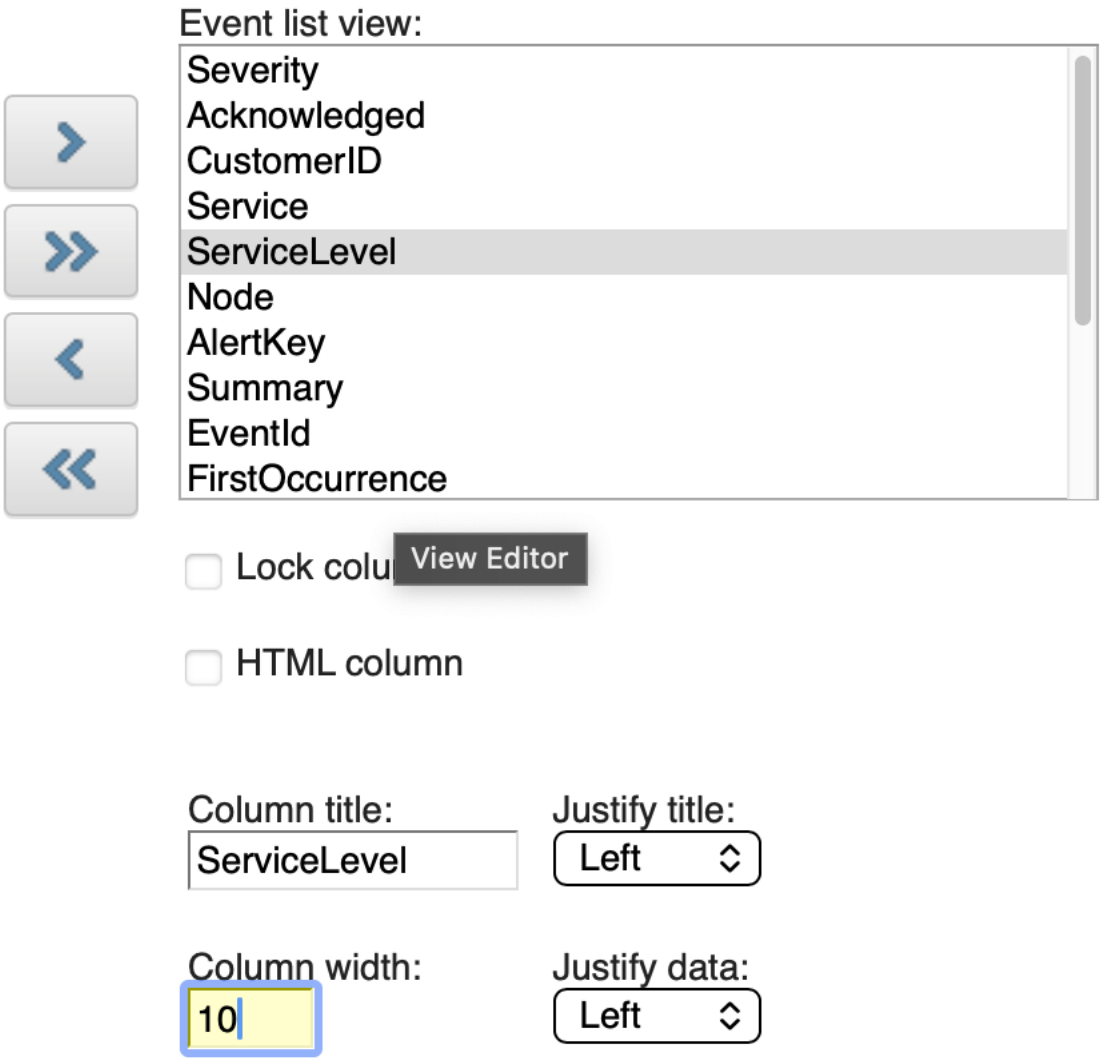


Figure 56: SyslogdView editor showing ServiceLevel column with width set to 10

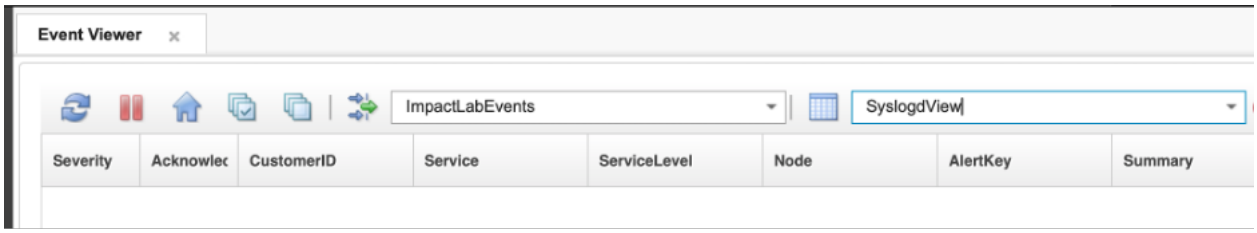


Figure 57: SyslogdView with ServiceLevel column added between Service and Node

```
EOF
```

6. Wait 60–90 seconds for the enrichment service to process all three alerts. In the OMNIbus Web GUI, locate them in the `ImpactLab Events` filter. Confirm:
   - `router-lon-01` and `router-lon-02` both have `ServiceLevel = Platinum`.
   - `switch-nyc-01` has `ServiceLevel` populated with its own service level (or `NOT_FOUND` if not in the CMDB).

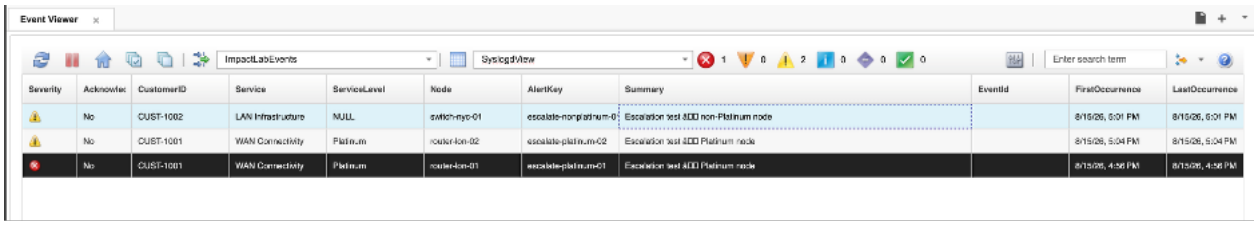


Figure 58: All three test alerts in the event list after enrichment

7. Wait a further 60–90 seconds for the `EscalationActivator` to fire. Once the Platinum alerts have been open for more than 60 seconds, they will match the `GetByFilter` query and their severity will be raised.

   Confirm in the event list that:
   - `router-lon-01` and `router-lon-02` severity has changed from 3 to 4.
   - `switch-nyc-01` severity remains at 3.
8. Wait another 60–90 seconds. Confirm both `router-lon-01` and `router-lon-02` severity raises again from 4 to 5.
9. Wait another 60–90 seconds. Both Platinum alerts are now at severity 5. The policy will set `Flash = 1` on each. Double-click one of them and confirm `Flash` shows 1 in the event details.

   On the next cycle both alerts no longer match the `GetByFilter` filter (`NOT (Severity = 5 AND Flash = 1)` excludes them), so escalation stops.

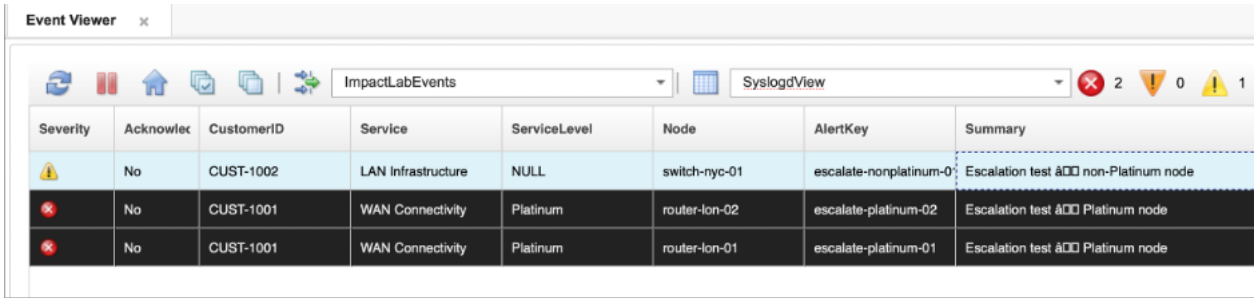


Figure 59: Platinum alerts at severity 5 with Flash=1

10. Check the policy log to review the escalation trail:

```
tail -50 $IMPACT_HOME/logs/NCI_policylogger_AA_EscalatePlatinum.log
```

    You should see log entries for `router-lon-01` and `router-lon-02` at each cycle showing severity being raised, then the Flash entries, then silence as both alerts drop out of the filter. You should see no entries for `switch-nyc-01`.
11. Acknowledge `router-lon-01` in the event list. Confirm that on the next cycle the alert does not appear in the policy log — it no longer matches `Acknowledged = 0` in the `GetByFilter` filter.

Validation The exercise is successful when all of the following are true:

- `router-lon-01` progresses from severity 3 → 4 → 5 → Flash=1 across successive polling cycles.
- `switch-nyc-01` is not escalated at any point.
- Once `Flash = 1` is set on `router-lon-01`, the alert no longer matches the `GetByFilter` filter and escalation stops.
- After acknowledgment, `router-lon-01` is excluded from the `GetByFilter` filter and no longer appears in the policy log.
- The policy log shows entries only for Platinum-tier alerts.
- You can describe the complete escalation path and explain why each step stops at the right point.

Expected result You have now built a service-level-aware, scheduled escalation pipeline. The PolicyActivator fires on a fixed interval, the policy queries for qualifying Platinum-tier alerts, and each one is escalated one step per cycle until it reaches maximum escalation state or is acknowledged. Non-Platinum alerts are unaffected. This pattern extends naturally to more complex scenarios: tiered escalation across multiple service levels, on-call notifications, ticket creation, or routing alerts to different teams based on service classification.

7. Automation and hibernation

Not every alert that arrives in the ObjectServer represents a genuine, persistent fault. Network devices can throw transient errors that clear within seconds — a momentary link flap, a brief CPU spike, or a probe timeout that does not repeat. Treating every one of these as a real incident wastes operator time and generates noise.

Impact can handle this automatically using **hibernation**. A policy can set a flag on an event, pause mid-execution for a specified number of seconds, and then resume exactly where it left off — at which point it can re-test the condition and decide what to do. This section uses a ping verification scenario to demonstrate the pattern: when a down event is received, Impact pings the node immediately, waits 60 seconds, and pings again. Only if the node is still unreachable after the second check does the event remain open.

This section also introduces **JRExecAction**, the IPL function for running local commands and scripts on the Impact server and capturing their output.

7.1 Write the hibernation ping policy

Objective In this exercise, you will write a policy that pings the node from a down event, hibernates for 60 seconds if the first ping fails, pings again after waking up, and updates the event with a journal entry recording the outcome. By the end of the exercise, you will understand how `Hibernate`, `RemoveHibernation`, `JRExecAction`, and a re-entry guard work together in a single policy.

Prerequisites

- Completion of sections 4 and 5.
- The `ImpactFlag` integer field exists in `alerts.status` (added in section 4.1). Verify with:

```
$OMNIHOME/bin/nco_sql -server EVENTS -user root -password netcool << 'EOF'
describe alerts.status;
go
EOF
```

  Confirm `ImpactFlag` appears as an `INTEGER` column. If it is missing, add it:

```
$OMNIHOME/bin/nco_sql -server EVENTS -user root -password netcool << 'EOF'
ALTER TABLE alerts.status ADD COLUMN ImpactFlag INTEGER;
go
EOF
```

- The `HibernatingPolicyActivator` service is running. This service is not started by default and must be started manually before any policy that uses `Hibernate` can wake up. In the Impact GUI, select the **Services** tab, locate `HibernatingPolicyActivator` in the list, right-click it, and select **Start**. Confirm the state changes to **Running**.

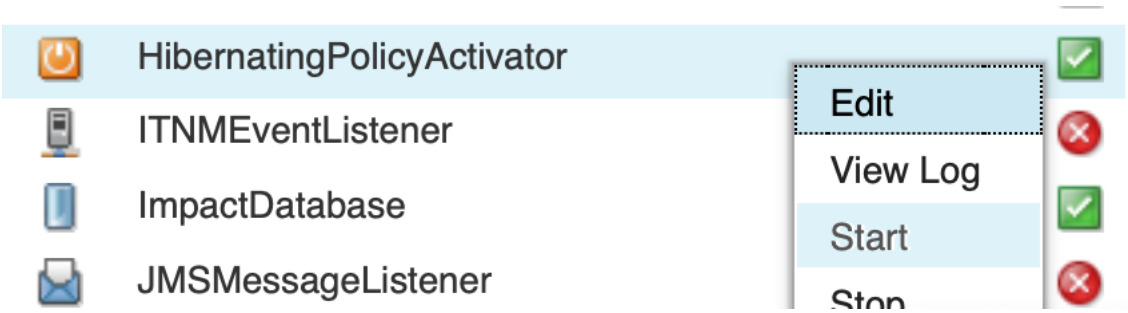


Figure 60: HibernatingPolicyActivator service in Running state

- The `nci_jrexec` process is running on the Impact server. This is a background process launched from the command line — it does not appear in the Impact GUI Services tab. SSH into the lab server and run:

```
nohup "$IMPACT_HOME/bin/nci_jrexec" \
  > "$IMPACT_HOME/logs/nci_jrexec.out" 2>&1 &
ps -ef | grep '[n]ci_jrexec'
```

  The `ps` output should show a running `nci_jrexec` process. If no output is returned, the process did not start — check `$IMPACT_HOME/logs/nci_jrexec.out` for errors.

- The value of the Node field in your test events must be pingable from the Impact server (or deliberately unpingable, to test the down path).

Background

JRExecAction `JRExecAction` runs a local command on the Impact server and returns the output as a string:

```
output = JRExecAction("/path/to/command", {arg1, arg2, ...}, false, timeoutSeconds);
```

For ping, the command path is `/usr/sbin/ping` and the node name is passed as the first argument. The output string will contain the full ping response, including newline characters. Use `Strip(output, "\n")` to collapse newlines before doing a pattern match.

```
Hibernate(ActionKey, Reason, TimeoutSeconds);
```

Hibernate and RemoveHibernation `Hibernate` suspends the executing policy thread at that exact line for `TimeoutSeconds` seconds. When the timer expires, execution continues on the very next line — the policy does not restart. All variables set before `Hibernate` are still available after waking up.

- `ActionKey` must be a unique string across all simultaneously hibernating policies. A common pattern combines a fixed prefix with `GetDate()` in combination with the `Identifier` field of the alert to make it unique per invocation.
- `Reason` is a descriptive string written to the audit log. It can be `null`.
- `RemoveHibernation(ActionKey)` cleans up the hibernation record after the policy resumes. Always call it.

The `HibernatingPolicyActivator` service manages the hibernation timer queue. Without it running, policies that call `Hibernate` will sleep indefinitely.

Re-entry guard The OMNIbusEventReader fires on every insert or update to a matching event. If the policy sets `ImpactFlag = 88` and calls `ReturnEvent` *before* hibernating, that write triggers an update on the event — which would normally cause the EventReader to fire again. Setting `ImpactFlag = 88` as the guard value ensures the sleeping event is excluded by the `ImpactFlag <> 88` clause in the ping mapping filter, preventing the policy from being triggered a second time on the same event while it sleeps.

Ping output pattern On Linux/Unix, a successful `ping` produces output that includes `0% packet loss` when the host responds. The `Strip` + `like` pattern used in the policy collapses newlines and then matches against this string:

```
If (Strip(PingOutput, "\n") like ".*received, 0% packet loss.*")
```

Tasks

1. Select the **Policies** tab from the top navigation bar. In the toolbar, click the **dropdown arrow** next to the New Policy icon and select **IPL Policy**.
2. Click the **Save** icon, enter `AA_PingVerify` in the **Policy Name** field, and click **Save**.
3. Enter the following policy code in the editor:

```
/*
 * Policy: AA_PingVerify
 * Purpose: When a down event is received, ping the node immediately.
 *          If the first ping fails, hibernate for 60 seconds then ping again.
 *          Update the event with a journal entry recording the outcome.
```

```
 *          ImpactFlag values:
 *            0 = unprocessed (EventReader filter target)
 *           88 = in progress (set before hibernating — excluded by mapping filter)
 *            2 = processing complete
 * Trigger: OMNIbusEventReader (event-driven).
 */

/* Set ImpactFlag = 88 immediately to prevent the EventReader from
 * re-triggering this policy while it is sleeping. */

EventContainer.ImpactFlag = 88;
ReturnEvent(EventContainer);

Log("AA_PingVerify: first ping attempt for Node=" + EventContainer.Node);

/* Ping the node. Output is returned as a string. */

PingOutputOne = JRExecAction("/usr/sbin/ping", { "-c", "1", "-W", "2", EventContainer.Node},
 ↪  false, 120);

/* Strip newlines and check whether the node responded. */

if (Strip(PingOutputOne, "\n") like ".*received, 0% packet loss.*") {

    /* Node is reachable — clear the event and mark complete. */

    EventContainer.Severity = 0;
    EventContainer.JournalEntry = "AA_PingVerify: node is reachable on first ping — event
 ↪  cleared";

    Log("AA_PingVerify: Node=" + EventContainer.Node + " is alive on first ping");

} else {

    /* First ping failed — hibernate for 60 seconds then try again. */

    Log("AA_PingVerify: Node=" + EventContainer.Node + " did not respond — hibernating 60s");

    ActionKey = "PingVerify:" + EventContainer.Identifier + GetDate();
    WaitTime = 60;
    Hibernate(ActionKey, "Waiting 60s before second ping", WaitTime);

    /* ---- Policy resumes here after 60 seconds ---- */

    Log("AA_PingVerify: woke up — second ping attempt for Node=" + EventContainer.Node);

    PingOutputTwo = JRExecAction("/usr/sbin/ping", { "-c", "1", "-W", "2",
 ↪  EventContainer.Node}, false, 120);

    if (Strip(PingOutputTwo, "\n") like ".*received, 0% packet loss.*") {

        /* Node recovered during the wait — clear the event. */

        EventContainer.Severity = 0;
        EventContainer.JournalEntry = "AA_PingVerify: node recovered — alive on second ping";

        Log("AA_PingVerify: Node=" + EventContainer.Node + " recovered on second ping");

    } else {
```

```
        /* Node is still unreachable — increase severity, mark complete. */
        if(EventContainer.Severity == 5) {
         EventContainer.Flash = 1;
        } else {
         EventContainer.Severity = EventContainer.Severity + 1;
        }
        EventContainer.JournalEntry = "AA_PingVerify: node unreachable on both pings —
↪  confirmed down";

        Log("AA_PingVerify: Node=" + EventContainer.Node + " still unreachable after second
↪  ping");
    }

    RemoveHibernation(ActionKey);
}

/* Commit all field changes back to the ObjectServer. */

ReturnEvent(EventContainer);
```

4. Click the **Syntax Check** icon. Confirm the **Syntax check successful** dialog appears, then click **Close**.
5. Click the **Save** icon. Confirm `AA_PingVerify` appears in the policy list.

7.2 Configure the EnrichmentEventReader to trigger the ping policy

Objective In this exercise, you will extend the existing `EnrichmentEventReader` service to also trigger `AA_PingVerify`. Rather than creating a new service, you will enable **Get Updated Events** on the service and add a second event mapping row that targets enriched ping-failure events by their `EventId` and `ImpactFlag` value. By the end of the exercise, a single OMNIbusEventReader handles both the enrichment mapping and the ping verification mapping.

Prerequisites

- Completion of section 7.1.
- The `AA_PingVerify` policy is saved.
- The `EnrichmentEventReader` service exists and is running (created in section 4.2).

Background An OMNIbusEventReader service can hold multiple rows in its Event Mapping table. Each row has its own filter and its own policy. When an event is inserted or updated, the service evaluates every row in order and invokes the policy for each row whose filter matches.

By default, an OMNIbusEventReader only fires on **inserted** events. Enabling **Get Updated Events** causes it to also fire whenever an existing event is updated — which is necessary here because `AA_PingVerify` is triggered by an update to an already-enriched event (when `ImpactFlag` is written back by `AA_EnrichAlert`), not by the original insert.

The filter `ImpactFlag > 0 AND ImpactFlag <> 88 AND EventId = 'PingFailure'` targets events that:

- Have already been processed by the enrichment policy (`ImpactFlag > 0` — not new/unprocessed)
- Have not been excluded by an administrative hold (`ImpactFlag <> 88` — a sentinel value reserved for events that should be left alone)
- Specifically identify themselves as ping failure events (`EventId = 'PingFailure'`)

This means `AA_PingVerify` will only run on events that have been enriched first and carry the `PingFailure` identifier — not on every event the service sees.

Tasks

1. Select the **Services** tab from the top navigation bar. Double-click `EnrichmentEventReader` to open it.
2. In the **Event Mapping** tab, check the **Get Updated Events** checkbox to enable firing on event updates as well as inserts.
3. In the Event Mapping table, add a second mapping row beneath the existing `AA_EnrichAlert` row:

- **Filter**: `ImpactFlag > 0 AND ImpactFlag <> 88 AND EventId = 'PingFailure'`
- **Policy**: `AA_PingVerify`

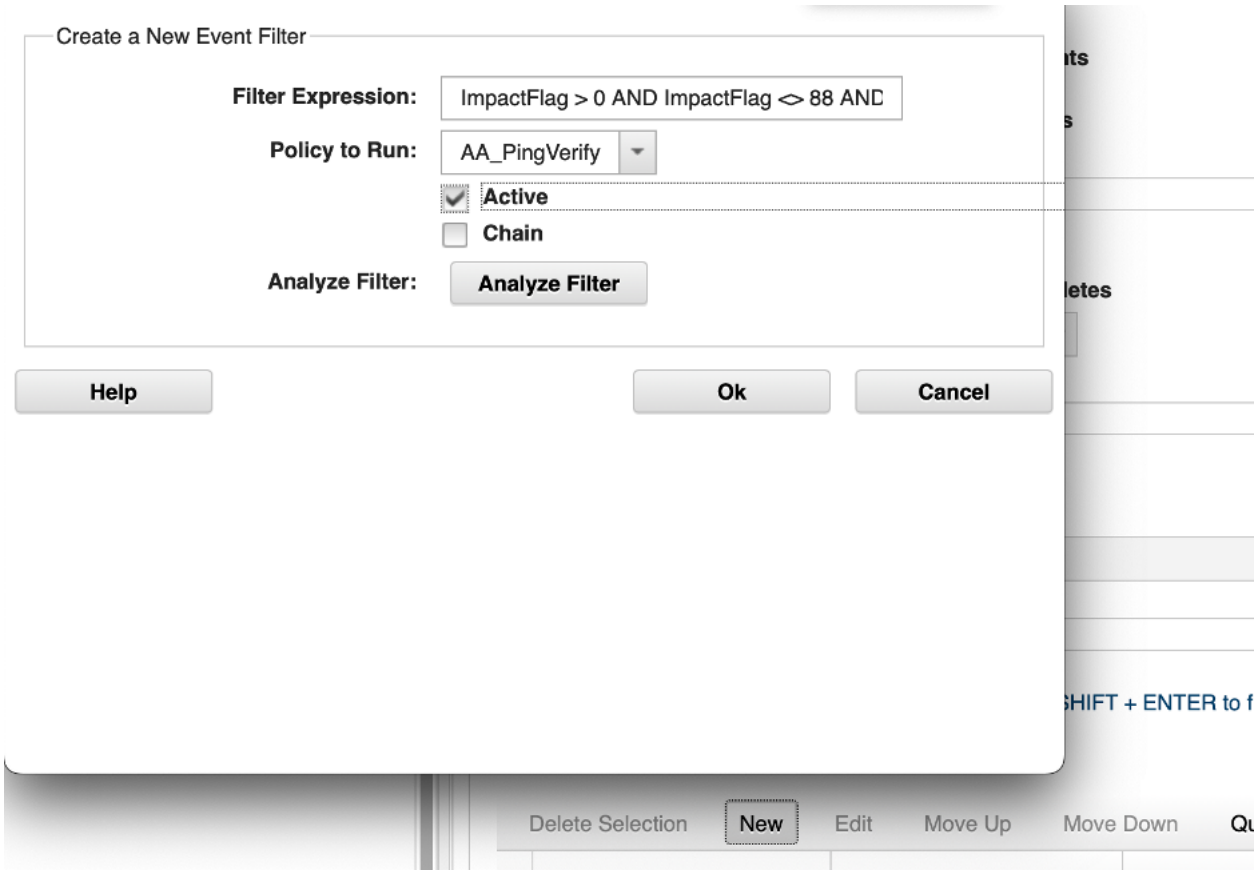


Figure 61: New AA_PingVerify mapping row being entered in the Event Mapping table

4. Confirm the Event Mapping table now shows both rows — the original `AA_EnrichAlert` row and the new `AA_PingVerify` row with its filter:
5. Click the **Save** icon to save the service changes.
6. Right-click `EnrichmentEventReader` and select **Stop**. Wait for the state to change to **Stopped**, then right-click again and select **Start**. Confirm the state returns to **Running**.

   Note: A stop/start is required whenever you change an OMNIbusEventReader's Event Mapping configuration. The service loads its mapping table at startup; changes made while the service is running do not take effect until it is restarted.

Validation The exercise is successful when all of the following are true:

- The `EnrichmentEventReader` service has **Get Updated Events** enabled.
- The Event Mapping table contains two rows: the existing `AA_EnrichAlert` row and the new `AA_PingVerify` row with filter `ImpactFlag > 0 AND ImpactFlag <> 88 AND EventId = 'PingFailure'`.
- The service has been stopped and restarted and is in **Running** state.
- The `HibernatingPolicyActivator` service is **Running** and the `nci_jrexec` process is running on the Impact server.
- You can explain why **Get Updated Events** is needed for this mapping.
- You can explain what `ImpactFlag <> 88` guards against.

7.3 Validate the hibernation policy

Objective In this exercise, you will insert test events for a reachable node and an unreachable node and observe the policy outcome in both cases. You will also verify the hibernation behavior through the policy log.

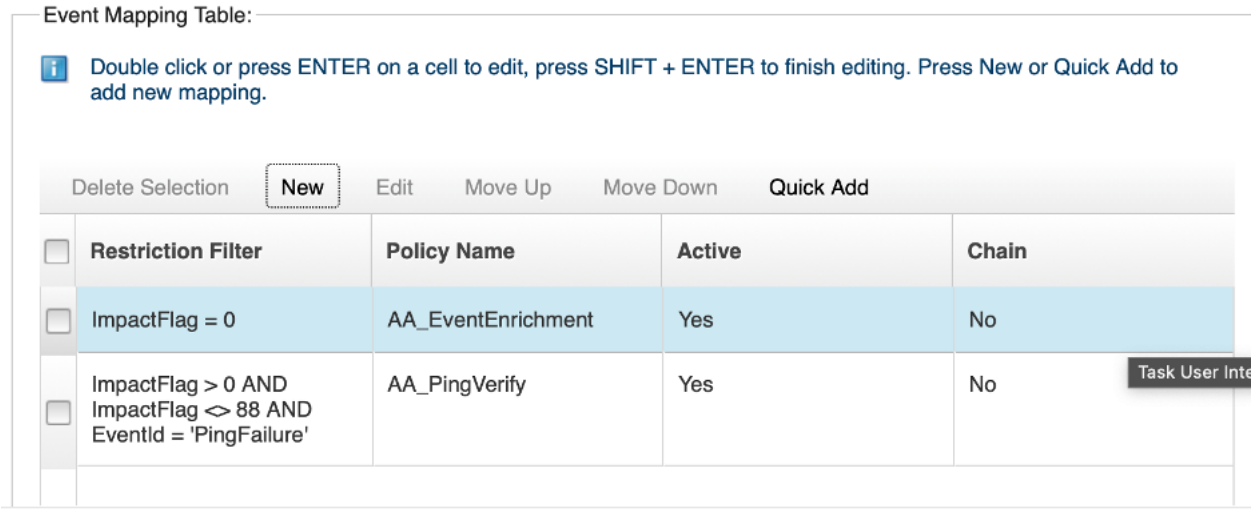


Figure 62: Event Mapping table showing both AA_EnrichAlert and AA_PingVerify rows



Figure 63: EnrichmentEventReader service in Running state after restart

Prerequisites

- Completion of section 7.2.
- At least one hostname that is pingable from the Impact server and one that is not.

Background Test events need `EventId = 'PingFailure'` set at insert time so the `EnrichmentEventReader` mapping filter matches them. The enrichment policy (`AA_EnrichAlert`) runs first (on `ImpactFlag = 0`), sets `ImpactFlag = 1`, and calls `ReturnEvent`. That update then triggers the second mapping row, which fires `AA_PingVerify` because `ImpactFlag > 0 AND ImpactFlag <> 88 AND EventId = 'PingFailure'` now matches. A ping falure alert that is hibernating will have their ImpactFlag set to 88 to ensure it is not processed whilst already being worked on.

Tasks

1. In the OMNIbus Web GUI (open in your private/incognito browser), navigate to the **Event Viewer**. Select the **ImpactLab Events** filter and the **SyslogdView** view. Leave this window visible so you can observe events being processed in real time as you work through the tasks below.
2. Insert a test event for a **reachable** node. Use `localhost` — the Impact server can always ping itself:

```
$OMNIHOME/bin/nco_sql -server EVENTS -user root -password netcool << 'EOF'
INSERT INTO alerts.status (Node, Summary, AlertKey, AlertGroup, Severity, Type, Identifier,
 ↪  Manager, OwnerUID, EventId)
VALUES ('127.0.0.1', 'Ping verify test — reachable node', 'ping-test-01', 'TrainingAlerts',
 ↪  3, 1, 'localhost:ping-test-01:TrainingAlerts', 'ImpactLab', 65534, 'PingFailure');
go
EOF
```

3. Wait 30–60 seconds for enrichment to run, then wait a further 30–60 seconds for `AA_PingVerify` to complete. Check the event in the OMNIbus Web GUI. Confirm:

- `Severity` has been set to `0` (cleared).
- `ImpactFlag` is 2.
- The journal entry reads `AA_PingVerify: node is reachable on first ping — event cleared`.

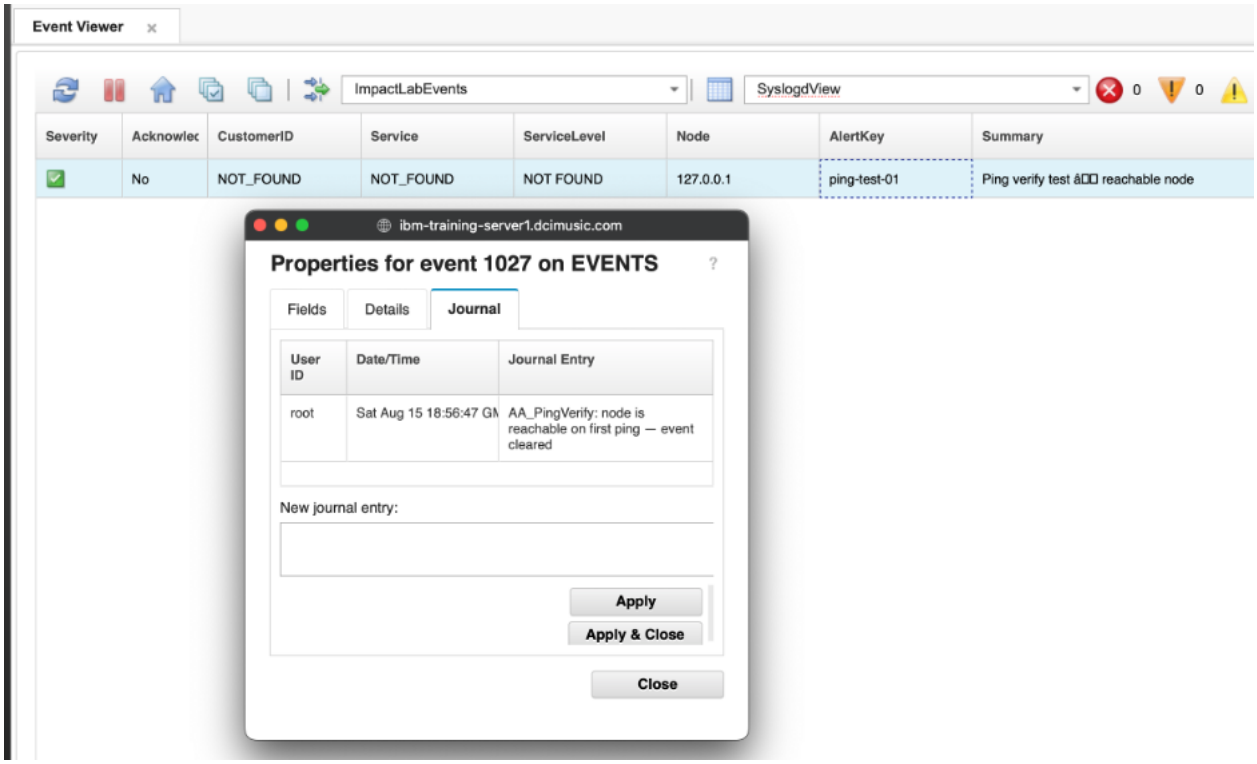


Figure 64: Event list showing 127.0.0.1 alert cleared at Severity 0 after first ping

This event was cleared on the first ping without hibernating.

4. Insert a test event for an **unreachable** node:

```
$OMNIHOME/bin/nco_sql -server EVENTS -user root -password netcool << 'EOF'
INSERT INTO alerts.status (Node, Summary, AlertKey, AlertGroup, Severity, Type, Identifier,
 ↪  Manager, OwnerUID, EventId)
VALUES ('192.0.2.1', 'Ping verify test — unreachable node', 'ping-test-02', 'TrainingAlerts',
 ↪  3, 1, '192.0.2.1:ping-test-02:TrainingAlerts', 'ImpactLab', 65534, 'PingFailure');
go
EOF
```

> **Note:** `192.0.2.1` is from the TEST-NET-1 range (RFC 5737), reserved for documentation and unlikely to be routable in most environments. However, depending on your network configuration it may respond. If the policy clears the event instead of hibernating, choose a different address that is definitively unreachable from the Impact server and repeat this task with that address.

5. Watch the policy log to observe the hibernation in real time:

```
tail -f $IMPACT_HOME/logs/NCI_policylogger_AA_PingVerify.log
```

You should see the following sequence of entries:

- `AA_PingVerify: first ping attempt for Node=192.0.2.1`
- `AA_PingVerify: Node=192.0.2.1 did not respond — hibernating 60s`
- *(~60 second pause)*
- `AA_PingVerify: woke up — second ping attempt for Node=192.0.2.1`
- `AA_PingVerify: Node=192.0.2.1 still unreachable after second ping`

Press `Ctrl+C` to stop tailing the log once you have seen all entries.

6. After the policy completes, confirm in the OMNIbus Web GUI that:

- `Severity` has been raised from 3 to 4 (the policy bumped it on the confirmed-down path).
- The journal entry reads `AA_PingVerify: node unreachable on both pings — confirmed down`.

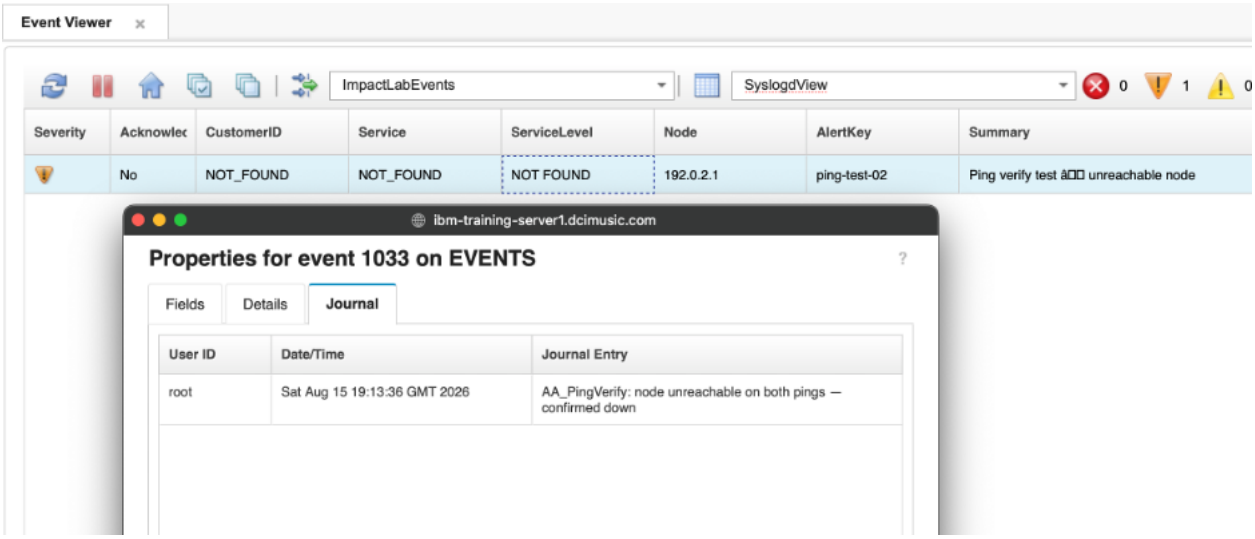


Figure 65: Event list showing 192.0.2.1 alert with severity raised after both pings failed

Validation The exercise is successful when all of the following are true:

- `127.0.0.1` was enriched, then resolved on the first ping and the event was cleared immediately (no hibernation).
- `192.0.2.1` was enriched, failed both pings, and the policy log shows the 60-second gap between ping attempts.
- After the second failed ping, the event severity was raised by one and the correct journal entry was set.
- A second `192.0.2.1` event triggered the full pipeline again.
- You can explain why `EventId = 'PingFailure'` must be set in the insert for the mapping filter to match.
- You can explain why **Get Updated Events** is required for `AA_PingVerify` to fire.
- You can explain why `ImpactFlag = 88` is set and `ReturnEvent` is called *before* `Hibernate`.
- You can explain what would happen if `RemoveHibernation` were not called.
- You can explain why the `HibernatingPolicyActivator` service must be running.

Expected result You have built a verification loop that uses hibernation to add a deliberate delay between two checks. Events for genuinely reachable nodes are cleared on the first pass with no hibernation at all. Events for truly unreachable nodes go through the full 60-second wait, re-check, and are confirmed as down. The same guard-hibernate-resume-cleanup pattern applies to any scenario where a policy needs to wait before acting — retry logic, deferred notifications, or staged remediation workflows.

8. Making a Java call

IPL gives you direct access to the Java runtime through the `JavaCall` function. This means any Java class available on the Impact server's classpath — whether from the standard library or a custom JAR — can be invoked from policy code. The result can be used immediately in IPL, giving you access to the full breadth of Java functionality without leaving the policy editor.

This section demonstrates the pattern using `java.util.Arrays.sort()`, a straightforward example that is easy to verify and clearly illustrates the calling convention. The same pattern applies to any static method or instance method on any Java class.

8.1 Write a policy that sorts an array using a Java call

Objective In this exercise, you will write a policy that builds an IPL array of strings, calls the static Java method `java.util.Arrays.s` to sort it in place, and logs the result. By the end of the exercise, you will understand the four-argument `JavaCall` signature and how IPL arrays map to Java arrays.

Prerequisites

- Completion of sections 4 through 6, or familiarity with IPL policy creation.
- You are logged in to the Impact GUI.

Background

The JavaCall function `JavaCall` has the following signature:

```
JavaCall(ClassName, ObjectInstance, MethodName, { arg1, arg2, ... });
```

Argument	Purpose
`ClassName`	Fully qualified Java class name (e.g. `"java.util.Arrays"`)
`ObjectInstance`	The Java object to invoke the method on. Use `null` for static methods
`MethodName`	The method name as a string (e.g. `"sort"`)
`{ arg1, ... }`	An IPL array of arguments to pass to the method

For static methods — methods that belong to the class rather than a particular instance — pass `null` as `ObjectInstance`. `java.util.Arrays.sort()` is a static method, so the call looks like:

```
JavaCall("java.util.Arrays", null, "sort", { words });
```

IPL arrays passed to `JavaCall` are automatically bridged to Java arrays and back. When `sort` is called, it sorts the array *in place*, meaning the original `words` variable in IPL reflects the sorted order immediately after the call returns — no return value needs to be captured.

Tasks

1. Select the **Policies** tab from the top navigation bar. In the toolbar, click the **dropdown arrow** next to the New Policy icon and select **IPL Policy**.
2. Click the **Save** icon, enter `AA_JavaSortDemo` in the **Policy Name** field, and click **Save**.
3. Enter the following policy code in the editor:

```
/*
 * Policy: AA_JavaSortDemo
 * Purpose: Demonstrate JavaCall by sorting an array of strings using
 *          the static method java.util.Arrays.sort().
 * Trigger: PolicyActivator (run manually via a one-time activation).
 */

/* Build an unsorted array of strings. */

words = {"zipper", "apple", "carrot"};

Log("AA_JavaSortDemo: before sort: " + words);

/* Call the static Java method java.util.Arrays.sort().
 * null = no object instance (static call).
 * The array is sorted in place — words is updated directly. */

JavaCall("java.util.Arrays", null, "sort", { words });

Log("AA_JavaSortDemo: after sort: " + words);
```

4. Click the **Syntax Check** icon. Confirm the **Syntax check successful** dialog appears, then click **Close**.
5. Click the **Save** icon to save the policy.
6. Open a terminal on the Impact server. The policy log file is not created until the policy runs for the first time, so create it now so that `tail` has a file to follow:

```
touch $IMPACT_HOME/logs/NCI_policylogger_AA_JavaSortDemo.log
```

   Then start a live tail:

```
tail -f $IMPACT_HOME/logs/NCI_policylogger_AA_JavaSortDemo.log
```

Leave this terminal open — output will appear here as soon as the policy runs.

7. Back in the Impact GUI, with `AA_JavaSortDemo` open in the policy editor, click the green **Run** icon in the toolbar to execute the policy immediately.

   > **Note — the Run icon is a useful testing tool.** Any policy can be run on demand this way, without attaching it to a service. The Run dialog also lets you supply parameter values before execution, which is convenient for testing policies that read named parameters. If you are developing a policy that will eventually be driven by an OMNIbusEventReader, you can stub out the event at the top of the policy code — create an `EventContainer` object explicitly and populate its fields — then use Run to test the logic before the event reader is wired up.

   Confirm the **Policy executed successfully** dialog appears, then click **Close**.

8. Switch back to the terminal. You should see two log entries appear:

   ```
   AA_JavaSortDemo: before sort: [zipper, apple, carrot]
   AA_JavaSortDemo: after sort: [apple, carrot, zipper]
   ```

   Confirm that the array is in alphabetical order after the `JavaCall`.

Validation The exercise is successful when all of the following are true:

- The `AA_JavaSortDemo` policy is saved and the syntax check passes.
- The policy log shows both the before and after arrays, with the after array in alphabetical order.
- You can describe the four arguments of `JavaCall` and explain when to pass `null` as the second argument.
- You can explain the difference between a static method call and an instance method call in this context.

Expected result You have demonstrated that IPL can bridge directly into the Java runtime. The `JavaCall` pattern — class name, instance (or null), method name, argument array — works for any accessible Java method. In production, this is used to invoke custom utility classes (string parsers, encryption helpers, web service clients) from within Impact policy code without requiring a separate integration layer.

9. Using the Eval function

The `Eval` function evaluates an IPL filter expression against an event or object at runtime, returning `true` if the object matches the expression and `false` if it does not. This inverts the normal query direction: instead of querying a data source for records that match a known filter, you query for *filter expressions* and test the current event against each one.

The practical effect is powerful: you can drive policy behavior entirely from a database table. Adding a new row to the table adds a new action rule — no policy edits required. This makes `Eval`-based policies suitable for environments where the rules change frequently or need to be managed by operators rather than developers.

This section builds a data-driven event action policy. A `TAKEACTION` table in the existing DB2 CMDB database holds filter expressions and associated action types. For each incoming event, the policy loops through the table, evaluates the event against each filter, and takes the corresponding action if a match is found.

9.1 Prepare the data source

Objective In this exercise, you will create the `TAKEACTION` table in the existing DB2 CMDB database, create a matching Impact data type, and populate the table with two test rules. By the end of the exercise, you will have a data-driven rule store that the escalation policy can query.

Prerequisites

- Completion of sections 4 through 6.
- The `CMDB` DB2 DSA is configured and connected.
- You are logged in to the Impact GUI and have SSH access to the lab server.

Background The `TAKEACTION` table stores one row per action rule. Each row contains:

Column	Type	Purpose
ACTID	INTEGER	Primary key — uniquely identifies the rule
FILTERSTAT	VARCHAR(255)	An IPL filter expression evaluated against each event
ACTION	VARCHAR(32)	What to do when the filter matches (`email` or `suppress`)
PARAMS	VARCHAR(255)	Action-specific parameters (see below)

For the `email` action, PARAMS contains the recipient address and email subject separated by a `|` character:

`oncall@example.com|Alert on @Node: @Summary`

For the `suppress` action, PARAMS contains the integer value to write to the `SuppressEscl` field (e.g. 4 for suppressed).

Tasks

1. SSH into the lab server and switch to the DB2 instance user:

```
su - db2inst1
```

2. Create the `TAKEACTION` table in the `CMDB` database:

```
db2 connect to CMDB
db2 "CREATE TABLE TAKEACTION (
    ACTID     INTEGER NOT NULL PRIMARY KEY,
    FILTERSTAT VARCHAR(255),
    ACTION    VARCHAR(32),
    PARAMS    VARCHAR(255)
)"
db2 connect reset
```

   You should see `DB20000I  The SQL command completed successfully.` after the `CREATE TABLE` statement.

3. Insert two test rules:

```
db2 connect to CMDB
db2 "INSERT INTO TAKEACTION VALUES (1,
    '(Node=\"router-lon-01\") AND (Severity=5)',
    'email',
    'oncall@example.com|Critical alert on router-lon-01')"
db2 "INSERT INTO TAKEACTION VALUES (2,
    '(Node=\"switch-nyc-01\") AND (Severity=3)',
    'suppress',
    '4')"
db2 connect reset
exit
```

   Rule 1 sends an email when `router-lon-01` reaches severity 5. Rule 2 suppresses events from `switch-nyc-01` at severity 3 by setting `SuppressEscl = 4`.

4. In the Impact GUI, create a data type for the new table:
   - Select the **Data Model** tab.
   - Locate the `CMDB` DSA in the left-hand tree, right-click it, and select **New Data Type**.
   - Set **Data Type Name** to `TAKEACTION`.
   - Set **Table** to `TAKEACTION`.
   - Click on the **Refresh Fields** button
   - Mark `ACTID` as the **Key** field.
   - Click **Save**.

Validation The exercise is successful when all of the following are true:

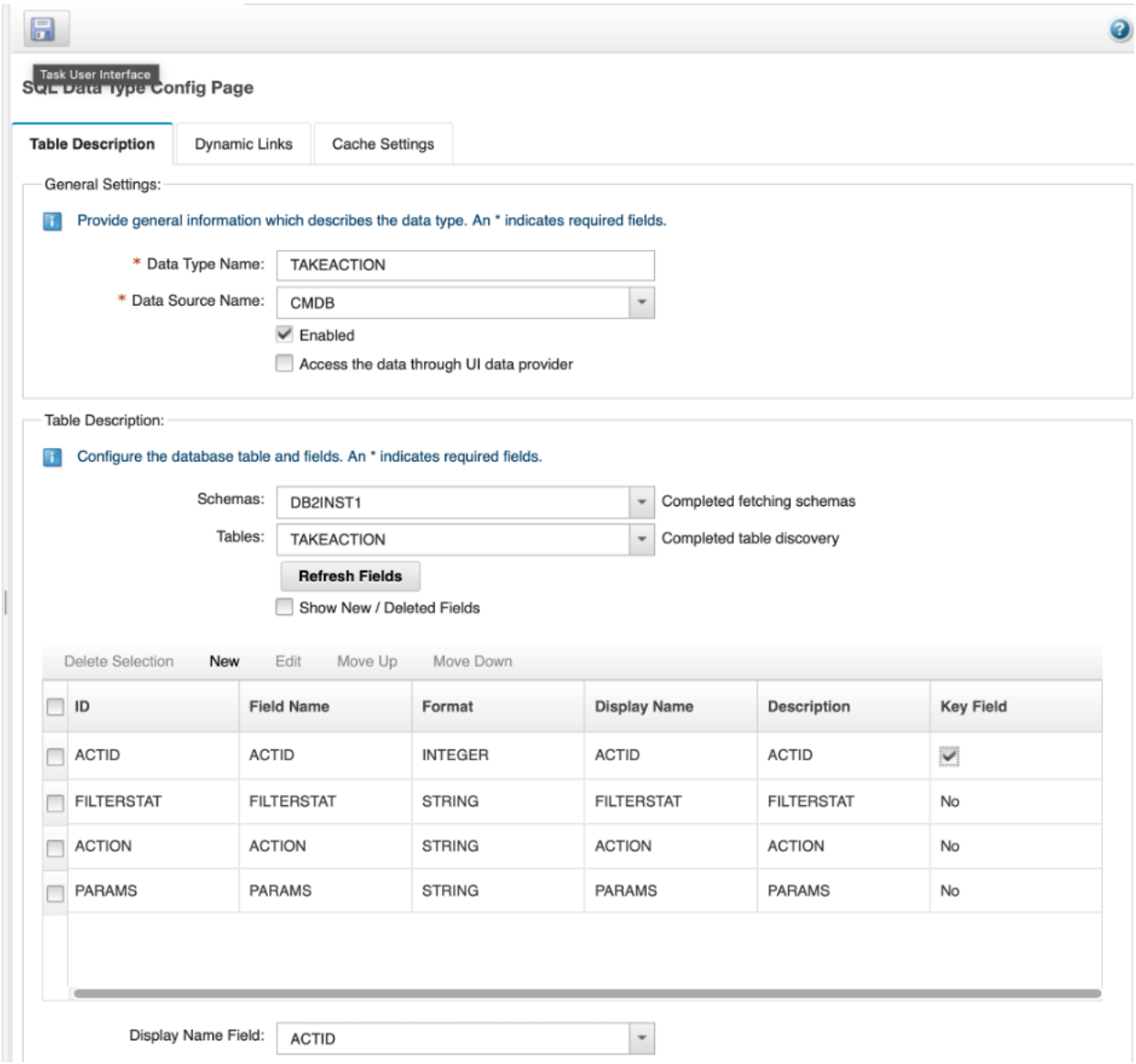


Figure 66: The completed TAKEACTION data type configuration in the Impact Data Model tab, showing the four discovered fields with ACTID marked as the key field.

- The TAKEACTION table exists in the CMDB database with four columns.
- Both rows are present in the table.
- The TAKEACTION data type is visible in the Impact Data Model under the CMDB DSA.
- You can explain why the filter expressions in FILTERSTAT use escaped double quotes (\") when inserted from the DB2 command line.

Expected result The TAKEACTION table is ready to use as a rule store. The next exercise writes the policy that queries this table and uses Eval to match each rule against the incoming event.

9.2 Write the Eval-based action policy

Objective In this exercise, you will write a policy that queries the TAKEACTION table, loops through every row, and evaluates the FILTERSTAT expression against the current event using Eval. When a match is found, the policy executes the corresponding action — either sending an email or suppressing the event. By the end of the exercise, you will understand how Eval inverts the normal query direction and why this pattern is useful for data-driven automation.

Prerequisites

- Completion of section 9.1.
- The TAKEACTION data type is configured in the Impact Data Model.
- The NotifyFlag integer field has been added to alerts.status and the EVENTS data type has been refreshed in Impact. Complete both steps now:

  1. Add the column to the ObjectServer:

  ```
  $OMNIHOME/bin/nco_sql -server EVENTS -user root -password netcool << 'EOF'
  ALTER TABLE alerts.status ADD COLUMN NotifyFlag INTEGER;
  go
  EOF
  ```

  You should see `0 rows affected.` confirming the column was added.

  2. Refresh the EVENTS data type in Impact:

  - In the Impact GUI, select the **Data Model** tab.
  - Locate the EVENTS data type in the left-hand tree under the defaultobjectserver data source and double-click on it to bring up its configuration
  - In the **Table Description** section of the data type configuration page, click **Refresh Fields**.
  - Confirm that NotifyFlag now appears in the field list.
  - Click **Save** and close the EVENTS configuration tab.

Background

EmailSender configuration In a production environment, the email action path would call SendEmail via a configured **EmailSender** service in Impact. To set one up, you would go to **Services → New Service → EmailSender** and supply the SMTP host, port, sender address, and credentials. In this lab the SMTP infrastructure is not available, so SendEmail is commented out in the policy below and replaced with a Log statement that records what the call *would* have done. The functional behavior of the Eval loop, the Split, and the action dispatch is identical — only the final delivery step is simulated.

```
Eval(filterExpression, objectToEvaluate)
```

How Eval works Eval takes a string containing an IPL filter expression and an object (typically EventContainer or a row returned by GetByFilter) and returns true if the object satisfies the expression, false otherwise.

The filter expressions stored in TAKEACTION.FILTERSTAT use the same syntax as OMNIbusEventReader event mappings — field names, comparison operators, AND, OR, LIKE, etc. This means the same filter language you use when configuring services can be driven from a database row.

Why this pattern scales Without `Eval`, adding a new action rule means editing a policy — checking the code into source control, testing, deploying. With `Eval`, adding a rule means inserting a row into a table. The policy itself never changes. This separation of logic (the policy) from configuration (the table) is one of the most powerful patterns in Impact automation.

Tasks

1. Select the **Policies** tab from the top navigation bar. In the toolbar, click the **dropdown arrow** next to the New Policy icon and select **IPL Policy**.
2. Click the **Save** icon, enter `AA_EvalActionPolicy` in the **Policy Name** field, and click **Save**.
3. Enter the following policy code in the editor:

```
/*
 * Policy: AA_EvalActionPolicy
 * Purpose: Query the TAKEACTION rule table and evaluate the current event
 *          against each filter expression using Eval(). When a match is found,
 *          execute the corresponding action (email or suppress).
 * Trigger: OMNIbusEventReader (event-driven).
 */

/* Query all rows from the TAKEACTION table. */

Actions = GetByFilter("TAKEACTION", "ACTID > 0", False);
numActions = Length(Actions);

Log("AA_EvalActionPolicy: " + numActions + " rule(s) loaded from TAKEACTION");

i = 0;

while (i < numActions) {

    /* Evaluate the current event against this row's filter expression. */

    if (Eval(Actions[i].FILTERSTAT, EventContainer)) {

        Log("AA_EvalActionPolicy: rule " + Actions[i].ACTID
            + " matched (action=" + Actions[i].ACTION + ") for Node=" + EventContainer.Node);

        if (Actions[i].ACTION == "email") {

            /* PARAMS format: "recipient|subject"
             * Split on | to get the two parts. */

            spParams = Split(Actions[i].PARAMS, "|");
            Contact = spParams[0];
            Subject = spParams[1];
            Message = EventContainer.Summary;

            EventContainer.NotifyFlag = 1;
            EventContainer.JournalEntry = "Emailed per rule " + Actions[i].ACTID
                + " (" + Contact + ")";

            /* SendEmail is commented out — SMTP not available in this lab.
             * In production: SendEmail(null, Contact, Subject, Message, null, false); */

            Log("AA_EvalActionPolicy: [SIMULATED] email to " + Contact
                + " | subject: " + Subject + " | body: " + Message);

        } elseif (Actions[i].ACTION == "suppress") {

            /* PARAMS contains the integer value for SuppressEscl. */
```

```
            EventContainer.SuppressEscl = Int(Actions[i].PARAMS);
            EventContainer.NotifyFlag = 1;
            EventContainer.JournalEntry = "Suppressed per rule " + Actions[i].ACTID
                + " (SuppressEscl=" + Actions[i].PARAMS + ")";

            Log("AA_EvalActionPolicy: SuppressEscl set to " + Actions[i].PARAMS
                + " for Node=" + EventContainer.Node);
        }
    }

    i = i + 1;
}

/* Commit any field changes back to the ObjectServer. */

ReturnEvent(EventContainer);
```

4. Click the **Syntax Check** icon. Confirm the check passes.
5. Click the **Save** icon to save your policy.

Validation The exercise is successful when all of the following are true:

- The `AA_EvalActionPolicy` policy is saved and the syntax check passes.
- You can trace through the policy and explain what happens for each of the two rows in the `TAKEACTION` table.
- You can explain what `Eval` returns for an event that does not match any row.
- You can explain why `ReturnEvent` is placed at the very end of the policy rather than inside each action branch.
- You can describe how to add a new action rule without touching the policy code.

Expected result You have a fully data-driven action policy. Rules live in the database, not in the policy. Adding, changing, or removing rules is as simple as modifying rows in the `TAKEACTION` table.

9.3 Create the service and validate

Objective In this exercise, you will create an OMNIbusEventReader service that triggers `AA_EvalActionPolicy` on incoming events, insert test alerts that match each rule, and confirm that the email and suppress actions fire correctly.

Prerequisites

- Completion of section 9.2.
- The `AA_EvalActionPolicy` policy is saved.
- You have viewed the default EmailSender service in Impact. Take a moment to do this now:
  - Select the **Services** tab and locate the **EmailSender** service in the service list.
  - Double-click it to open its configuration. You will see fields for SMTP host, port, sender address, and SSL configuration — the setup is straightforward.
  - Note that the service is stopped and leave it that way. In a production environment you would fill in the SMTP details and start it, and the `SendEmail` call in `AA_EvalActionPolicy` would deliver real email. In this lab, SMTP is not available, so the call remains commented out — but the policy code shows exactly what a live implementation would look like.
  - Close the EmailSender configuration without making any changes.

Tasks

1. Select the **Services** tab. Click the **dropdown arrow** next to the New Service icon and select **OMNIbusEventReader**.
2. Configure the service:
   - **Service Name**: `EvalActionEventReader`
   - **ObjectServer DSA**: `defaultobjectserver`

- **Event Mapping filter**: `NotifyFlag = 0`
- **Policy**: `AA_EvalActionPolicy`
- **Active**: checked

3. Click **Save**, then right-click the service and select **Start**. Confirm the state changes to **Running**.
4. Switch to the OMNIbus Web GUI in your private/incognito browser. Select the **ImpactLab Events** filter and the **SyslogdView** view. Select all existing events and delete them so the list is clear before inserting the test alerts.
5. Insert a severity 5 test alert for `router-lon-01` — this should match rule 1 and trigger the email action:

```
$OMNIHOME/bin/nco_sql -server EVENTS -user root -password netcool << 'EOF'
INSERT INTO alerts.status (Node, Summary, AlertKey, AlertGroup, Severity, Type, Identifier,
 ↪  Manager, OwnerUID)
VALUES ('router-lon-01', 'Critical interface failure', 'eval-test-01', 'TrainingAlerts', 5,
 ↪  1, 'router-lon-01:eval-test-01:TrainingAlerts', 'ImpactLab', 65534);
go
EOF
```

6. Insert a severity 3 test alert for `switch-nyc-01` — this should match rule 2 and set `SuppressEscl = 4`:

```
$OMNIHOME/bin/nco_sql -server EVENTS -user root -password netcool << 'EOF'
INSERT INTO alerts.status (Node, Summary, AlertKey, AlertGroup, Severity, Type, Identifier,
 ↪  Manager, OwnerUID)
VALUES ('switch-nyc-01', 'Minor link degradation', 'eval-test-02', 'TrainingAlerts', 3, 1,
 ↪  'switch-nyc-01:eval-test-02:TrainingAlerts', 'ImpactLab', 65534);
go
EOF
```

7. Insert a severity 3 test alert for `server-fra-01` — this should not match any rule and no action should be taken:

```
$OMNIHOME/bin/nco_sql -server EVENTS -user root -password netcool << 'EOF'
INSERT INTO alerts.status (Node, Summary, AlertKey, AlertGroup, Severity, Type, Identifier,
 ↪  Manager, OwnerUID)
VALUES ('server-fra-01', 'Disk usage warning', 'eval-test-03', 'TrainingAlerts', 3, 1,
 ↪  'server-fra-01:eval-test-03:TrainingAlerts', 'ImpactLab', 65534);
go
EOF
```

8. Check the policy log to confirm the rules fired as expected:

```
tail -30 $IMPACT_HOME/logs/NCI_policylogger_AA_EvalActionPolicy.log
```

   You should see:

   - A log entry for `router-lon-01` showing `action=email` matched and the email was sent.
   - A log entry for `switch-nyc-01` showing `action=suppress` matched and `SuppressEscl` was set.
   - A log entry for each alert showing `2 rule(s) loaded from TAKEACTION`.
   - No match log entries for `server-fra-01`.

9. In the OMNIbus Web GUI, locate the `switch-nyc-01` test alert. Double-click it and confirm `SuppressEscl = Suppressed` in the event details.
10. Add a new rule to the `TAKEACTION` table — no policy changes needed. First, switch to the DB2 instance user:

```
su - db2inst1
```

11. Insert the new rule and exit:

```
db2 connect to CMDB
db2 "INSERT INTO TAKEACTION VALUES (3,
    '(Severity >= 4) AND (Node LIKE \"server-fra.*\")',
    'email',
    'dba@example.com|High severity alert on Frankfurt server')"
db2 connect reset
exit
```

12. Insert a fresh severity 4 test alert for `server-fra-01` — this should now match rule 3 and trigger the email action. Note the distinct `AlertKey` and `Identifier` (`eval-test-04`) to ensure this arrives as a new event rather than deduplicating against the earlier severity 3 insert:

```
$OMNIHOME/bin/nco_sql -server EVENTS -user root -password netcool << 'EOF'
INSERT INTO alerts.status (Node, Summary, AlertKey, AlertGroup, Severity, Type, Identifier,
 ↪  Manager, OwnerUID)
VALUES ('server-fra-01', 'High disk utilisation', 'eval-test-04', 'TrainingAlerts', 4, 1,
 ↪  'server-fra-01:eval-test-04:TrainingAlerts', 'ImpactLab', 65534);
go
EOF
```

    Check the policy log and confirm the email action for rule 3 appears — with no policy edits made:

```
tail -20 $IMPACT_HOME/logs/NCI_policylogger_AA_EvalActionPolicy.log
```

13. In the OMNIbus Web GUI, check the journal entries on the actioned alerts to confirm the policy wrote them correctly:
    - Double-click the `router-lon-01` alert and open the **Journal** tab. You should see an entry reading `Emailed per rule 1 (oncall@example.com)`.
    - Double-click the `switch-nyc-01` alert and open the **Journal** tab. You should see an entry reading `Suppressed per rule 2 (SuppressEscl=4)`.
    - Double-click the `server-fra-01` (severity 4) alert and open the **Journal** tab. You should see an entry reading `Emailed per rule 3 (dba@example.com)`.

Validation The exercise is successful when all of the following are true:

- `router-lon-01` (sev 5) triggered the email action.
- `switch-nyc-01` (sev 3) had `SuppressEscl` set to 4.
- `server-fra-01` (sev 3) produced no action entries in the policy log.
- After adding rule 3, a severity 4 alert from `server-fra-01` triggered the email action without any policy edits.
- Journal entries are visible on all three actioned alerts, correctly identifying the rule and action taken.
- You can explain how `Eval` enables this data-driven approach.

Expected result You have built a data-driven action policy where all rules live in a database table. The policy code handles the loop, the `Eval` call, and the action dispatch — but the *rules themselves* are data. This is one of the most flexible patterns in Impact: it lets operators or administrators add, modify, or retire automation rules without involving a policy developer or a deployment cycle.

Lab completion summary

Congratulations on completing the Netcool Impact lab guide. Across eleven sections you have built a fully working Impact automation environment from scratch — from configuring data source adapters on day one to managing user access on the last. This page summarises what you covered in each section and the skills you have developed along the way.

Section 1 — Impact fundamentals and UI navigation

You learned what IBM Netcool Impact is and how it fits into the OMNIbus architecture. You explored the Impact GUI, navigated the main tabs — **Policies**, **Services**, **Data Model**, **Operator View**, and **Maintenance Window** — and established the lab environment conventions that carried through the rest of the guide.

Section 2 — Data source adapters

You configured the two data sources used throughout the lab: the **ObjectServer DSA** (`defaultobjectserver`) connecting Impact to the OMNIbus EVENTS ObjectServer, and the **DB2 CMDB DSA** (`CMDB`) connecting Impact to the device inventory database. You created the `DEVICE_INVENTORY` data type and validated both connections.

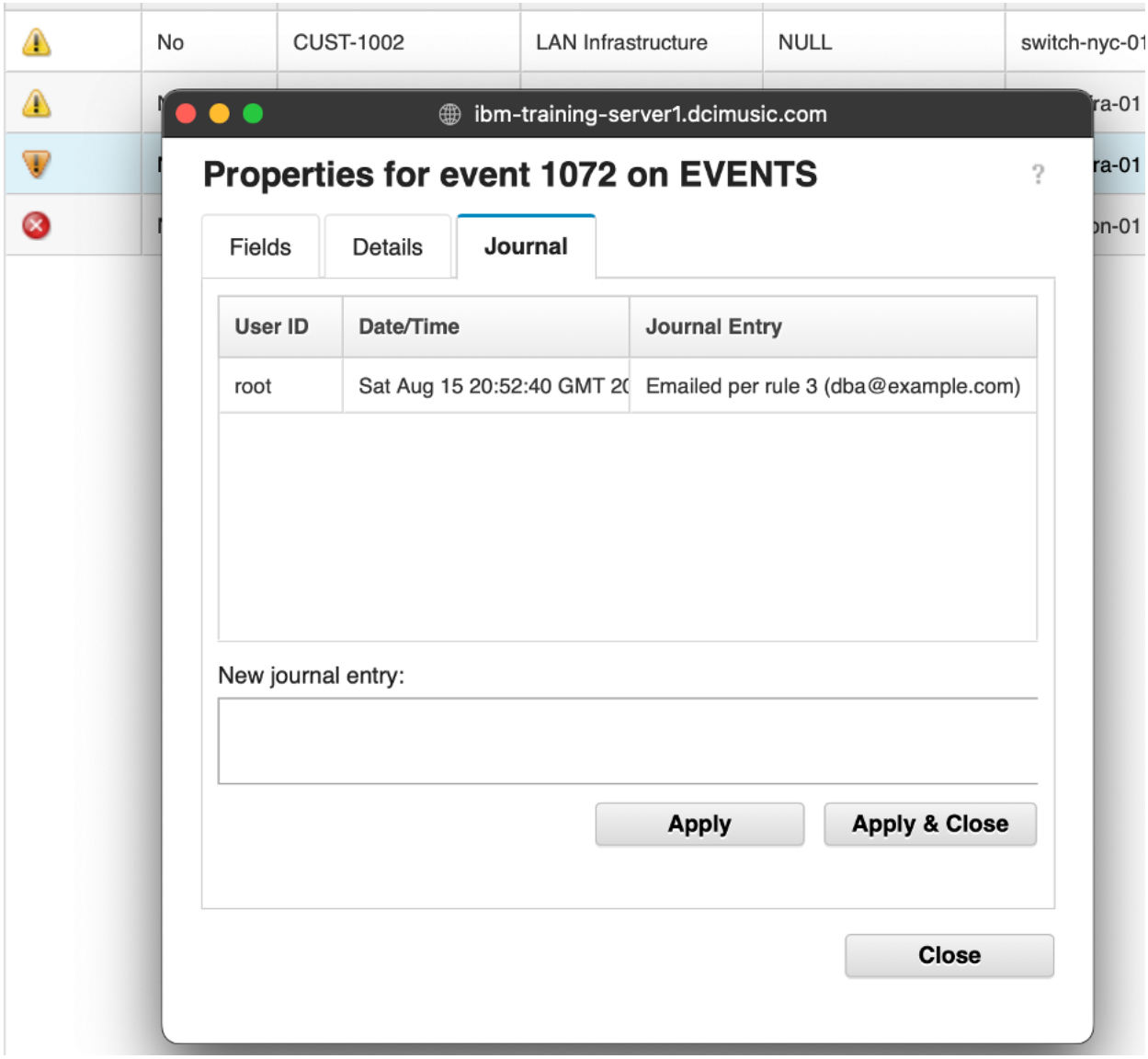


Figure 67: Journal tab of the router-lon-01 alert showing the ‘Emailed per rule 1’ journal entry written by AA_EvalActionPolicy.

Section 3 — Impact policy language

You learned the IPL syntax fundamentals: variables, conditionals, loops, `GetByFilter`, `Log`, direct field assignment, `ReturnEvent`, and `DirectSQL`. You worked through the key patterns that appear in every policy in the lab — reading event fields from `EventContainer`, querying external data, writing results back to the ObjectServer, and understanding when to use `ReturnEvent` versus `DirectSQL`.

Section 4 — Event enrichment

You wrote your first production-style policy, `AA_EnrichAlert`, which queries the `DEVICE_INVENTORY` CMDB table and writes customer, service, and location data back to incoming alerts. You configured the `EnrichmentEventReader` OMNIbusEventReader service to trigger the policy on new events, and validated the full enrichment pipeline — both the found path (`ImpactFlag = 1`) and the not-found path (`ImpactFlag = 2`).

Section 5 — Scheduled automation

You extended the enrichment pipeline to handle events that could not be enriched on first attempt — nodes not yet in the CMDB. The `AA_DeferredEnrichment` policy and its `DeferredEnrichmentActivator` PolicyActivator service poll every 60 seconds, retry enrichment for `ImpactFlag = 2` events, and enforce a retry limit using the `Poll` counter field. You validated the retry loop and confirmed that alerts exceeding the retry limit are silently dropped from further processing.

Section 6 — Event escalation

You added a `SERVICE_LEVEL` dimension to the enrichment pipeline and wrote `AA_EscalatePlatinum`, a time-based escalation policy that progressively raises the severity of unacknowledged Platinum-tier alerts. You learned why age-based escalation requires a PolicyActivator rather than an event reader, how to write back to the ObjectServer using `DirectSQL` from a scheduled policy, and how to use the `Flash` field to drive visual notifications in the Web GUI.

Section 7 — Automation and hibernation

You wrote `AA_PingVerify`, a policy that uses `JRExecAction` to ping a node and confirm whether a failure is real before raising severity. You used the **Hibernate** function to pause policy execution and re-evaluate after a delay, and extended the `EnrichmentEventReader` service with a second mapping row to trigger the ping policy selectively. You validated both the reachable-node path (event clears) and the unreachable-node path (severity is raised after hibernation).

Section 8 — Making a Java call

You learned how to call Java methods directly from IPL using `JavaCall`. The exercise demonstrated the four-argument calling convention — class name, object instance, method name, argument array — using `java.util.Arrays.sort()` as a clear, verifiable example. You also used the policy editor's **Run** icon to execute a policy on demand without attaching it to a service, establishing a useful testing pattern.

Section 9 — Using the Eval function

You built a fully data-driven action policy using `Eval`. The `AA_EvalActionPolicy` queries a `TAKEACTION` rule table in the CMDB, evaluates the current event against each stored filter expression at runtime, and dispatches the corresponding action — email or suppression — without any hardcoded conditions in the policy code itself. You demonstrated that adding a new automation rule requires only a database insert, with no policy changes or redeployment.

Section 10 — Organizing artifacts using projects

You used the Impact **project selector** to create `ImpactLabProject` and populate it with all of the lab's data sources, data types, policies, and services. You learned how projects scope the Policies and Services views to show only the relevant artifacts. You then used `nci_export` to export the project from the command line, deliberately deleted two artifacts to simulate a configuration loss, and used `nci_import` to restore them — directly demonstrating the rollback use case.

Section 11 — Granting users access to Impact

You learned how Impact's role-based access control model works. You observed the blank workspace presented to an authenticated but unmapped user, granted the `ncoadmin` account the Maintenance Window Manager roles to give it scoped access, and then granted it the `impactAdminUser` role to give it full administrative access. You understand the distinction between authentication (handled by the ObjectServer in this lab) and authorization (controlled by role mappings applied with `mapRoles.sh`).

What you built

Over the course of the lab you assembled a complete, layered Impact automation environment:

Component	Purpose
`AA_EnrichAlert` + `EnrichmentEventReader`	Real-time enrichment of incoming alerts from the CMDB
`AA_DeferredEnrichment` + `DeferredEnrichmentActivator`	Scheduled retry enrichment with a poll-count limit
`AA_EscalatePlatinum` + `EscalationActivator`	Time-based severity escalation for Platinum-tier alerts
`AA_PingVerify` + `HibernatingPolicyActivator`	Ping-based verification with hibernation before severity raise
`AA_JavaSortDemo`	Demonstration of Java runtime integration via `JavaCall`
`AA_EvalActionPolicy` + `EvalActionEventReader`	Data-driven action dispatch using `Eval` and a CMDB rule table
`ImpactLabProject`	Project packaging all artifacts for export, import, and recovery

Each layer built on the one before it. Real-time enrichment feeds deferred enrichment. Enrichment populates the `ServiceLevel` field that escalation depends on. The hibernation policy reuses the same event pipeline. The Eval policy generalizes the dispatch pattern used throughout.

Where to go next

The patterns you practised here — enrichment, deferred retry, time-based escalation, Java integration, data-driven dispatch, and project-based packaging — are the same patterns used in production Impact deployments. From here you can:

- Explore the **Operator View** tab to build custom event dashboards.
- Configure a real **EmailSender** service and remove the `SendEmail` comment from `AA_EvalActionPolicy`.
- Extend the escalation policy to notify an on-call engineer when an alert reaches severity 5.
- Use `nci_export` on a schedule to maintain regular configuration backups.
- Apply role mappings to additional users or explore LDAP and Keycloak/OIDC authentication for production deployments.